



Computer Vision and Image Processing

Lecture 08

Thresholding

Thresholding is a fundamental segmentation technique used to convert a grayscale image into a binary image. It separates foreground (object) from background based on grey-level intensity.

Thresholding Techniques:

1. Global Thresholding
2. Adaptive (Local) Thresholding
3. Otsu's Method (Automatic Global Thresholding)

Global Thresholding

Global thresholding uses a **single threshold value (T)** for the entire image.

When to use it

- Image has **uniform illumination**
- Background and foreground intensities are clearly separated

How it works

- Choose a threshold T manually or automatically.
- Compare all pixel values to this T .
- Output is binary.

Global Thresholding

Algorithm

- Compute histogram
- Pick a threshold
- Apply binary formula

Example

- If $T = 120$:
Pixels $> 120 \rightarrow$ white (1)
Pixels $\leq 120 \rightarrow$ black (0)

Global Thresholding

Pros

- Very fast
- Simple to implement

Cons

- Fails in uneven/poor lighting
- Sensitive to shadows and highlights

Adaptive (Local) Thresholding

Instead of one global threshold, **each pixel gets its own threshold** based on its **local neighbourhood**.

Useful when:

- There is **non-uniform illumination**
- Background varies across the image
- Shadows or bright regions exist

Adaptive (Local) Thresholding

How it works

Divide the image into small windows (e.g., 15×15). For each window:

Compute a threshold using:

- Mean
- Weighted mean (Gaussian)
- Median

Then threshold that region separately.

Adaptive (Local) Thresholding

Algorithm

- For each pixel at (x, y) :

$$T(x, y) = \text{mean of neighbourhood} - C$$

Pros

- Works well in varying illumination
- Better for document images, medical images

Cons

- More computationally expensive
- Window size selection is critical

Otsu's Method (Automatic Global Thresholding)

Otsu's method, named after its inventor Nobuyuki Otsu, is one of many binarization algorithms.

Otsu's Method works best with Bimodal Histogram.

A **bimodal histogram** is a histogram of image grey levels that contains **two distinct peaks (modes)**. These peaks represent **two dominant intensity groups** in the image, usually corresponding to **foreground** and **background**.

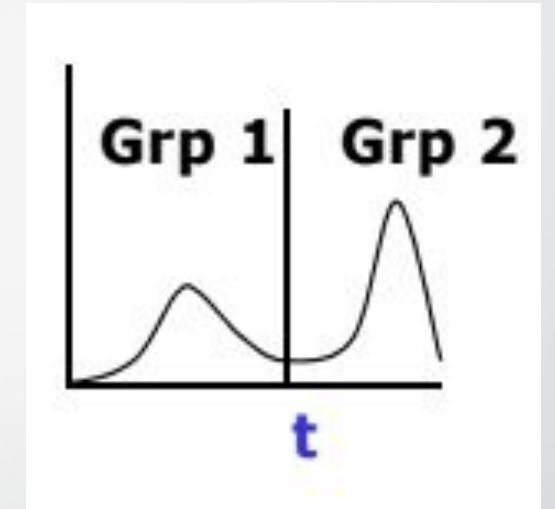
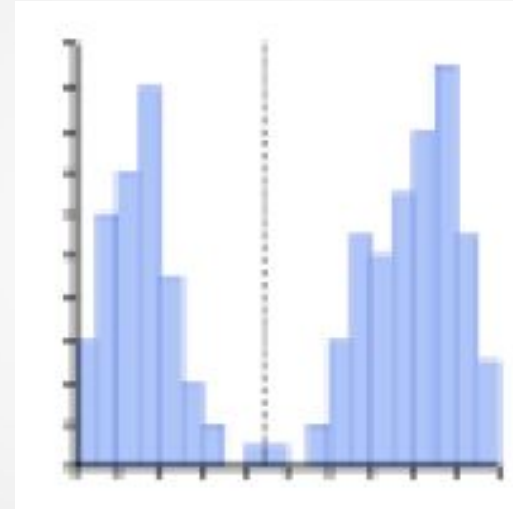
Otsu's Method (Automatic Global Thresholding)

A **bimodal histogram** has:

- Two **major peaks**
- A **valley** between them
- Two **clusters** of pixel intensities

This usually occurs in images where:

- The **foreground object** has intensities very different from the **background**
- The image contains **two primary classes** of brightness



Otsu's Method (Automatic Global Thresholding)

1. Within-Class Variance (How "Pure" are the baskets?)

- Imagine you draw a cutoff line and throw your clothes into the two baskets.
- Now, look *inside* the "Lights" basket. Are they all exactly the same shade of pure white? Probably not. You might have some eggshell, some light gray, and maybe a pale-yellow shirt. The differences between the clothes *inside this single basket* is the **Within-Class Variance**.
- **High Within-Class Variance:** You did a bad job sorting. Your "Lights" basket has dark red shirts in it, and your "Darks" basket has white socks in it. The colors are all over the place.
- **Low Within-Class Variance:** You did a great job. The "Lights" basket is entirely pale colors, and the "Darks" basket is entirely dark colors.

Otsu's Goal: Make the "spread" of colors inside each basket as tiny as possible.

Otsu's Method (Automatic Global Thresholding)

2. Between-Class Variance (How "Different" are the two baskets from each other?)

- Now, step back and look at the two baskets as a whole. Ignore individual socks; just look at the **average color** of Basket 1 and the **average color** of Basket 2.
- **Low Between-Class Variance:** The average color of Basket 1 is medium-gray, and the average color of Basket 2 is slightly-darker-gray. They look too similar. Your cutoff line was in the wrong place.
- **High Between-Class Variance:** The average color of Basket 1 is bright white, and the average color of Basket 2 is pitch black. They are drastically different from each other.
- **Otsu's Goal:** Push the average color of the two baskets as far apart as possible.

Otsu's Method (Automatic Global Thresholding)

Pros

- Fully automatic
- Works well for bimodal images

Cons

- Fails if lighting is uneven
- Fails if histogram is not bimodal (e.g., multimodal or low contrast)

Neighborhoods in Binary Images

- In digital image processing, a **binary image** is an image where each pixel can only have one of two values: typically, **0** (representing the black background) or **1** (representing the white foreground).
- When analyzing these images to find shapes, boundaries, or connected components, computers need a mathematical way to look at a pixel's relationship with the pixels immediately surrounding it. This localized surrounding area is called a pixel's **neighborhood**.
- If we take a target pixel **p** located at coordinates **(x, y)** on an image grid, there are three primary ways to define its neighbors:

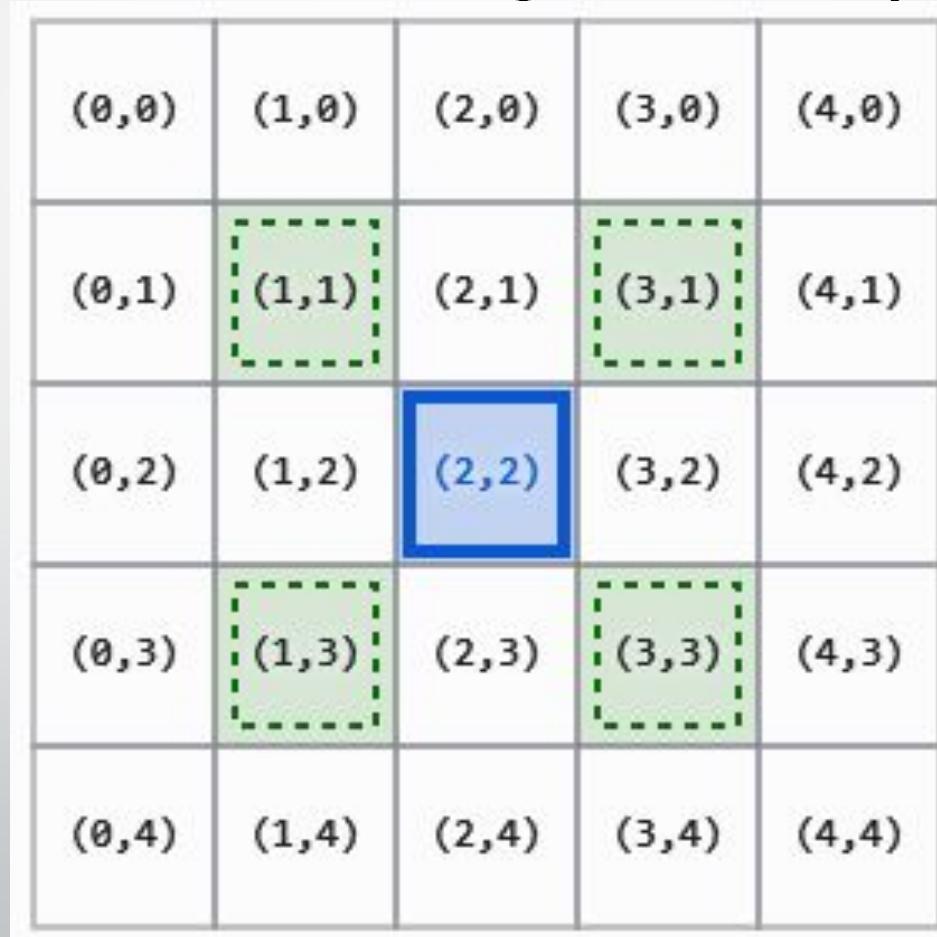
1. The 4-Neighborhood (N4)

- The 4-neighborhood consists only of the pixels that share a direct, flat edge with the center pixel p . These are the pixels directly above, below, to the left, and to the right

$(0,0)$	$(1,0)$	$(2,0)$	$(3,0)$	$(4,0)$
$(0,1)$	$(1,1)$	$(2,1)$	$(3,1)$	$(4,1)$
$(0,2)$	$(1,2)$	$(2,2)$	$(3,2)$	$(4,2)$
$(0,3)$	$(1,3)$	$(2,3)$	$(3,3)$	$(4,3)$
$(0,4)$	$(1,4)$	$(2,4)$	$(3,4)$	$(4,4)$

2. The Diagonal Neighborhood (ND)

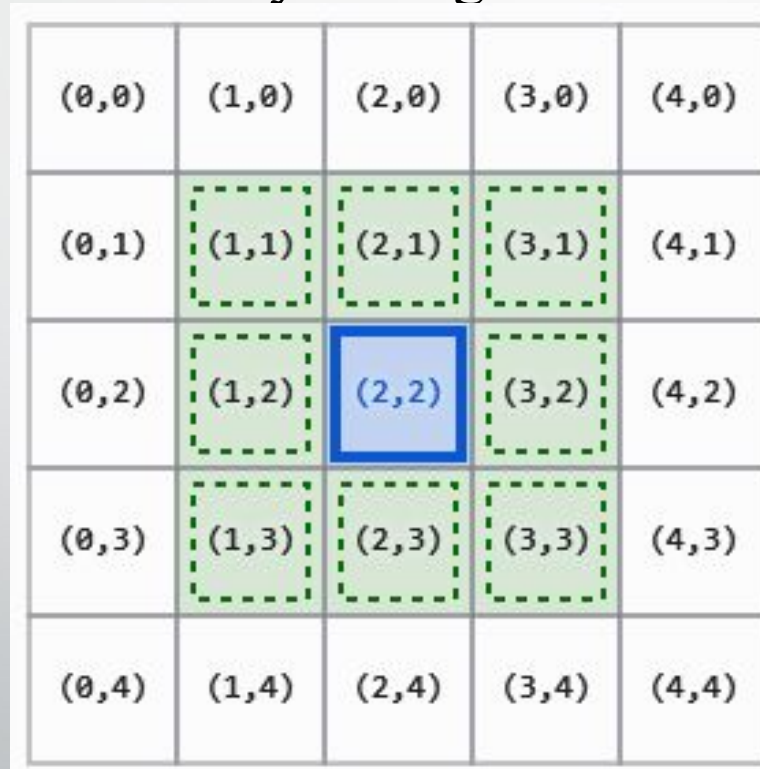
- The diagonal neighborhood consists of the four pixels that only share a corner with the center pixel p .



(0,0)	(1,0)	(2,0)	(3,0)	(4,0)
(0,1)	(1,1)	(2,1)	(3,1)	(4,1)
(0,2)	(1,2)	(2,2)	(3,2)	(4,2)
(0,3)	(1,3)	(2,3)	(3,3)	(4,3)
(0,4)	(1,4)	(2,4)	(3,4)	(4,4)

3. The 8-Neighborhood (N8)

- The 8-neighborhood is simply the combination of the 4-neighborhood and the diagonal neighborhood. It includes every single pixel that touches the center pixel p in any way, whether by an edge or a corner.



(0,0)	(1,0)	(2,0)	(3,0)	(4,0)
(0,1)	(1,1)	(2,1)	(3,1)	(4,1)
(0,2)	(1,2)	(2,2)	(3,2)	(4,2)
(0,3)	(1,3)	(2,3)	(3,3)	(4,3)
(0,4)	(1,4)	(2,4)	(3,4)	(4,4)

Neighborhood Ambiguity

- The concept of **neighborhood ambiguity**—often called the **Connectivity Paradox**—is one of the most famous quirks in digital image processing.
- It happens because a grid of square pixels doesn't behave perfectly like continuous space. In the real world, if you draw a closed circle, it clearly separates the "inside" from the "outside." In a pixel grid, depending on how you define your neighborhoods, that logic can break down.
- Here is how the ambiguity happens, specifically when dealing with diagonal lines.

Neighborhood Ambiguity

- Imagine a simple 2x2 block of pixels where the top-right and bottom-left pixels are **black** (foreground), and the top-left and bottom-right are **white** (background).
- The ambiguity arises when we try to answer a simple question: **Are the black pixels connected to each other? And are the white pixels connected to each other?**

Neighborhood Ambiguity

If we apply the same neighborhood rules to both the foreground and the background, we run into logical paradoxes:

- **Scenario A: Everything is 4-Connected** If we decide that pixels only connect via flat edges (4-neighborhood), then the two black pixels are **not** connected. But simultaneously, the two white pixels are also **not** connected. The black line is broken, but the white space can't flow through it either.
- **Scenario B: Everything is 8-Connected (The Paradox)** If we decide that pixels connect via corners too (8-neighborhood), then the two black pixels **are** connected. They form a diagonal line. However, the two white pixels are *also* connected through the exact same corner point. This means a continuous background line crosses completely through a continuous foreground line without breaking it—a geometric impossibility in normal space.

Neighborhood Ambiguity

The Standard Solution

- To fix this ambiguity and ensure that a closed loop actually traps the background inside it, image processing software usually uses a mixed rule: **We define the foreground objects as 8-connected, and the background space as 4-connected (or vice versa).** By doing this, the black pixels connect diagonally to form a solid boundary, and the white background cannot squeeze through the corners, keeping the "inside" strictly separated from the "outside."



The End



Computer Vision and Image Processing

Lecture 09

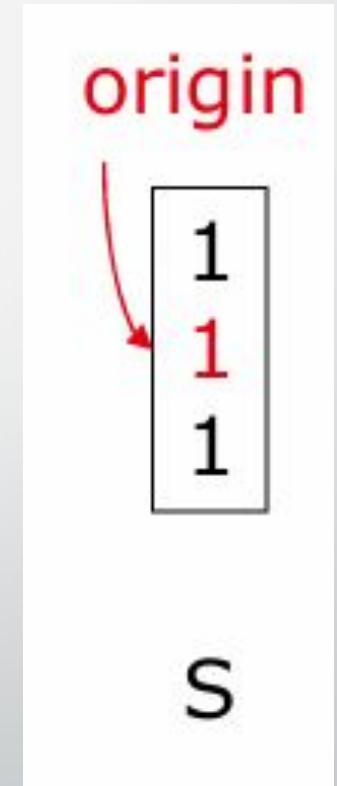
Morphology

- In digital image processing, morphology is a framework of non-linear operations used to modify, extract, or clean up the **shapes and structures** of objects within an image.
- It is most commonly applied to **binary images** (where pixels are either 0 or 1, black or white), but it can also be extended to **grayscale images**.

Morphology

The Structuring Element (SE)

- Morphological operations work by passing a small shape or template called a **Structuring Element** (or kernel) over the input image, much like a convolution matrix in linear filtering.
- The structuring element can be of any size and shape (e.g., a square, a circle, a cross, or a line). As it slides across the image, a logical comparison (like an AND or OR operation) is made between the structuring element and the underlying image pixels to determine the value of the output pixel.



Morphology

1	1	1	1	1
1	1	1	1	1
1	1	1	1	1

(a) BOX(3,5)

	1	1	1	
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
	1	1	1	

(b) DISK(5)

	1	1	1	
1				1
1				1
1				1
	1	1	1	

(c) RING(5)

1	1		
1	1		
1	1	1	1
1	1	1	1

(d)

1	1	1	1	1	1
1		1	1		1
1		1	1		1
1		1	1		1

(e)

1
1
1
1

(f)

Morphology

1. Fundamental Operations:

- a) Dilation
- b) Erosion

2. Composite Operations:

- a) Opening
- b) Closing
- c) Morphological Gradient

3. Top-Hat Transforms:

- a) White Top-Hat (Top-Hat)
- b) Black Top-Hat (Bottom-Hat)

Dilation

Dilation expands the boundaries of foreground objects, effectively making them **larger** and filling in small holes or gaps.

$$I = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

$$SE = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

Dilation

$$I \oplus SE = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{pmatrix}$$

Erosion

Erosion shrinks the boundaries of foreground objects, effectively making them **smaller** and removing small objects or thin structures.

$$I = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

$$SE = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

Erosion

$$I \ominus SE = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Opening

Opening consists of an **Erosion** followed by a **Dilation**.

$$I \circ SE = (I \ominus SE) \oplus SE$$

- **What it does:** It removes small bright objects (noise), breaks thin connections, and smooths sharp peaks from the *inside* of an object.
- **Key characteristic:** It preserves the overall shape and size of larger objects that can completely contain the structuring element.

Closing

Closing consists of a **Dilation** followed by an **Erosion**.

$$I \bullet SE = (I \oplus SE) \ominus SE$$

- **What it does:** It fills in small dark holes, fuses narrow gaps or breaks, and smooths contours from the *outside*.
- **Key characteristic:** It bridges gaps that are smaller than the size of the structuring element.

Morphological Gradient

The **Morphological Gradient** (or simply "Gradient") in image processing is a fundamental operation that calculates the **difference** between the **Dilation** and the **Erosion** of an image. It is often used to highlight the **boundaries** or **edges** of objects within the image.

$$G(I) = (I \oplus SE) - (I \ominus SE)$$

Morphological Gradient

Why It Highlights Edges

- **Dilation** expands the object, making the boundary thicker (the expansion is the region *outside* the original object).
- **Erosion** shrinks the object, making the boundary thinner (the shrinking is the region *inside* the original object).
- When you **subtract** the smaller, eroded image from the larger, dilated image, the only remaining pixels are the ones that were added by dilation and/or removed by erosion. These pixels form a **thickened outline** or **boundary** of the original object.

White Top-Hat Transform (WTH)

The White Top-Hat transform extracts **small, bright objects** or **details** from an image, especially when the background illumination is uneven.

$$\text{WTH}(I) = I - (I \circ SE)$$

Black Top-Hat Transform (BTH)

The Black Top-Hat transform (or Bottom-Hat) extracts **small, dark objects** or **details** from an image.

$$\text{BTH}(I) = (I \bullet SE) - I$$



The End



Computer Vision and Image Processing

Lecture 14

Edge Detection

- An **edge in an image** is a boundary or contour where the **image intensity (brightness or colour) changes sharply** over a short distance.
- Edges are critical features in computer vision and image processing because they represent the important structural information of a scene, such as the **boundaries of objects**, changes in surface orientation, or variations in lighting.
- Edge detection identifies **rapid intensity changes** in an image, which typically correspond to **object boundaries, texture changes, or depth discontinuities**.

Edge Types

Edge Type	Description (Intensity Change)	Derivative Tool
Step Edge	An abrupt change from one intensity level to another. (Ideal)	First Derivative has a sharp peak (maximum magnitude). Second Derivative has a zero-crossing (changes sign) at the edge center.
Ramp Edge	A gradual change in intensity over a short distance. (More realistic)	First Derivative has a peak over the ramp's width. Second Derivative has a zero-crossing at the centre of the ramp.
Roof Edge	A peak or ridge in intensity, often corresponding to lines thinner than the filter size.	First Derivative has two peaks of opposite sign. Second Derivative has a strong minimum (for a positive roof/ridge) at the centre.

Edge Profiles

This is the **plot of intensity values** along a cross-section of the image.

- The **shape** of the plot indicates the type of edge.
- Used to understand how smooth or abrupt the change is.

Example:

- Step edge → sharp vertical jump.
- Ramp edge → slanted line.
- Roof edge → triangular peak.

Edge Operators

These are filters (kernels) that detect changes in intensity.

Main categories:

- **Gradient-based operators** → detect edges by finding *first derivative*.
- **Laplacian-based operators** → detect edges by finding *second derivative*.

Sobel Operator (First-Order Derivative)

The Sobel operator is a gradient-based operator that approximates the first derivative. It uses two 3*3 convolution kernels: **G_x** for the horizontal derivative (detects vertical edges) and **G_y** for the vertical derivative (detects horizontal edges). The center row/column is weighted by 2 for increased smoothing and noise reduction.

Horizontal gradient (G_x):

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Vertical gradient (G_y):

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Sobel Operator (First-Order Derivative)

Magnitude:

$$G = \sqrt{G_x^2 + G_y^2}$$

Direction:

$$\theta = \tan^{-1} \left(\frac{G_y}{G_x} \right)$$

Edge angle:

$$\theta_{edge} = \theta + 90^\circ$$

Sobel Operator (First-Order Derivative)

$$I = \begin{bmatrix} 10 & 10 & 80 \\ 10 & 10 & 80 \\ 10 & 10 & 80 \end{bmatrix}$$

Laplacian (Second Derivative Operator)

The Laplacian detects edges by checking **zero-crossings** of second derivatives.

Common kernel:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Or 8-connected:

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Laplacian (Second Derivative Operator)

Characteristics

- **Positive Laplacian value:** centre pixel is a local intensity **peak** relative to neighbours (second derivative positive).
- **Negative Laplacian value:** centre is a local **valley** (second derivative negative).
- **Zero or near 0:** flat region or balanced neighbourhood.
- **Zero-crossings between adjacent pixels:** indicate possible edges (where Laplacian changes sign).

Because Laplacian measures the **second derivative**, it responds strongly where the first derivative changes quickly — i.e., at edges. But the second derivative is **very sensitive to noise**, so in practice you usually:

1. Smooth the image with a Gaussian (to reduce noise), then
2. Apply the Laplacian (the combined operator is called **LoG — Laplacian of Gaussian**), then
3. Detect zero-crossings (possibly with a threshold on magnitude).

Laplacian (Second Derivative Operator)

$$I = \begin{pmatrix} 10 & 10 & 10 & \mathbf{200} & 200 \\ 10 & 10 & 10 & \mathbf{200} & 200 \\ 10 & 10 & 10 & \mathbf{200} & 200 \\ 10 & 10 & 10 & \mathbf{200} & 200 \\ 10 & 10 & 10 & \mathbf{200} & 200 \end{pmatrix}$$

Image Smoothing

- Image smoothing removes high-frequency noise, producing a soft/blurred image. It can be done in **spatial domain** or **frequency domain**.
- Image Smoothing can be applied in both **Spatial** and **Frequency** Domain.

Image Smoothing (Spatial Domain)

- Spatial Domain Smoothing Filters:
 1. Averaging (Mean) Filter
 2. Weighted Averaging Filter
 3. Gaussian Smoothing Filter
 4. Median Filter
 5. Bilateral Filter (*edge-preserving*)

Averaging (Mean) Filter

- The averaging filter replaces each pixel with the **average** of its neighbouring pixels.

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

For a pixel (x, y) :

$$g(x, y) = \frac{1}{mn} \sum_{i=-a}^a \sum_{j=-b}^b f(x+i, y+j)$$

Averaging (Mean) Filter

Property	Behaviour
Blurring strength	High (strong smoothing)
Noise removal	Good for random noise
Edge preservation	Poor (edges softened)
Computational cost	Very low

Use Cases

- Removing random noise
- Pre-processing before edge detection
- Reducing details in an image

Causes **strong blurring** and loses fine details.

Weighted Averaging Filter

- Assigns **higher weights** to central pixels and **lower weights** to neighbours.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

This gives more importance to the center pixel → smoother and less blurry than simple averaging.

Weighted Averaging Filter

Property

Behaviour

Blurring

Moderate

Noise removal

Better than simple averaging

Edge preservation

Better than uniform averaging

Smoothness

Higher

Use Cases

- Natural image smoothing
- Removing mild noise
- Pre-processing for computer vision tasks

Gaussian Smoothing Filter

- Uses a kernel based on the **2D Gaussian distribution**:

$$G(x, y) = e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Example (Gaussian kernel, $\sigma = 1$):

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

(Same pattern as weighted average but mathematically derived.)

Gaussian Smoothing Filter

Property	Behaviour
Blurring	Moderate
Ringing artifacts	None
Edge preservation	Good
Noise reduction	Excellent
Smoothness	Very natural

Use Cases

- Natural image denoising
- Removing Gaussian noise
- Pre-processing for:
 - Face detection
 - Object detection
 - Medical imaging
- Suitable where visual quality matters

Gaussian filter produces the **most visually pleasant smoothing** in spatial domain.

Median Filter (Nonlinear)

- Replaces each pixel with the **median** of its neighbours values.

Neighborhood:

10 200 15

18 20 22

21 16 19

Sorted:

10, 15, 16, 18, 19, 20, 21, 22, 200

Median = 19

Median Filter (Nonlinear)

Property	Behaviour
Blurring	Very low
Edge preservation	Excellent
Type	Non-linear

Use Cases

- Salt-and-pepper noise removal
- Preserving sharp edges (e.g., medical images)
- Removing outliers in sensor data

Median filter is the **best filter** for salt-and-pepper noise.

Bilateral Filter (Advanced Edge-preserving Smoothing)

- Performs smoothing based on:

1. Spatial closeness

2. Pixel intensity similarity

$$g(x, y) = \frac{1}{W} \sum_i \sum_j f(i, j) \cdot e^{-\frac{(x-i)^2 + (y-j)^2}{2\sigma_s^2}} \cdot e^{-\frac{(f(x, y) - f(i, j))^2}{2\sigma_r^2}}$$

Bilateral Filter (Advanced Edge-preserving Smoothing)

Property	Behaviour
Edge preservation	Best among smoothing filters
Blurring	Very low
Noise removal	High
Complexity	High (slow)

Use Cases

- Face smoothing
- Image beautification
- Removing noise while preserving edges
- Depth maps and 3D images

Often used in **computer photography** (e.g., skin smoothing in smartphone cameras).

Comparison Summary (Spatial Domain Smoothing)

Filter	Type	Edge Preservation	Blurring	Best For
Mean (Averaging)	Linear	Poor	Strong	Basic noise reduction
Weighted Mean	Linear	Medium	Moderate	Natural smoothing
Gaussian	Linear	Good	Moderate	High-quality smoothing
Median	Non-linear	Excellent	Low	Salt-and-pepper noise
Bilateral	Non-linear	Excellent	Very low	Edge-preserving denoising

Comparison Summary (Spatial Domain Smoothing)

Which Spatial Filter Should You Use?

- For **random noise reduction** → **Averaging / Gaussian**
- For **natural smooth image** → **Gaussian**
- For **salt-and-pepper noise** → **Median**
- For **smoothing without losing edges** → **Bilateral**
- For **face/skin smoothing** → **Bilateral Filter**

Image Smoothing (Frequency Domain)

- The most common frequency-domain smoothing filters are:
 - 1. Ideal Low-Pass Filter (ILPF)**
 - 2. Butterworth Low-Pass Filter (BLPF)**
 - 3. Gaussian Low-Pass Filter (GLPF)**
- Each filter reduces high-frequency components differently — this affects image quality, ringing effects, and smoothness.

Ideal Low-Pass Filter (ILPF)

- An ideal LPF passes **all frequencies inside a radius D_0** and completely removes all frequencies outside it.

Mathematically:

$$H(u, v) = \begin{cases} 1, & D(u, v) \leq D_0 \\ 0, & D(u, v) > D_0 \end{cases}$$

Where

$$D(u, v) = \sqrt{(u - M/2)^2 + (v - N/2)^2}$$

Ideal Low-Pass Filter (ILPF)

Property

Cutoff shape

Passband

Stopband

Transition band

Artifacts

Smoothness

Complexity

Behaviour

Perfect circle (sharp boundary)

Frequencies inside D_0 fully preserved

Frequencies outside D_0 completely removed

None ($0 \rightarrow 1$ abruptly)

Strong **ringing** (Gibbs effect) in spatial domain

Poor (because of sharp cutoff)

Very simple

Ideal Low-Pass Filter (ILPF)

Applications

Synthetic images
Removing small details
Educational purposes
Demonstrating frequency cutoffs

Why reason

No noise in edges, acceptable ringing
Strong high-frequency removal
Easy to visualize and compute
Clear, abrupt frequency blocking

Not good for natural images, because the ringing artifacts distort edges.

Butterworth Low-Pass Filter (BLPF)

- A smoother version of ILPF. Instead of abrupt cut-offs, Butterworth decreases **gradually**.

$$H(u, v) = \frac{1}{1 + \left(\frac{D(u, v)}{D_0} \right)^{2n}}$$

Where n is the **order**.

- Higher $n \rightarrow$ more similar to Ideal LPF
- Lower $n \rightarrow$ smoother transition

Butterworth Low-Pass Filter (BLPF)

Property

Transition band

Ringing

Smoothness

Flexibility

Complexity

Behaviour

Gradual (controlled by n)

Very small (much less than Ideal)

Good

Order n controls sharpness

Moderate

Application

Medical images

General denoising

Images requiring contrast preservation

Preprocessing for segmentation

Why

Smooth without extreme distortions

Removes noise without strong artifacts

Better edge behavior

Smooth output without ringing

Gaussian Low-Pass Filter (GLPF)

- A perfectly smooth filter — no ringing, extremely natural smoothing.

$$H(u, v) = e^{-\frac{D(u, v)^2}{2D_0^2}}$$

Gaussian Low-Pass Filter (GLPF)

Property

Transition

Ringing

Smoothness

Edge preservation

Complexity

Behaviour

Smoothest of all (no slope discontinuity)

None (best in this regard)

Excellent

Reasonably good

Slightly heavier but manageable

Gaussian Low-Pass Filter (GLPF)

Application

Natural image smoothing
Removing Gaussian noise
Computer vision preprocessing
Medical imaging

Why

Most visually pleasant
Matches noise distribution
Does not distort edges sharply
Important for reducing artifacts

Comparison Summary: Ideal vs Butterworth vs Gaussian

Property	Ideal LPF	Butterworth LPF	Gaussian LPF
Cutoff	Sharp	Smooth (controlled by n)	Very smooth
Transition	Abrupt	Moderate	Smoothest
Ringings	High	Low	None
Natural appearance	Poor	Good	Best
Complexity	Low	Medium	Medium
Best for	Synthetic images	General smoothing	Natural images

Comparison Summary: Ideal vs Butterworth vs Gaussian

Which filter should you use?

- For **teaching or demonstration** → **Ideal LPF**
- For **balanced smoothing** → **Butterworth ($n=2$ or 3)**
- For **natural, artifact-free output** → **Gaussian LPF**



The End



Computer Vision and Image Processing

Lecture

Noise Models

Noise models provide a **formal, mathematical representation of how unwanted disturbances affect images**. Without noise models, it would not be possible to **analyze, simulate, remove, or design systems robust to noise** in a systematic way.

1. Gaussian Noise (Additive Noise)
2. Salt & Pepper Noise (Impulse Noise)
3. Speckle Noise (Multiplicative Noise)

Gaussian Noise (Additive Noise)

Gaussian noise is a type of **statistical noise** where the noise values follow a **normal (Gaussian) probability distribution**.

It usually occurs due to **sensor noise, poor illumination, or electronic interference**.

Mathematical Model

$$g(x, y) = f(x, y) + n(x, y)$$

- $f(x, y)$ → original image
- $n(x, y)$ → Gaussian noise with mean μ and variance σ^2
- $g(x, y)$ → noisy image

Gaussian Noise (Additive Noise)

Matrix Example

Original Image Matrix

$$F = \begin{bmatrix} 100 & 102 & 104 \\ 98 & 100 & 102 \\ 96 & 98 & 100 \end{bmatrix}$$

Gaussian Noise Matrix

$$N = \begin{bmatrix} 2 & -1 & 0 \\ -2 & 1 & 3 \\ 0 & -1 & 2 \end{bmatrix}$$

Gaussian Noise (Additive Noise)

Add corresponding elements:

$$G = F + N$$

$$G = \begin{bmatrix} 100 + 2 & 102 - 1 & 104 + 0 \\ 98 - 2 & 100 + 1 & 102 + 3 \\ 96 + 0 & 98 - 1 & 100 + 2 \end{bmatrix}$$

Resulting Noisy Image

$$G = \begin{bmatrix} 102 & 101 & 104 \\ 96 & 101 & 105 \\ 96 & 97 & 102 \end{bmatrix}$$

Salt & Pepper Noise (Impulse Noise)

Salt and Pepper noise is an **impulse noise** where pixels are randomly replaced by **minimum (0)** or **maximum (255)** intensity values.

- **Salt noise** → bright white pixels
- **Pepper noise** → dark black pixels

Mathematical Representation

$$g(x, y) = \begin{cases} 0 & \text{with probability } p \\ 255 & \text{with probability } q \\ f(x, y) & \text{otherwise} \end{cases}$$

Salt & Pepper Noise (Impulse Noise)

- **Step-by-Step Explanation**

1. Random pixels are selected.
2. Selected pixels are replaced with:
 - **255 (salt)** or
 - **0 (pepper)**
3. Remaining pixels stay unchanged.

Original Image

$$F = \begin{bmatrix} 120 & 122 & 124 \\ 118 & 120 & 122 \\ 116 & 118 & 120 \end{bmatrix}$$

Injected Salt & Pepper Noise

$$G = \begin{bmatrix} 120 & 255 & 124 \\ 0 & 120 & 122 \\ 116 & 118 & 0 \end{bmatrix}$$

Speckle Noise (Multiplicative Noise)

Speckle noise is a **multiplicative noise** commonly found in **radar, ultrasound, and medical imaging**.

Mathematical Model

$$g(x, y) = f(x, y) + f(x, y) \times n(x, y)$$

or

$$g(x, y) = f(x, y)(1 + n(x, y))$$

Speckle Noise (Multiplicative Noise)

Original Image

$$F = \begin{bmatrix} 50 & 52 & 54 \\ 48 & 50 & 52 \\ 46 & 48 & 50 \end{bmatrix}$$

Speckle Noise Matrix

$$N = \begin{bmatrix} 0.1 & -0.05 & 0 \\ 0.2 & -0.1 & 0.05 \\ 0 & 0.1 & -0.05 \end{bmatrix}$$

Speckle Noise (Multiplicative Noise)

Final Speckled Image Matrix

$$G = \begin{bmatrix} 55 & 49.4 & 54 \\ 57.6 & 45 & 54.6 \\ 46 & 52.8 & 47.5 \end{bmatrix}$$

Noise Removal Filters

Noise Removal Filters are techniques in image processing used to **reduce or remove unwanted disturbances (noise) from images**. Noise can be introduced during image acquisition, transmission, or storage, and it degrades image quality by altering pixel values randomly or systematically. The goal of these filters is to restore the image as close as possible to its original state while preserving important details like edges and textures.

1. Mean Filter (Linear Smoothing Filter)
2. Adaptive Filter
3. Order-Statistics Filter (Median Filter)
4. Inverse Filtering
5. Wiener Filter

Mean Filter (Linear Smoothing Filter)

Mean Filter Reduces random noise by averaging the pixel values in a neighborhood.

Formula:

$$g(i, j) = \frac{1}{m \times n} \sum_{k=-a}^a \sum_{l=-b}^b f(i + k, j + l)$$

Where $m \times n$ is the size of the filter kernel (e.g., 3×3).

Mean Filter (Linear Smoothing Filter)

Mean Filter Reduces random noise by averaging the pixel values in a neighborhood.

Formula:

$$g(i, j) = \frac{1}{m \times n} \sum_{k=-a}^a \sum_{l=-b}^b f(i + k, j + l)$$

Where $m \times n$ is the size of the filter kernel (e.g., 3×3).

Mean Filter (Linear Smoothing Filter)

Noisy image patch (3×3):

$$f = \begin{bmatrix} 52 & 55 & 61 \\ 68 & 70 & 65 \\ 61 & 59 & 60 \end{bmatrix}$$

Step 1: Apply 3×3 mean filter:

$$\text{Mean} = \frac{52 + 55 + 61 + 68 + 70 + 65 + 61 + 59 + 60}{9} = \frac{551}{9} \approx 61.22$$

Step 2: Replace central pixel (70) with 61.22.

Resulting patch:

$$g = \begin{bmatrix} 52 & 55 & 61 \\ 68 & 61 & 65 \\ 61 & 59 & 60 \end{bmatrix}$$

Adaptive Filter

Adaptive Filter adjusts the filter according to local variance. Good for non-uniform noise.

Formula (for pixel (i, j)):

$$g(i, j) = f(i, j) - \frac{\sigma_n^2}{\sigma_L^2} [f(i, j) - \mu_L]$$

Where:

- σ_n^2 = noise variance
- σ_L^2 = local variance of neighborhood
- μ_L = local mean

Adaptive Filter

Noisy patch:

$$f = \begin{bmatrix} 100 & 102 & 103 \\ 99 & 105 & 104 \\ 101 & 100 & 102 \end{bmatrix}$$

Assume $\sigma_n^2 = 4$.

Adaptive Filter

Step 1: Compute local mean $\mu_L = \frac{100+102+103+99+105+104+101+100+102}{9} = \frac{916}{9} \approx 101.78$

Step 2: Compute local variance $\sigma_L^2 = \frac{\sum(f(i,j) - \mu_L)^2}{9} \approx 5.73$

Step 3: Apply adaptive filter to central pixel (105):

$$g(2, 2) = 105 - \frac{4}{5.73}(105 - 101.78) \approx 105 - 2.28 \approx 102.72$$

Resulting central pixel: 102.72 (smoothed according to local noise).

Order-Statistics Filter (Median Filter)

Order-Statistics Filter (Median Filter) Removes **salt & pepper noise**. Uses median instead of mean.

Step:

1. Sort the pixel values in the neighborhood.
2. Replace the center pixel with the median.

Order-Statistics Filter (Median Filter)

Noisy patch:

$$f = \begin{bmatrix} 10 & 255 & 10 \\ 10 & 100 & 10 \\ 10 & 10 & 10 \end{bmatrix}$$

Step 1: Flatten neighborhood: [10, 255, 10, 10, 100, 10, 10, 10, 10]

Step 2: Sort: [10, 10, 10, 10, 10, 10, 10, 100, 255]

Step 3: Median = 10 → replace center pixel (100) with 10.

Resulting patch:

$$g = \begin{bmatrix} 10 & 255 & 10 \\ 10 & 10 & 10 \\ 10 & 10 & 10 \end{bmatrix}$$

Inverse Filtering

Inverse Filtering Removes blur caused by a linear system $H(u, v)$. Works in frequency domain.

Formula:

$$\hat{F}(u, v) = \frac{G(u, v)}{H(u, v)}$$

Where:

- $G(u, v)$ = degraded image in frequency domain
- $H(u, v)$ = degradation function (e.g., motion blur)
- $\hat{F}(u, v)$ = estimated original image

Inverse Filtering

Image patch (1D for simplicity):

$$f = [10, 20, 30]$$

Degradation kernel $h = [1, 1, 1]$ (simple averaging blur)

- Convolution: $g = f * h = [10, 30, 60, 50, 30]$
- To restore f , divide in frequency domain: $F = G/H \rightarrow$ approximate inverse filter

Step:

$$F \approx [10, 20, 30]$$

Note: Inverse filter amplifies noise if $H(u, v)$ has small values.

Wiener Filter

Wiener Filter Minimizes **mean square error**. Works for both noise and blur.

Formula (frequency domain):

$$\hat{F}(u, v) = \frac{H^*(u, v)}{|H(u, v)|^2 + S_n(u, v)/S_f(u, v)} G(u, v)$$

Where:

- $H(u, v)$ = degradation function
- S_n, S_f = noise & image power spectra
- H^* = complex conjugate of H

Wiener Filter

Noisy patch: $g = [10, 50, 100]$

Noise variance $\sigma_n^2 = 25$, original variance $\sigma_f^2 = 100$

- Wiener coefficient: $\frac{\sigma_f^2}{\sigma_f^2 + \sigma_n^2} = \frac{100}{125} = 0.8$
- Apply filter to pixel 50: $50 \times 0.8 = 40$

Resulting patch:

$$g = [10, 40, 80]$$

Smooths the noise while preserving edges better than mean filter.



The End



Computer Vision and Image Processing

Lecture

Image Segmentation

- **Image Segmentation** is a fundamental process in **Digital Image Processing and Computer Vision** that involves **dividing an image into multiple meaningful regions or segments**. Each segment corresponds to an object or a part of an object in the image.
- The primary objective is to **simplify or change the representation of an image** so that it becomes **easier to analyse and interpret**.

Image Segmentation

Image segmentation is a **pre-processing step** for higher-level vision tasks such as:

- Object detection and recognition
- Medical image analysis (tumour detection)
- Remote sensing and satellite imagery
- Autonomous driving
- Video surveillance

Image Segmentation

All segmentation techniques are based on **two fundamental image properties**:

1. Discontinuity

- Detects **abrupt changes** in pixel intensity
- Leads to **edge-based segmentation**

2. Similarity

- Groups pixels with **similar properties**
- Leads to **region-based and colour-based segmentation**

Image Segmentation

Basic Image Segmentation Techniques:

1. Edge-Based Segmentation
2. Colour-Based Segmentation
3. Region-Based Segmentation

Edge-Based Segmentation

Edge-based segmentation partitions an image by **detecting boundaries**—locations where there is a **significant discontinuity** in intensity or colour. These discontinuities typically correspond to **object borders**, making edges a natural cue for segmentation when region interiors are relatively uniform.



(a) Original



(b) Canny,
 $\sigma = 1$



(c) Canny,
 $\sigma = 3$

Edge-Based Segmentation

Roberts Operator

Uses 2×2 masks.

Very sensitive to noise; fast but coarse.

$$G_x = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad G_y = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

Prewitt Operator

Uses 3×3 masks; uniform weights.

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

Edge-Based Segmentation

Sobel Operator

Similar to Prewitt but emphasizes central pixels (better noise robustness).

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Edge-Based Segmentation

Canny Edge Detector

A multi-stage, optimal detector:

1. Gaussian smoothing (noise reduction)
2. Gradient computation
3. Non-maximum suppression
4. Double thresholding
5. Edge tracking by hysteresis

Provides thin, well-connected edges with low false detections.

Edge-Based Segmentation

Advantages

- Precise localization of object boundaries
- Computationally efficient (especially gradient-based methods)
- Effective when object borders are well-defined

Limitations

- Highly sensitive to noise (unless smoothed)
- Produces **edges**, **not regions**—additional processing is required for full segmentation
- Broken or spurious edges in textured or low-contrast images

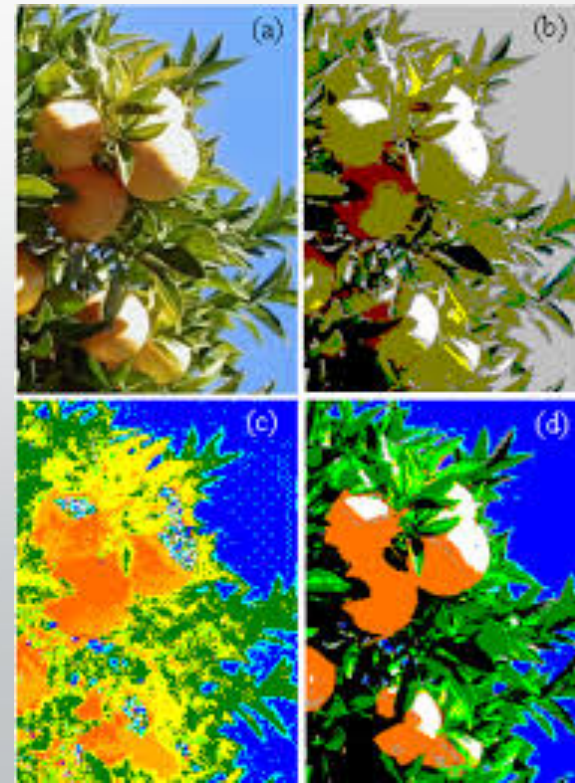
Edge-Based Segmentation

Applications

- Boundary detection in medical imaging
- Industrial inspection (crack detection)
- Document analysis
- Pre-processing for shape-based object recognition

Colour-Based Segmentation

Colour-based segmentation partitions an image by grouping pixels that are **similar in colour**. Unlike edge-based methods, which rely on intensity discontinuities, colour-based segmentation exploits the fact that **objects often have distinctive and consistent colour distributions**, making colour a powerful discriminative feature.



Colour-Based Segmentation

Basic Principle

Pixels are assigned to the same segment if their **colour distance** is within a threshold.

$$\|\mathbf{p} - \mathbf{c}\| \leq T$$

Where:

- $\mathbf{p}$ = pixel color vector
- $\mathbf{c}$ = reference color (cluster center)
- T = threshold

Colour-Based Segmentation

Distance Measures

Euclidean Distance (Most Common)

$$D = \sqrt{(R - R_c)^2 + (G - G_c)^2 + (B - B_c)^2}$$

Manhattan Distance

$$D = |R - R_c| + |G - G_c| + |B - B_c|$$

Colour-Based Segmentation

Input RGB Image (2×2)

$$I = \begin{bmatrix} (255, 0, 0) & (250, 10, 10) \\ (0, 255, 0) & (10, 250, 10) \end{bmatrix}$$

Reference Color (Red)

$$\mathbf{c} = (255, 0, 0)$$

Distance Calculation

Pixel (250,10,10):

$$\begin{aligned} D &= \sqrt{(250 - 255)^2 + (10 - 0)^2 + (10 - 0)^2} \\ &= \sqrt{25 + 100 + 100} = 15 \end{aligned}$$

Pixel (0,255,0):

$$D = \sqrt{(0 - 255)^2 + (255 - 0)^2 + (0 - 0)^2} \approx 361$$

Assume $T = 30$.

Segmented Output (Binary Mask)

$$S = \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix}$$

Pixels labeled 1 belong to the red segment.

Colour-Based Segmentation

Advantages

- Effective for images with **distinct colours**
- Simple mathematical formulation
- Works well for **multi-object scenes**
- Robust when using HSV or Lab spaces

Limitations

- Sensitive to illumination in RGB
 - Poor performance when object and background share similar colours
- Shadows and highlights can cause misclassification

Colour-Based Segmentation

Applications

- Object tracking and recognition
- Medical image analysis (tissue segmentation)
- Satellite and aerial image processing
- Traffic sign and lane detection

Region-Based Segmentation

Region-based segmentation partitions an image into regions by **grouping neighbouring pixels that are similar** according to a predefined homogeneity criterion (intensity, colour, texture).

Unlike edge-based segmentation, which focuses on **boundaries**, region-based segmentation focuses on **region interiors** and produces **closed, connected regions directly**.

Region-Based Segmentation

The fundamental idea is **similarity + spatial connectivity**.

Mathematically, a region R satisfies:

$$|I(x, y) - \mu_R| \leq T \quad \forall (x, y) \in R$$

Where:

- $I(x, y)$ = pixel intensity (or color vector)
- μ_R = mean value of the region
- T = similarity threshold

Region-Based Segmentation

Why Region-Based Segmentation Is Needed

- Edge-based methods:
- Detect boundaries only
- Often produce broken edges

Region-based methods:

- Produce **complete object regions**
- Are more robust in **low-contrast or noisy images**
- Are well-suited for **medical, satellite, and document images**

Region-Based Segmentation

Main Types of Region-Based Segmentation

- 1. Region Growing**
- 2. Region Splitting**
- 3. Region Merging**
- 4. Split-and-Merge (Hybrid)**

Region Growing

Region growing starts from **seed pixels** and **grows regions outward** by adding neighbouring pixels that are similar.

Algorithm Steps

1. Select one or more **seed pixels**
2. Initialize region with seed
3. Compare neighbouring pixels with region mean
4. Add pixel if similarity condition is satisfied
5. Update region mean

Repeat until no more pixels qualify

Region Growing

Input Image

$$I = \begin{bmatrix} 50 & 52 & 80 \\ 49 & 51 & 82 \\ 48 & 50 & 85 \end{bmatrix}$$

Seed Pixel

$$(0, 0) = 50$$

Threshold:

$$T = 3$$

Step-by-Step Growth

- Neighbor (0,1) = 52 → $|52 - 50| = 2 \leq 3$ ✓
- Neighbor (1,0) = 49 → $|49 - 50| = 1 \leq 3$ ✓
- Neighbor (1,1) = 51 → $|51 - 50| = 1 \leq 3$ ✓
- Neighbor (0,2) = 80 → $|80 - 50| = 30$ ✗
- Neighbor (1,2) = 82 → ✗

Final Region Mask

$$R = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 0 \end{bmatrix}$$

Region Growing

Advantages

- Produces connected regions
- Intuitive and simple

Limitations

- Sensitive to seed selection
- Computationally expensive for large images

Region Splitting

Concept

- Region splitting follows a **top-down approach**:
- Start with the whole image
- Split regions if homogeneity condition fails
- Continue recursively (usually into quadrants)

Region Splitting

Input Image

$$I = \begin{bmatrix} 10 & 10 & 80 & 80 \\ 10 & 10 & 80 & 80 \\ 12 & 12 & 82 & 82 \\ 12 & 12 & 82 & 82 \end{bmatrix}$$

Analysis

- Entire image variance → high ✗
- Split into 4 quadrants:

$$Q_1 = \begin{bmatrix} 10 & 10 \\ 10 & 10 \end{bmatrix} \quad Q_2 = \begin{bmatrix} 80 & 80 \\ 80 & 80 \end{bmatrix}$$

Each quadrant variance → low ✓

Region Splitting

Why Splitting Works

- Large regions often contain multiple objects
- Splitting isolates homogeneous subregions automatically

Limitations

- Produces block-shaped regions
- Can lead to over-segmentation

Region Merging

Region merging is a **bottom-up approach**:

- Start with many small regions
- Merge adjacent regions if they are similar

$$|\mu_{R_1} - \mu_{R_2}| \leq T$$

Region Merging

Two Adjacent Regions

$$R_1 = \begin{bmatrix} 50 & 51 \\ 49 & 50 \end{bmatrix} \Rightarrow \mu_{R_1} = 50$$

$$R_2 = \begin{bmatrix} 52 & 51 \\ 50 & 53 \end{bmatrix} \Rightarrow \mu_{R_2} = 51.5$$

Threshold:

$$T = 3$$

$$|50 - 51.5| = 1.5 \leq 3 \Rightarrow \text{Merge}$$

Region Merging

Why Merging Is Needed

- Reduces over-segmentation
- Combines meaningful regions
- Smooths noisy boundaries

Split-and-Merge (Hybrid Approach)

Concept

- First **split** regions until homogeneous
- Then **merge** adjacent similar regions

This balances:

- Over-segmentation (splitting)
- Under-segmentation (merging)



The End



Computer Vision and Image Processing

Lecture 13

Features

- **Feature point / keypoint / interest point:** A specific location (x,y) in the image where something "interesting" happens.
- **Feature descriptor:** A numerical vector that encodes what the neighborhood around that point looks like.

When someone says "*We detected 500 features in this image,*" they usually mean **keypoints**. When they say "*We matched features between two images,*" they mean **descriptors**.

Features

In computer vision, when we look at two images of the same scene (e.g., taken from different angles or lighting conditions), how do we know they match?

- **Global features** (like color histograms) look at the whole image and fail if there's occlusion or a viewpoint change.
- **Local features** focus on distinct, invariant patches (like corners, blobs, or distinctive textures) that can be reliably found and tracked across different images.

Features

A good feature should ideally be:

- **Distinctive** – easy to distinguish from other features.
- **Repeatable** – detected again if the same object appears in another image.
- **Robust** – should remain useful under:
 - Rotation
 - Scale changes
 - Illumination changes
 - Noise
 - Small viewpoint changes

Features

The Pipeline

- **Detection:** Find interesting points (keypoints/corners).
- **Description:** Create a mathematical fingerprint (descriptor) for the local neighborhood around each keypoint.
- **Matching:** Compare descriptors between Image A and Image B to find correspondences.
- **Verification:** Use geometric constraints (like RANSAC) to throw out false matches. *(Focusing here on steps 1–3).*

Feature Detection Algorithms

The detector identifies the location of interesting points

1. Harris Corner Detector
2. Shi-Tomasi
3. FAST (Features from Accelerated Segment Test)
4. SIFT Detector (Scale-Invariant Feature Transform)
5. SURF Detector (Speeded-Up Robust Features)

Feature Descriptors Algorithms

For each keypoint, we generate a numerical representation called a **feature descriptor**. The descriptor is essentially the **signature** of that local feature. A **feature descriptor** is a numerical representation of the visual characteristics of a local image region.

1. SIFT (Scale-Invariant Feature Transform)
2. SURF (Speeded-Up Robust Features)
3. ORB (Oriented FAST and Rotated BRIEF)
4. HOG (Histogram of Oriented Gradients)

Feature Matching

Feature matching is the process of establishing correspondences between the local features (keypoints and their descriptors) of two or more different images of the same scene or object.

Once you have detected keypoints and generated their mathematical "fingerprints" (descriptors, like SIFT or ORB) in **Image A** and **Image B**, feature matching is how your algorithm figures out which descriptor in Image A belongs to the exact same physical point in Image B.

Feature Matching

Distance Metrics

- **Euclidean Distance (L_2):** Standard for floating-point descriptors like SIFT.
- **Hamming Distance:** Used for binary descriptors (like BRIEF, ORB) where features are strings of bits (counted via XOR + bit-counting).
-

Feature Matching

Matching Strategies

1. Brute-Force Matching

- Compare every descriptor in Image A with every descriptor in Image B. Find the closest one based on distance.
- *Drawback:* Prone to false positives because ambiguous features might accidentally find a "closest" neighbor that isn't actually a true match.

Feature Matching

2. Lowe's Ratio Test (The Gold Standard)

David Lowe introduced a brilliant heuristic in his SIFT paper to filter out bad matches:

- For a descriptor in Image A, find the **1st nearest neighbor** (NN_1) and the **2nd nearest neighbor** (NN_2) in Image B.

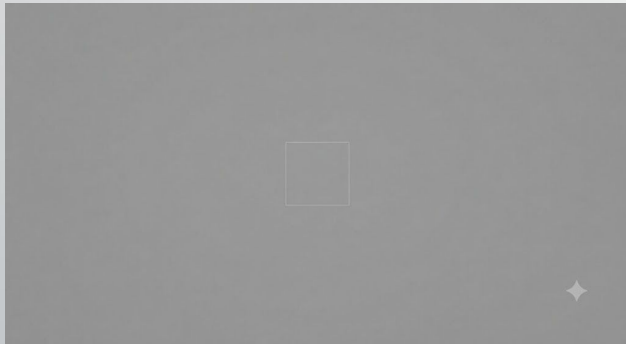
Compute the ratio:

- $$\text{Ratio} = \frac{\text{Distance}(NN_1)}{\text{Distance}(NN_2)}$$

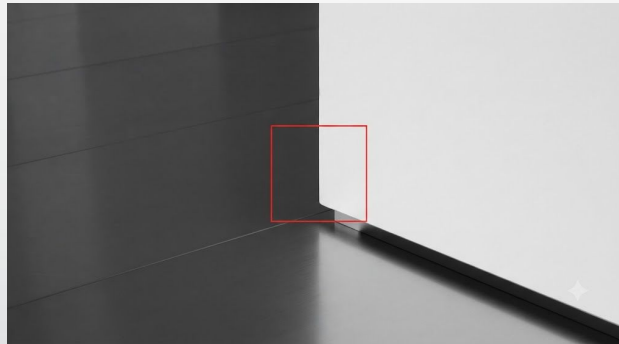
- **Intuition:** If NN_1 is truly distinct, its distance will be much smaller than NN_2 (ratio $\ll 1$). If the point is ambiguous, NN_1 and NN_2 will have very similar distances (ratio ≈ 1).
- **Threshold:** Typically, discard matches where the ratio is greater than 0.7 or 0.8.

Harris Corner Detection

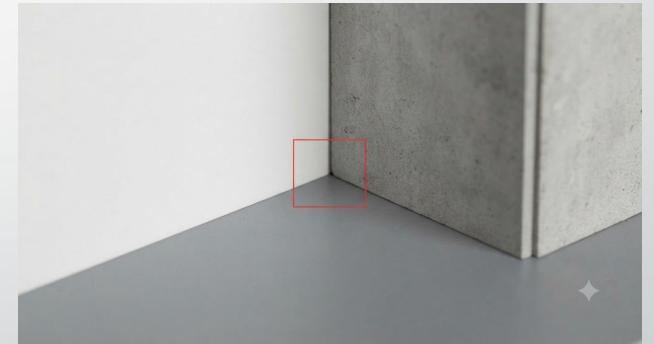
Developed by **Chris Harris and Mike Stephens** in **1988**, this detector relies on the idea of shifting a small window over the image and measuring the intensity change.



Flat

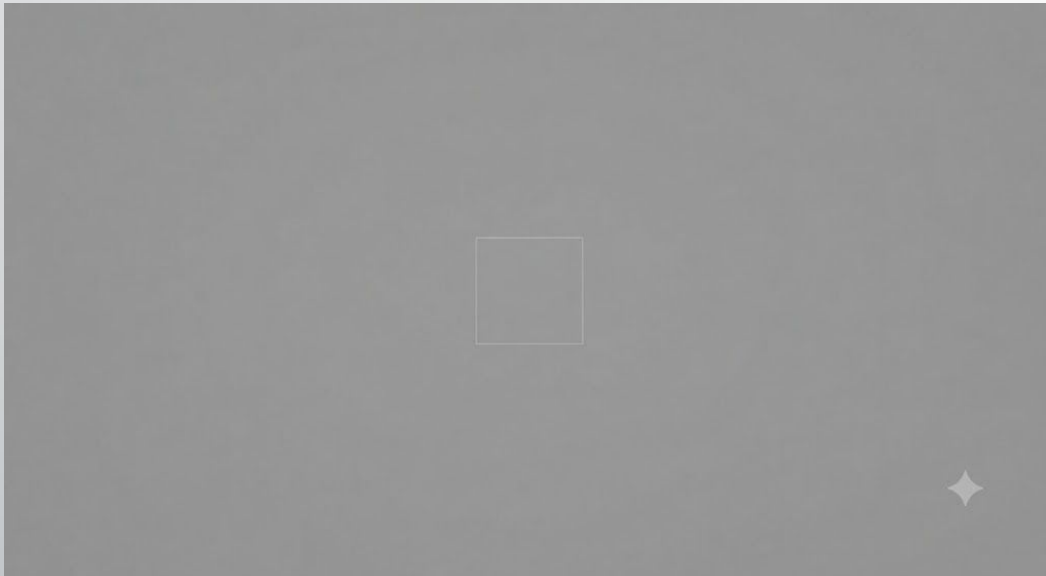


Edge



Corner

Harris Corner Detection

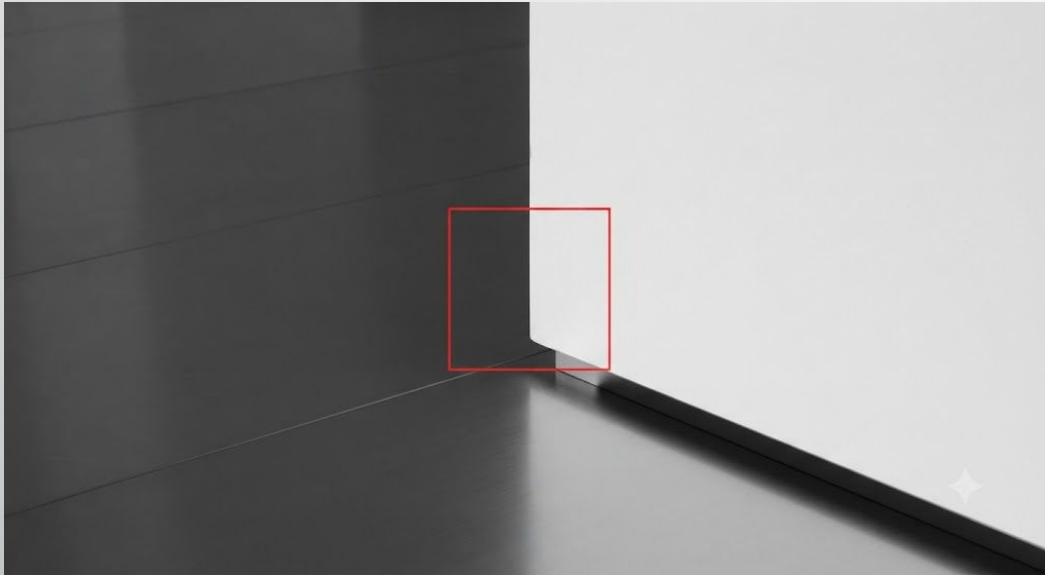


If we move a small window in any direction, the appearance does not change much.

Therefore:

- No significant intensity change
- This is not a useful feature.

Harris Corner Detection

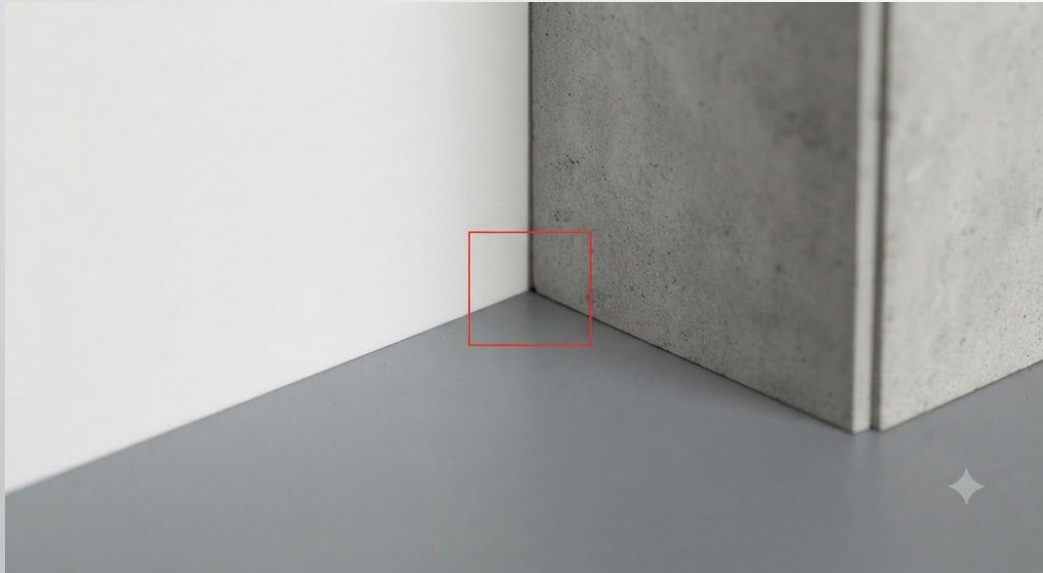


If we move the window:

- Along the edge → little change
- Across the edge → significant change

Therefore, an edge has strong intensity variation in **one direction**.

Harris Corner Detection



If we move the window:

- Horizontally → significant change
- Vertically → significant change

Therefore, a corner has strong intensity variation in **two directions**.

This is the key idea behind Harris Corner Detection.

Harris Corner Detection

- Suppose we take a small window around a pixel: $W(x, y)$
- Now we shift the window by: (u, v)
- We compare the original window with the shifted window.
- The change can be represented using the **sum of squared differences**:

Harris Corner Detection

- $$E(u, v) = \sum_{x, y} W(x, y) [I(x + u, y + v) - I(x, y)]^2$$

Where:

- $I(x, y)$ = image intensity
- $W(x, y)$ = weighting function
- (u, v) = displacement

The interpretation is:

- If $E(u, v)$ is small for every direction $\rightarrow$ flat region
- If $E(u, v)$ is large in one direction $\rightarrow$ edge
- If $E(u, v)$ is large in all directions $\rightarrow$ corner

Harris Corner Detection

Using a first-order Taylor approximation:

$$I(x + u, y + v) \approx I(x, y) + I_x u + I_y v$$

Where:

•

and

$$I_x = \frac{\partial I}{\partial x}$$

$$I_y = \frac{\partial I}{\partial y}$$

Harris Corner Detection

- Substituting into the error function: $E(u, v) \approx \sum W(x, y)(I_x u + I_y v)^2$
- This can be written in matrix form:

$$E(u, v) = [u \quad v] M \begin{bmatrix} u \\ v \end{bmatrix}$$

Where M is the **second-moment matrix**, also called the **structure tensor**:

$$M = \sum W(x, y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

- Usually, the matrix is written as:

$$M = \begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix}$$

Harris Corner Detection

- The matrix M has two eigenvalues: λ_1, λ_2
- These eigenvalues describe the amount of intensity variation in two principal directions.
- **Flat Region:** $\lambda_1 \approx 0, \lambda_2 \approx 0$ (Very little change in any direction)
- **Edge:** $\lambda_1 \gg 0, \lambda_2 \approx 0$ (Strong variation in one direction)
- **Corner:** $\lambda_1 \gg 0, \lambda_2 \gg 0$ (Strong variation in both directions)

Harris Corner Detection

- Instead of explicitly calculating the eigenvalues, Harris proposed the response function:

$$R = \det(M) - k(\text{trace}(M))^2$$

Where:

$$\det(M) = \lambda_1 \lambda_2$$

and:

$$\text{trace}(M) = \lambda_1 + \lambda_2$$

- The parameter k is typically:

$$k \approx 0.04 \text{ to } 0.06$$

Harris Corner Detection

Instead of explicitly calculating the eigenvalues,
Harris proposed the response function:

$$R = \det(M) - k(\text{trace}(M))^2$$

Where:

$$\det(M) = \lambda_1 \lambda_2$$

and:

$$\text{trace}(M) = \lambda_1 + \lambda_2$$

The parameter k is typically:

$$k \approx 0.04 \text{ to } 0.06$$

$R > 0 \rightarrow \text{Corner}$

$R < 0 \rightarrow \text{Edge}$

$R \approx 0 \rightarrow \text{Flat region}$

Harris Corner Detection Algorithm

- Step 1: Convert Image to Grayscale
- Step 2: Compute Image Gradients: I_x and I_y by Usually using Sobel operators:
 - $I_x = I * S_x$
 - $I_y = I * S_y$
- Step 3: Compute Products: I_x^2 , I_y^2 , $I_x I_y$
- Step 4: Apply Gaussian Smoothing: The gradient products are averaged over a neighborhood: $\sum I_x^2 \sum I_y^2 \sum I_x I_y$ (This produces the matrix M)

Harris Corner Detection Algorithm

- Step 5: Calculate Harris Response: $R = \det(M) - k(\text{trace}(M))^2$
- Step 6: Thresholding: Only points satisfying: $R > T$ are considered corners.
- Step 7: Non-Maximum Suppression: If several nearby pixels are detected as corners, retain only the strongest one. This prevents a single physical corner from producing many redundant keypoints.

Limitation of Harris Corner Detection Algorithm

Harris corner detection is:

- **Translation invariant:** If the image moves, the corner can still be detected.
- **Rotation invariant:** The corner response is generally stable under rotation.
- However, standard Harris detection is **not inherently scale invariant:** The same corner may appear at different scales, making direct comparison difficult.
- This motivated the development of more advanced feature detectors and descriptors such as:
 - SIFT
 - SURF



The End



Computer Vision and Image Processing

Lecture 14

Image Transformations

An **image transformation** is the process of changing the position, orientation, scale, or shape of an image. Unlike image enhancement (brightness, contrast, filtering), image transformation changes the **geometry** of the image.

Why are Image Transformations Needed?

- Panorama stitching
- Face alignment
- Image registration
- Satellite image matching
- Medical image registration
- Object tracking
- Augmented Reality
- OCR document correction

Types of Image Transformations

A. Intensity Transformation

- Brightness
- Contrast
- Gamma correction

Does NOT move pixels.

B. Geometric Transformation

- Translation
- Rotation
- Scaling
- Reflection
- Shearing

Translation

Translation means **moving an image from one location to another without changing its size or orientation.**

Examples:

- Moving an object to the right
- Shifting a camera frame
- Correcting image displacement
- The shape remains exactly the same.

Translation

Suppose a point is: $P(x, y)$

After translation: $x' = x + t_x$

$y' = y + t_y$

Where: t_x = horizontal movement

t_y = vertical movement

Assume a point: $P(3, 5)$

We move it : $t_x = 4$

$t_y = 2$

Therefore: $x' = 3 + 4 = 7$

$y' = 5 + 2 = 7$

New position: $P'(7, 7)$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Multiplying this out yields:

- $x' = (1 \cdot x) + (0 \cdot y) + (t_x \cdot 1) = x + t_x$
- $y' = (0 \cdot x) + (1 \cdot y) + (t_y \cdot 1) = y + t_y$
- $1 = (0 \cdot x) + (0 \cdot y) + (1 \cdot 1) = 1$

Scaling

Scaling is a linear geometric transformation that alters the size of an image by expanding or compressing its dimensions along the coordinate axes.

Two Types of Scaling:

1. Enlargement
2. Shrinking

Scaling

Suppose a point is: $P(x, y)$

After translation: $x' = S_x \cdot x$

$y' = S_y \cdot y$

Where:

S_x = scaling factor in x direction

S_y = scaling factor in y direction

Assume a point: $P(3, 5)$

We move it : $S_x = 4$

$S_y = 2$

Therefore: $x' = 3 \cdot 4 = 12$

$y' = 5 \cdot 2 = 10$

New position: $P'(12, 10)$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- **The Scale Factors:**

- If $s > 1$, the image **enlarges** (magnification/zoom-in).
- If $0 < s < 1$, the image **shrinks** (reduction/zoom-out).
- If $s = 1$, the image remains unchanged.
- If $s < 0$, it introduces a **reflection** (flipping) across the axis in addition to scaling.

- **Uniform vs. Non-Uniform Scaling:**

- If $s_x = s_y$, it is *uniform scaling*. The aspect ratio is preserved, meaning the object grows or shrinks symmetrically without distortion.
- If $s_x \neq s_y$, it is *non-uniform scaling*. The image will stretch or squash horizontally or vertically (e.g., turning a square into a rectangle), which distorts the original proportions.

Rotation

Rotation is a rigid-body geometric transformation that turns an image around a fixed center point by a specific angle θ . It preserves all lengths, areas, and internal angles, meaning the shape and size of the object remain completely unaltered.

Examples:

- Rotating a photo
- Correcting camera tilt
- Aligning objects

Rotation

- $x' = x\cos\theta - y\sin\theta$
- $y' = x\sin\theta + y\cos\theta$
- $R = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$

Rotate a Point by 90°

- Given: $P(2,1)$
- Angle: $\theta = 90^\circ$

- We know:

$$\cos 90 = 0$$

$$\sin 90 = 1$$

- Now:
- $x' = 2(0) - 1(1) \quad x' = -1$
- $y' = 2(1) + 1(0) \quad y' = 2$
- Therefore: $\boxed{P'(-1, 2)}$

Reflection

Reflection (often called flipping or mirroring) is a linear geometric transformation that mirrors an image across a specified axis (such as the X-axis, Y-axis, or origin). Like rigid transformations, it preserves lengths and internal angles, but it introduces a **chirality (handedness) reversal**—turning a "left-handed" orientation into a "right-handed" one (like looking in a mirror).

Reflection

A. Reflection Across the Y-Axis (Horizontal Flip)

If you flip an image horizontally, every pixel at position (x, y) moves to $(-x, y)$ relative to the axis of reflection:

$$x' = -x$$

$$y' = y$$

Expressed in matrix form:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Reflection

B. Reflection Across the X-Axis (Vertical Flip)

If you flip an image vertically, every pixel at position (x, y) moves to $(x, -y)$:

$$x' = x$$

$$y' = -y$$

Expressed in matrix form:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Reflection

C. Reflection Across the Origin

Flipping across both axes simultaneously ($x' = -x, y' = -y$) multiplies both scale factors by -1 :

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Shearing

Shearing (or skewing) is a linear geometric transformation that pushes one part of an image in a certain direction while keeping the opposite part fixed, causing the shape to distort into a parallelogram. Unlike rotation or scaling, shearing alters the internal angles between lines.

Shearing is rarely used in isolation for creative effects, but it is heavily used as a **data augmentation** technique in deep learning (artificially slanting text or objects to train robust neural networks) and as an intermediate step in fast affine decomposition algorithms.

Shearing

A. Horizontal Shear (Shear along the X-axis)

In a horizontal shear, the X-coordinate of a pixel is shifted proportional to its Y-coordinate. The Y-coordinate remains entirely unaffected:

$$x' = x + sh_x \cdot y$$

$$y' = y$$

Expressed in matrix form:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & sh_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Shearing

B. Vertical Shear (Shear along the Y-axis)

In a vertical shear, the Y-coordinate is shifted proportional to its X-coordinate, while the X-coordinate remains untouched:

$$x' = x$$

$$y' = sh_y \cdot x + y$$

Expressed in matrix form:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Image Warping in Computer Vision

Image warping is the process of transforming an image by changing the spatial locations of its pixels according to a mathematical mapping function.

In simple words : Image warping changes the shape, size, orientation, or perspective of an image by moving pixels from their original positions to new positions (by applying transformations).

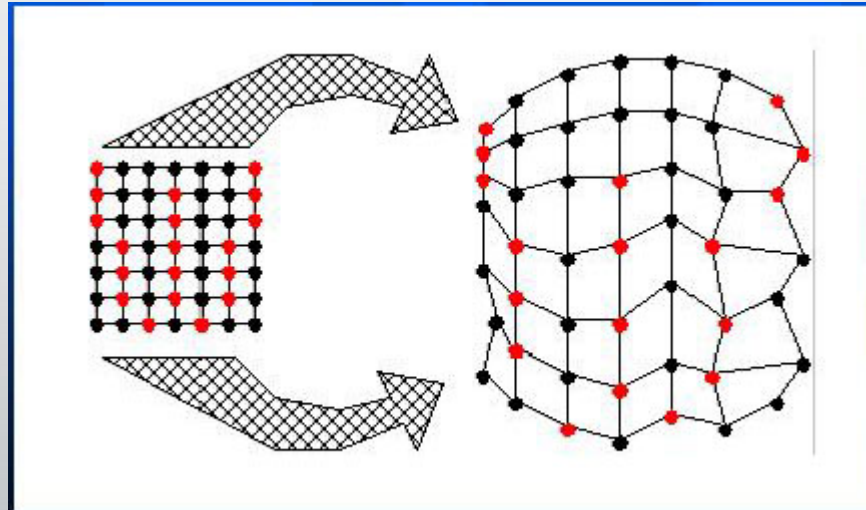


Image Warping in Computer Vision



1. Panorama Stitching



2. Face Alignment



3. Document Correction
(Deskewing)

Image Warping in Computer Vision

Forward Mapping: In forward mapping, every pixel from the original image is moved to a new position. The process: Source pixel $\rightarrow$ Destination pixel

Problem of Forward Mapping: Some pixels may not receive any value. Empty spaces are called : **holes** because the destination image contains pixels without intensity values.

Backward Mapping: Backward mapping works in the opposite direction. Instead of asking: Where should this source pixel go? it asks: From where should this destination pixel get its value? The process : Destination pixel $\rightarrow$ Find source pixel

Advantages:

- No holes
- Every output pixel gets a value
- Better image quality

Therefore, most computer vision libraries use backward warping.

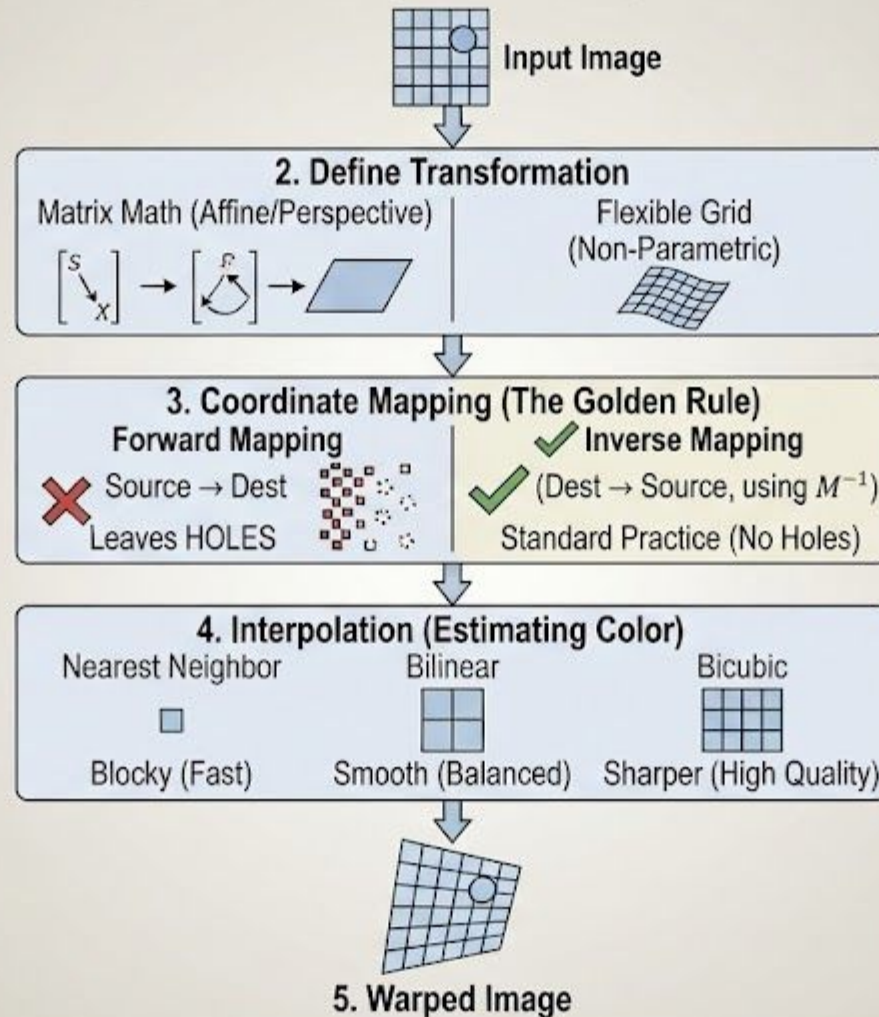
Image Warping in Computer Vision

Interpolation in Image Warping: After transformation, the calculated coordinate may not be an integer. Therefore, we need interpolation.

1. **Nearest Neighbour Interpolation**
2. **Bilinear Interpolation**
3. **Bicubic Interpolation**

Image Warping in Computer Vision

Simplified Image Warping Pipeline.



Affine vs Perspective

	Affine	Perspective
Think of it as	Moving/stretching an image	Changing camera viewpoint
Shape	Parallelogram	Trapezoid possible
Parallel lines	Stay parallel	May converge
Camera model	Approximate	Real camera projection
Common use	Alignment, rotation, scaling	Panorama, document correction



The End



Computer Vision and Image Processing

Lecture 15

Stereo, Multiview Stereo, Light and Perception

- Human beings perceive the **three-dimensional (3D) world** using two eyes. Each eye captures a slightly different image because of the small distance between them. The brain combines these two images and estimates **depth** and **distance** of objects.
- Computer vision attempts to replicate this ability using cameras. A camera captures only a **2D projection** of the real world, losing depth information. Stereo vision and multiview stereo techniques recover this missing depth information by analyzing images captured from multiple viewpoints.

Stereo, Multiview Stereo, Light and Perception

The topics **Stereo Vision, Multiview Stereo, Light, and Perception** are fundamental in:

- 3D reconstruction
- Autonomous vehicles
- Robot navigation
- Augmented Reality (AR)
- Virtual Reality (VR)
- Medical imaging
- Object measurement
- Computational photography

Human Depth Perception

- **Monocular Depth Cues:** These cues require only one eye.

a) Perspective: Objects farther away appear smaller. Example: Railway Track appears to converge at the horizon.

b) Occlusion: If one object blocks another, the blocking object is considered closer. Example: A nearby car blocks part of a distant car. The blocked car is perceived as being farther.

c) Texture Gradient: Texture becomes denser as distance increases. Example: A brick road.

d) Shadows and Lighting: The brain estimates shape and depth using shadows. Example: A ball's shadow on the ground helps us understand its 3D shape and position.

Human Depth Perception

- **Binocular Depth Perception:** Humans have two eyes separated by approximately 6–7 cm. Each eye receives a slightly different image. This difference is called: **Binocular Disparity**.
- The shift between positions of corresponding points in the two retinal images provides information. The brain calculates:

$$Depth \propto \frac{1}{Disparity}$$

- Large disparity → object is close
- Small disparity → object is far

Stereo Vision

Stereo vision is a computer vision technique that estimates depth from two images captured from different viewpoints.

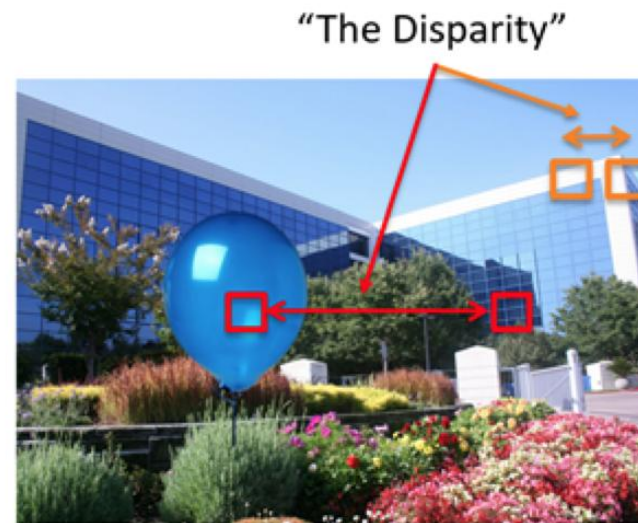
A typical stereo system consists of:

- Left camera
- Right camera
- Known distance between cameras (baseline)

Stereo Vision



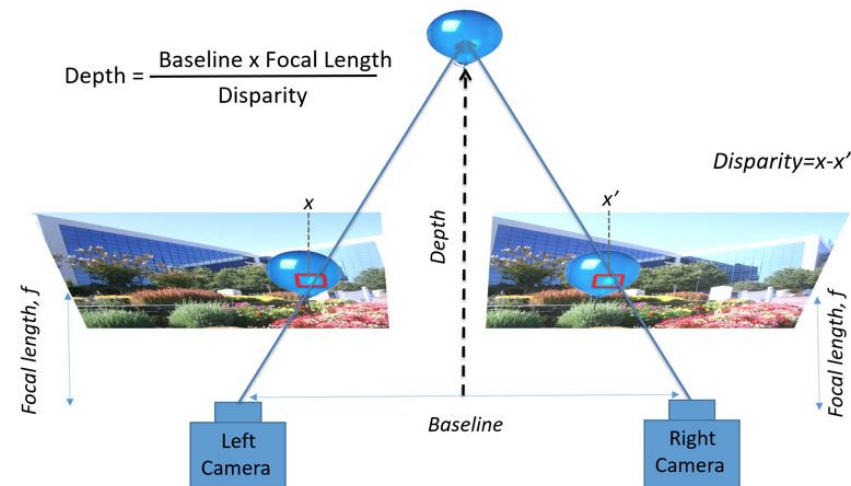
Left camera



Right camera



Disparity Map referenced to Right camera



Stereo Matching

- Stereo matching is the process of finding corresponding points between left and right images. The same object point appears at different locations.

$$d = x_L - x_R$$

- **Correspondence Problem:** The major challenge in stereo vision is: "Which pixel in the left image corresponds to which pixel in the right image?"
- Finding correct matches is difficult because of:
 - Similar textures
 - Repeated patterns
 - Occlusion
 - Illumination changes
- image corresponds to which pixel in the right image?"

Stereo Matching

- **Epipolar Geometry:** This is the underlying geometry of stereo vision. If you have a point in 3D space, it projects to a specific pixel in the left image. In the right image, that corresponding pixel is mathematically constrained to lie along a specific 1D line called the **epipolar line**. This reduces the search for matching pixels from a 2D plane to a 1D line, drastically improving computational efficiency.
- **Image Rectification:** Before searching for matches, images are mathematically transformed (rectified) so that the epipolar lines are perfectly horizontal. This means a pixel at coordinate (x, y) in the left image will only match with a pixel at (x', y) in the right image.

Stereo Matching

- **Depth Calculation:** Once disparity d is found (where $d = x_L - x_R$), depth Z is calculated using the focal length f and the baseline B (distance between camera centers) using the formula:

$$Z = \frac{f \cdot B}{d}$$

- This equation shows that depth is inversely proportional to disparity: closer objects have larger disparities, while objects at infinity have near-zero disparity.

Multiview Stereo (MVS)

Multiview Stereo extends stereo vision by using multiple images captured from many viewpoints.

Feature	Stereo Vision	Multiview Stereo
Number of images	2	Multiple
Cameras	Two	Many
Depth accuracy	Moderate	High
Reconstruction	Partial	Complete 3D model
Complexity	Low	High
Applications	Robotics, vehicles	3D scanning, reconstruction

Light

- Light plays an important role because cameras capture reflected light.

- Types of Reflection:

- **Diffuse Reflection:** Light scatters equally in all directions.
 - **Specular Reflection:** Light reflects in one direction.

- Computer vision can use shading to infer shape from illumination. Shape from Shading. Idea : Brightness indicates surface orientation.



Photometric Stereo

- Standard stereo relies on finding matching textures or features. But what if an object is perfectly smooth and featureless, like a porcelain vase? Standard stereo fails. This is where **Photometric Stereo** comes in.
 - Instead of changing the *camera viewpoint*, photometric stereo changes the *direction of light* while keeping the camera completely still.
 - Applications:
 - Face recognition
 - Industrial inspection
 - Medical imaging
- t* while keeping the camera completely still.

Perception in Computer Vision

Computer vision perception means understanding the environment from visual data. A perception system performs : Image Input -> Feature Extraction -> Depth Estimation -> Object Detection -> Scene Understanding

Components of Visual Perception:

- 1. Detection:** Finding objects.
- 2. Recognition:** Identifying objects.
- 3. Localization:** Finding position.
- 4. Scene Understanding:** Understanding relationships.

Applications

Autonomous Vehicles

- Uses:
 - Stereo cameras
 - LiDAR
 - Deep learning
- Tasks:
 - Lane detection
 - Depth estimation
 - Collision avoidance

Applications

- **Robotics**

- Navigate
- Avoid obstacles
- Map environments

- **AR/VR**

- 3D environment reconstruction
- Camera tracking

- **Medical Imaging**

- 3D organ reconstruction

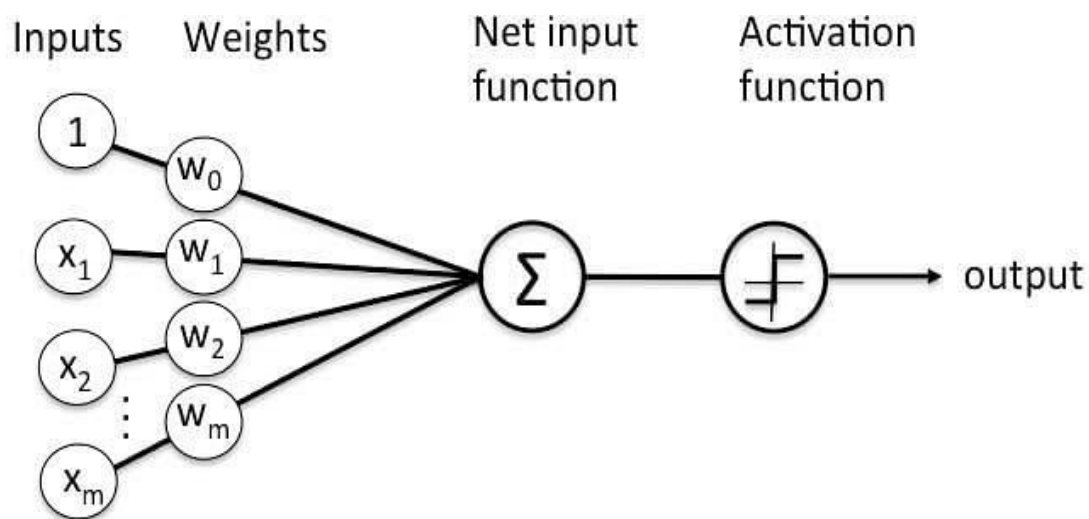
Surgical navigation



The End

Perceptron

Perceptron was introduced by Frank Rosenblatt in 1957. He proposed a Perceptron learning rule based on the original MCP neuron. A Perceptron is an algorithm for supervised learning of binary classifiers. This algorithm enables neurons to learn and processes elements in the training set one at a time.



Types of Perceptron:

1. Single layer: Single layer perceptron can learn only linearly separable patterns.
2. Multilayer: Multilayer perceptrons can learn about two or more layers having a greater processing power.

The Perceptron algorithm learns the weights for the input signals in order to draw a linear decision boundary.

Note: Supervised Learning is a type of Machine Learning used to learn models from labeled training data. It enables output prediction for future or unseen data. Let us focus on the Perceptron Learning Rule in the next section.

Perceptron in Machine Learning

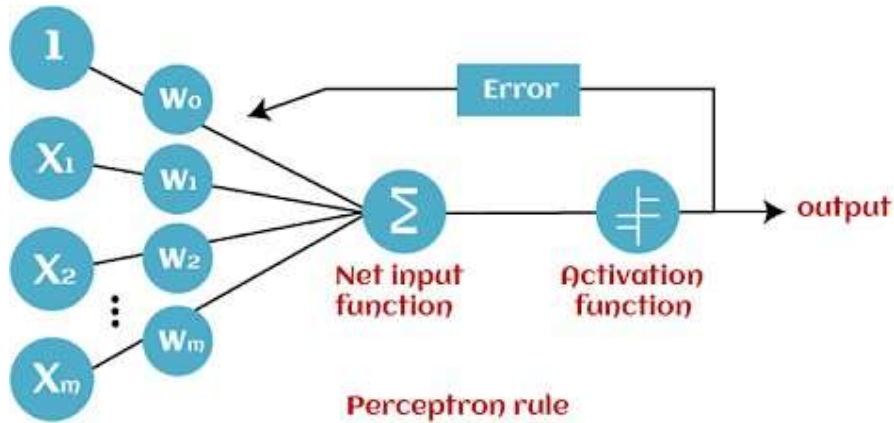
The most commonly used term in Artificial Intelligence and Machine Learning (AIML) is Perceptron. It is the beginning step of learning coding and Deep Learning technologies, which consists of input values, scores, thresholds, and weights implementing logic gates. Perceptron is the nurturing step of an Artificial Neural Link. In 19h century, Mr. Frank Rosenblatt invented the Perceptron to perform specific high-level calculations to detect input data capabilities or business intelligence. However, now it is used for various other purposes.

What is the Perceptron Model in Machine Learning?

A machine-based algorithm used for supervised learning of various binary sorting tasks is called Perceptron. Furthermore, Perceptron also has an essential role as an Artificial Neuron or Neural link in detecting certain input data computations in business intelligence. A perceptron model is also classified as one of the best and most specific types of Artificial Neural networks. Being a supervised learning algorithm of binary classifiers, we can also consider it a single-layer neural network with four main parameters: input values, weights and Bias, net sum, and an activation function.

How Does Perceptron Work?

AS discussed earlier, Perceptron is considered a single-layer neural link with four main parameters. The perceptron model begins with multiplying all input values and their weights, then adds these values to create the weighted sum. Further, this weighted sum is applied to the activation function 'f' to obtain the desired output. This activation function is also known as the step function and is represented by 'f.'



This step function or Activation function is vital in ensuring that output is mapped between (0,1) or (-1,1). Take note that the weight of input indicates a node's strength. Similarly, an input value gives the ability to shift the activation function curve up or down.

Step 1: Multiply all input values with corresponding weight values and then add to calculate the weighted sum. The following is the mathematical expression of it:

$$\sum w_i * x_i = x_1 * w_1 + x_2 * w_2 + x_3 * w_3 + \dots + x_n * w_n$$

Add a term called bias 'b' to this weighted sum to improve the model's performance.

Step 2: An activation function is applied with the above-mentioned weighted sum giving us an output either in binary form or a continuous value as follows:

$$Y = f(\sum w_i * x_i + b)$$

Types of Perceptron models

We have already discussed the types of Perceptron models in the Introduction. Here, we shall give a more profound look at this:

1. **Single Layer Perceptron model:** One of the easiest ANN (Artificial Neural Networks) types consists of a feed-forward network and includes a threshold transfer inside the model. The main objective of the single-layer perceptron model is to analyze the linearly separable objects with binary outcomes. A Single-layer perceptron can learn only linearly separable patterns.
2. **Multi-Layered Perceptron model:** It is mainly similar to a single-layer perceptron model but has more hidden layers.

Forward Stage: From the input layer in the on stage, activation functions begin and terminate on the output layer.

Backward Stage: In the backward stage, weight and bias values are modified per the model's requirement. The backstage removed the error between the actual output and demands originating backward on the output layer. A multilayer perceptron model has a greater processing power and can process linear and non-linear patterns. Further, it also implements logic gates such as AND, OR, XOR, XNOR, and NOR.

Advantages:

- A multi-layered perceptron model can solve complex non-linear problems.
- It works well with both small and large input data.
- Helps us to obtain quick predictions after the training.
- Helps us obtain the same accuracy ratio with big and small data.

Disadvantages:

- In multi-layered perceptron model, computations are time-consuming and complex.
- It is tough to predict how much the dependent variable affects each independent variable.
- The model functioning depends on the quality of training.

Characteristics of the Perceptron Model

1. It is a machine learning algorithm that uses supervised learning of binary classifiers.
2. In Perceptron, the weight coefficient is automatically learned.
3. Initially, weights are multiplied with input features, and then the decision is made whether the neuron is fired or not.
4. The activation function applies a step rule to check whether the function is more significant than zero.
5. The linear decision boundary is drawn, enabling the distinction between the two linearly separable classes +1 and -1.
6. If the added sum of all input values is more than the threshold value, it must have an output signal; otherwise, no output will be shown.

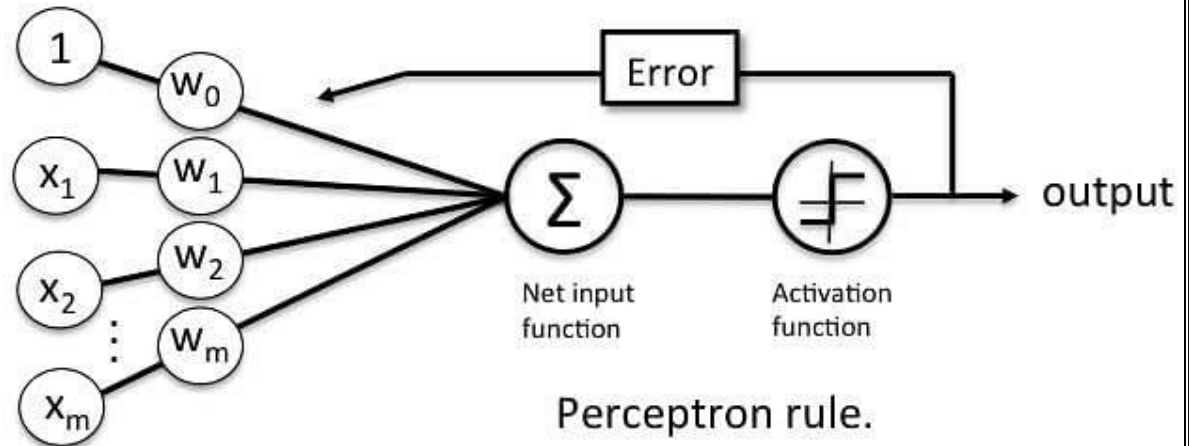
Limitation of Perceptron Model

The following are the limitation of a Perceptron model:

1. The output of a perceptron can only be a binary number (0 or 1) due to the hard-edge transfer function.
2. It can only be used to classify the linearly separable sets of input vectors. If the input vectors are non-linear, it is not easy to classify them correctly.

Perceptron Learning Rule

Perceptron Learning Rule states that the algorithm would automatically learn the optimal weight coefficients. The input features are then multiplied with these weights to determine if a neuron fires or not.



The Perceptron receives multiple input signals, and if the sum of the input signals exceeds a certain threshold, it either outputs a signal or does not return an output. In the context of supervised learning and classification, this can then be used to predict the class of a sample.

Next up, let us focus on the perceptron function.

Perceptron Function

Perceptron is a function that maps its input “x,” which is multiplied with the learned weight coefficient; an output value $f(x)$ is generated.

$$f(x) = \begin{cases} 1 & \text{if } w \cdot x + b > 0 \\ 0 & \text{otherwise} \end{cases}$$

In the equation given above:

- “w” = vector of real-valued weights
- “b” = bias (an element that adjusts the boundary away from origin without any dependence on the input value)
- “x” = vector of input x values

$$\sum_{i=1}^m w_i x_i$$

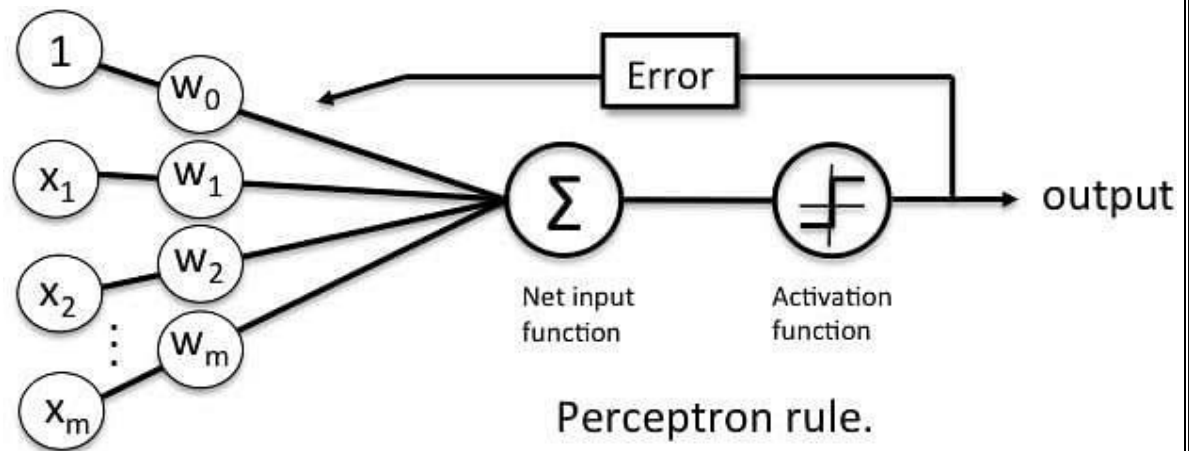
- “m” = number of inputs to the Perceptron

The output can be represented as “1” or “0.” It can also be represented as “1” or “-1” depending on which activation function is used.

Let us learn the inputs of a perceptron in the next section.

Inputs of a Perceptron

A Perceptron accepts inputs, moderates them with certain weight values, then applies the transformation function to output the final result. The image below shows a Perceptron with a Boolean output.



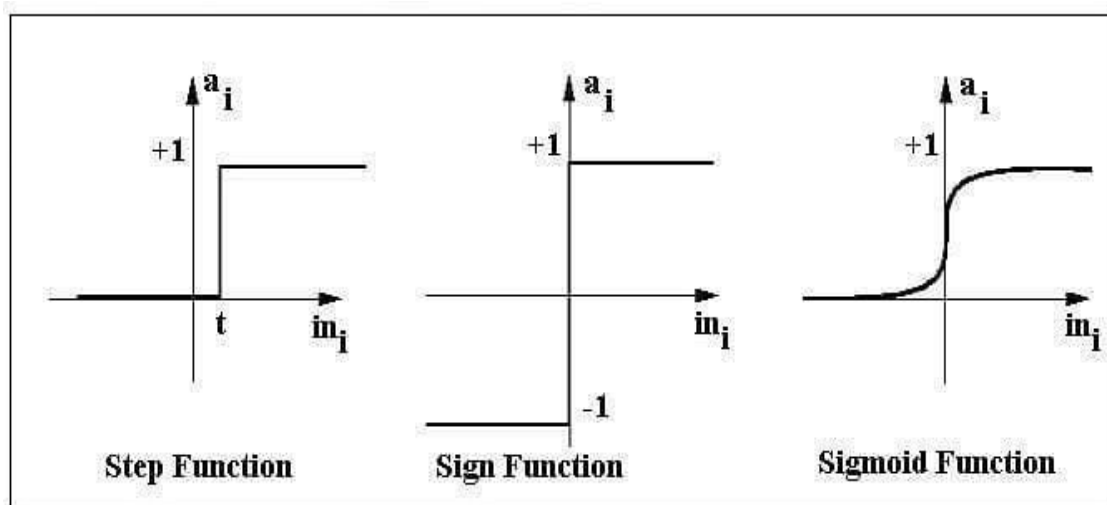
A Boolean output is based on inputs such as salaried, married, age, past credit profile, etc. It has only two values: Yes and No or True and False. The summation function " Σ " multiplies all inputs of "x" by weights "w" and then adds them up as follows:

$$w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$

In the next section, let us discuss the activation functions of perceptrons.

Activation Functions of Perceptron

The activation function applies a step rule (convert the numerical output into +1 or -1) to check if the output of the weighting function is greater than zero or not.



For example:

If $\sum w_i x_i > 0 \Rightarrow$ then final output “o” = 1 (issue bank loan)

Else, final output “o” = -1 (deny bank loan)

Step function gets triggered above a certain value of the neuron output; else it outputs zero. Sign Function outputs +1 or -1 depending on whether neuron output is greater than zero or not. Sigmoid is the S-curve and outputs a value between 0 and 1.

Output of Perceptron

Perceptron with a Boolean output:

Inputs: $x_1 \dots x_n$

Output: $o(x_1 \dots x_n)$

$$o(x_1, \dots, x_n) = \begin{cases} 1 & \text{if } w_0 + w_1 x_1 + w_2 x_2 + \dots + w_n x_n > 0 \\ -1 & \text{otherwise} \end{cases}$$

Weights: $w_i \Rightarrow$ contribution of input x_i to the Perceptron output;

$w_0 \Rightarrow$ bias or threshold

If $\sum w_i x_i > 0$, output is +1, else -1. The neuron gets triggered only when weighted input reaches a certain threshold value.

$$o(\vec{x}) = \text{sgn}(\vec{w} \cdot \vec{x})$$

$$\text{sgn}(y) = \begin{cases} 1 & \text{if } y > 0 \\ -1 & \text{otherwise} \end{cases}$$

An output of +1 specifies that the neuron is triggered. An output of -1 specifies that the neuron did not get triggered.

“sgn” stands for sign function with output +1 or -1.

Error in Perceptron

In the Perceptron Learning Rule, the predicted output is compared with the known output. If it does not match, the error is propagated backward to allow weight adjustment to happen.

Let us discuss the decision function of Perceptron in the next section.

Perceptron: Decision Function

A decision function $\phi(z)$ of Perceptron is defined to take a linear combination of x and w vectors.

$$\mathbf{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix}$$

The value z in the decision function is given by:

$$Z = w_1 x_1 + \dots + w_m x_m$$

The decision function is +1 if z is greater than a threshold θ , and it is -1 otherwise.

$$\phi(z) = \begin{cases} 1 & \text{if } z \geq \theta \\ -1 & \text{otherwise} \end{cases}$$

This is the Perceptron algorithm.

Bias Unit

For simplicity, the threshold θ can be brought to the left and represented as $w_0 x_0$, where $w_0 = -\theta$ and $x_0 = 1$.

$$Z = w_0 x_0 + w_1 x_1 + \dots + w_m x_m = \mathbf{w}^T \mathbf{x}$$

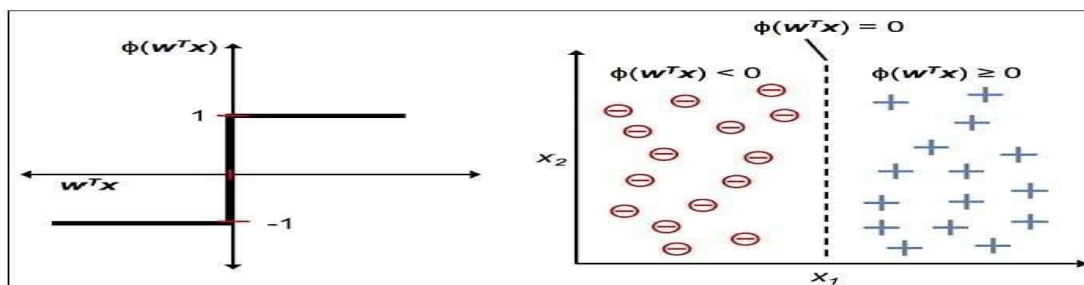
The value w_0 is called the bias unit.

The decision function then becomes:

$$\phi(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ -1 & \text{otherwise} \end{cases}$$

Output:

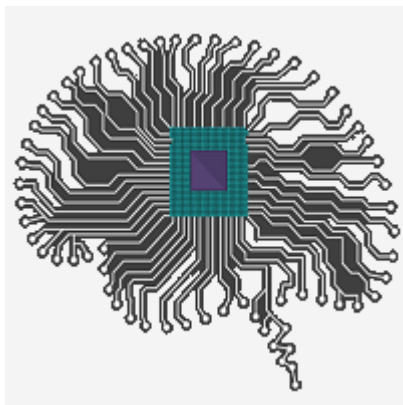
The figure shows how the decision function squashes $w^T x$ to either +1 or -1 and how it can be used to discriminate between two linearly separable classes.



UNIT – I

NEURAL NETWORKS – I

Artificial Neural Network Tutorial



Artificial Neural Network Tutorial provides basic and advanced concepts of ANNs. Our Artificial Neural Network tutorial is developed for beginners as well as professions.

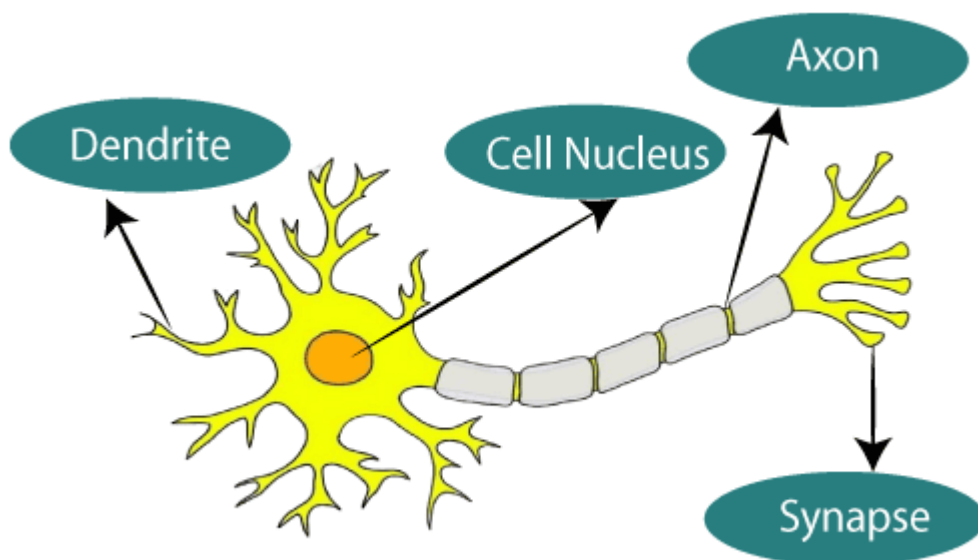
The term "Artificial neural network" refers to a biologically inspired sub-field of artificial intelligence modeled after the brain. An Artificial neural network is usually a computational network based on biological neural networks that construct the structure of the human brain. Similar to a human brain has neurons interconnected to each other, artificial neural networks also have neurons that are linked to each other in various layers of the networks. These neurons are known as nodes.

Artificial neural network tutorial covers all the aspects related to the artificial neural network. In this tutorial, we will discuss ANNs, Adaptive resonance

theory, Kohonen self-organizing map, Building blocks, unsupervised learning, Genetic algorithm, etc.

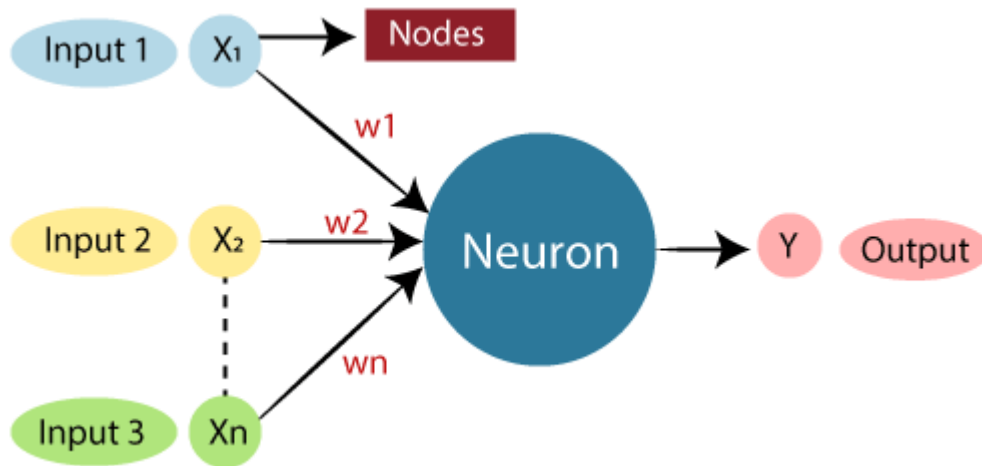
What is Artificial Neural Network?

The term "**Artificial Neural Network**" is derived from Biological neural networks that develop the structure of a human brain. Similar to the human brain that has neurons interconnected to one another, artificial neural networks also have neurons that are interconnected to one another in various layers of the networks. These neurons are known as nodes.



The given figure illustrates the typical diagram of Biological Neural Network.

The typical Artificial Neural Network looks something like the given figure.



Dendrites from Biological Neural Network represent inputs in Artificial Neural Networks, cell nucleus represents Nodes, synapse represents Weights, and Axon represents Output.

Relationship between Biological neural network and artificial neural network:

Biological Neural Network	Artificial Neural Network
Dendrites	Inputs
Cell nucleus	Nodes
Synapse	Weights
Axon	Output

An **Artificial Neural Network** in the field of **Artificial intelligence** where it attempts to mimic the network of neurons makes up a human brain so that computers will have an option to understand things and make decisions in a human-like manner. The artificial neural network is designed by programming computers to behave simply like interconnected brain cells.

There are around 1000 billion neurons in the human brain. Each neuron has an association point somewhere in the range of 1,000 and 100,000. In the human

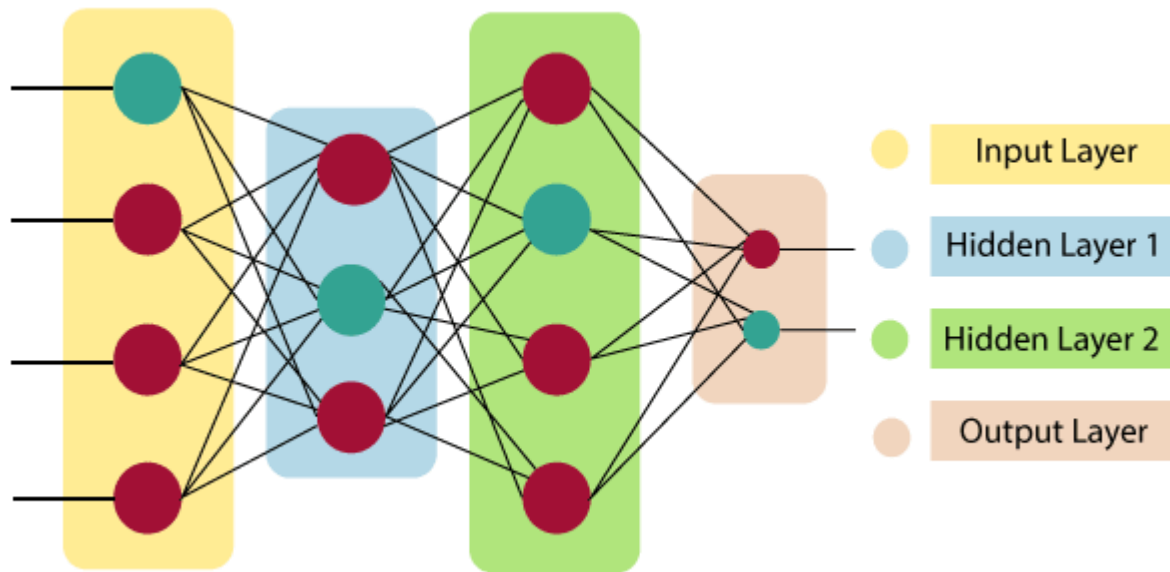
brain, data is stored in such a manner as to be distributed, and we can extract more than one piece of this data when necessary from our memory parallelly. We can say that the human brain is made up of incredibly amazing parallel processors.

We can understand the artificial neural network with an example, consider an example of a digital logic gate that takes an input and gives an output. "OR" gate, which takes two inputs. If one or both the inputs are "On," then we get "On" in output. If both the inputs are "Off," then we get "Off" in output. Here the output depends upon input. Our brain does not perform the same task. The outputs to inputs relationship keep changing because of the neurons in our brain, which are "learning."

The architecture of an artificial neural network:

To understand the concept of the architecture of an artificial neural network, we have to understand what a neural network consists of. In order to define a neural network that consists of a large number of artificial neurons, which are termed units arranged in a sequence of layers. Lets us look at various types of layers available in an artificial neural network.

Artificial Neural Network primarily consists of three layers:



Input Layer:

As the name suggests, it accepts inputs in several different formats provided by the programmer.

Hidden Layer:

The hidden layer presents in-between input and output layers. It performs all the calculations to find hidden features and patterns.

Output Layer:

The input goes through a series of transformations using the hidden layer, which finally results in output that is conveyed using this layer.

The artificial neural network takes input and computes the weighted sum of the inputs and includes a bias. This computation is represented in the form of a transfer function.

$$\sum_{i=1}^n W_i * X_i + b$$

It determines weighted total is passed as an input to an activation function to produce the output. Activation functions choose whether a node should fire or not. Only those who are fired make it to the output layer. There are distinctive activation functions available that can be applied upon the sort of task we are performing.

Advantages of Artificial Neural Network (ANN)

Parallel processing capability:

Artificial neural networks have a numerical value that can perform more than one task simultaneously.

Storing data on the entire network:

Data that is used in traditional programming is stored on the whole network, not on a database. The disappearance of a couple of pieces of data in one place doesn't prevent the network from working.

Capability to work with incomplete knowledge:

After ANN training, the information may produce output even with inadequate data. The loss of performance here relies upon the significance of missing data.

Having a memory distribution:

For ANN is to be able to adapt, it is important to determine the examples and to encourage the network according to the desired output by demonstrating these examples to the network. The succession of the network is directly proportional to the chosen instances, and if the event can't appear to the network in all its aspects, it can produce false output.

Having fault tolerance:

Extortion of one or more cells of ANN does not prohibit it from generating output, and this feature makes the network fault-tolerance.

Disadvantages of Artificial Neural Network:

Assurance of proper network structure:

There is no particular guideline for determining the structure of artificial neural networks. The appropriate network structure is accomplished through experience, trial, and error.

Unrecognized behavior of the network:

It is the most significant issue of ANN. When ANN produces a testing solution, it does not provide insight concerning why and how. It decreases trust in the network.

Hardware dependence:

Artificial neural networks need processors with parallel processing power, as per their structure. Therefore, the realization of the equipment is dependent.

Difficulty of showing the issue to the network:

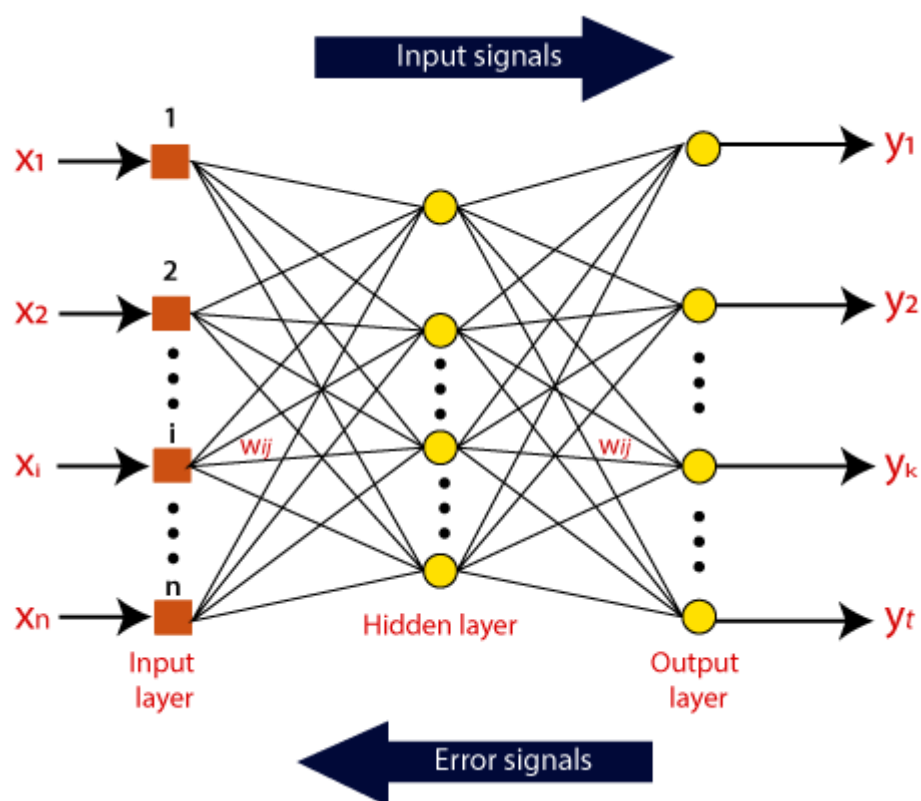
ANNs can work with numerical data. Problems must be converted into numerical values before being introduced to ANN. The presentation mechanism to be resolved here will directly impact the performance of the network. It relies on the user's abilities.

The duration of the network is unknown:

The network is reduced to a specific value of the error, and this value does not give us optimum results.

How do artificial neural networks work?

Artificial Neural Network can be best represented as a weighted directed graph, where the artificial neurons form the nodes. The association between the neurons outputs and neuron inputs can be viewed as the directed edges with weights. The Artificial Neural Network receives the input signal from the external source in the form of a pattern and image in the form of a vector. These inputs are then mathematically assigned by the notations $x(n)$ for every n number of inputs.



Afterward, each of the input is multiplied by its corresponding weights (these weights are the details utilized by the artificial neural networks to solve a specific problem). In general terms, these weights normally represent the strength of the interconnection between neurons inside the artificial neural network. All the weighted inputs are summarized inside the computing unit.

If the weighted sum is equal to zero, then bias is added to make the output non-zero or something else to scale up to the system's response. Bias has the same

input, and weight equals to 1. Here the total of weighted inputs can be in the range of 0 to positive infinity. Here, to keep the response in the limits of the desired value, a certain maximum value is benchmarked, and the total of weighted inputs is passed through the activation function.

The activation function refers to the set of transfer functions used to achieve the desired output. There is a different kind of the activation function, but primarily either linear or non-linear sets of functions. Some of the commonly used sets of activation functions are the Binary, linear, and Tan hyperbolic sigmoidal activation functions

Types of Artificial Neural Network:

There are various types of Artificial Neural Networks (ANN) depending upon the human brain neuron and network functions, an artificial neural network similarly performs tasks. The majority of the artificial neural networks will have some similarities with a more complex biological partner and are very effective at their expected tasks. For example, segmentation or classification.

Feedback ANN:

In this type of ANN, the output returns into the network to accomplish the best-evolved results internally. As per the **University of Massachusetts**, Lowell Centre for Atmospheric Research. The feedback networks feed information back into itself and are well suited to solve optimization issues. The Internal system error corrections utilize feedback ANNs.

Feed-Forward ANN:

A feed-forward network is a basic neural network comprising of an input layer, an output layer, and at least one layer of a neuron. Through assessment of its output by reviewing its input, the intensity of the network can be noticed based

on group behavior of the associated neurons, and the output is decided. The primary advantage of this network is that it figures out how to evaluate and recognize input patterns.

What is Activation Function?

*It's just a thing function that you use to get the output of node. It is also known as **Transfer Function**.*

Why we use Activation functions with Neural Networks?

It is used to determine the output of neural network like yes or no. It maps the resulting values in between 0 to 1 or -1 to 1 etc. (depending upon the function).

The Activation Functions can be basically divided into 2 types-

1. Linear Activation Function
2. Non-linear Activation Functions

Linear or Identity Activation Function

As you can see the function is a line or linear. Therefore, the output of the functions will not be confined between any range.

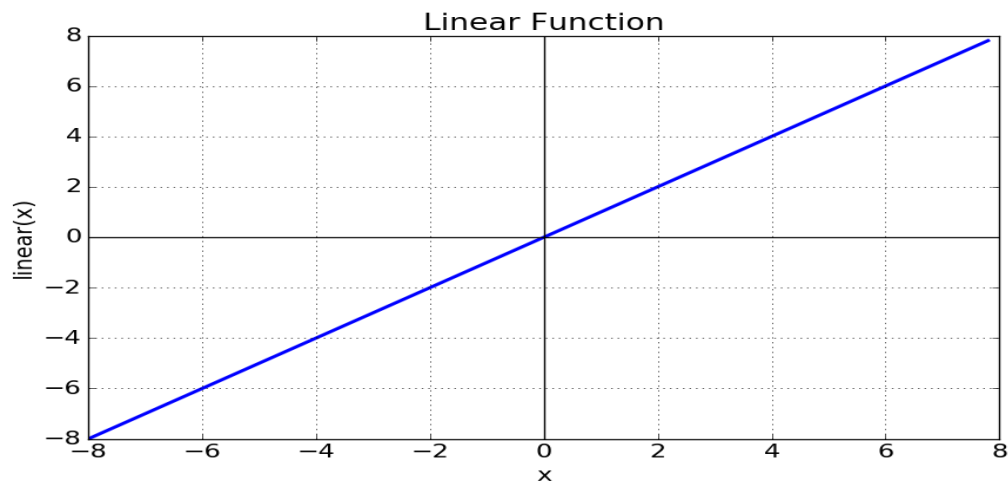


Fig: Linear Activation Function

Equation : $f(x) = x$

Range : (-infinity to infinity)

It doesn't help with the complexity or various parameters of usual data that is fed to the neural networks.

Non-linear Activation Function

The Nonlinear Activation Functions are the most used activation functions. Nonlinearity helps to makes the graph look something like this

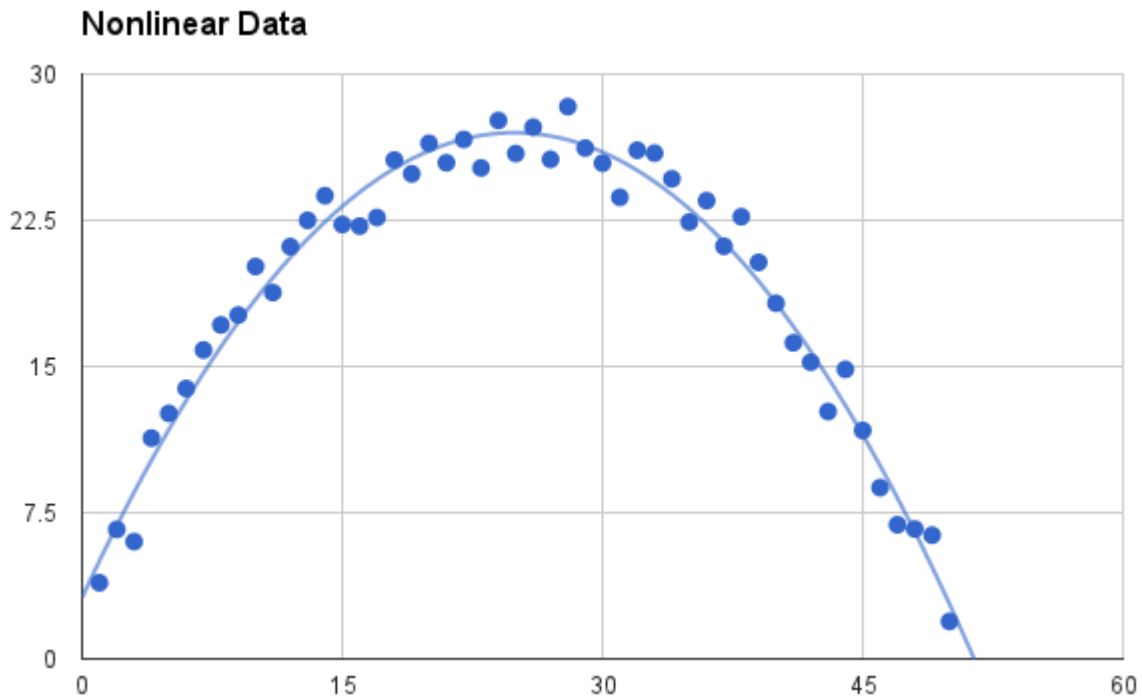


Fig: Non-linear Activation Function

It makes it easy for the model to generalize or adapt with variety of data and to differentiate between the output.

The main terminologies needed to understand for nonlinear functions are:

Derivative or Differential: Change in y -axis w.r.t. change in x -axis. It is also known as slope.

Monotonic function: A function which is either entirely non-increasing or non-decreasing.

The Nonlinear Activation Functions are mainly divided on the basis of their **range** or **curves**-

1. Sigmoid or Logistic Activation Function

The Sigmoid Function curve looks like a S-shape.

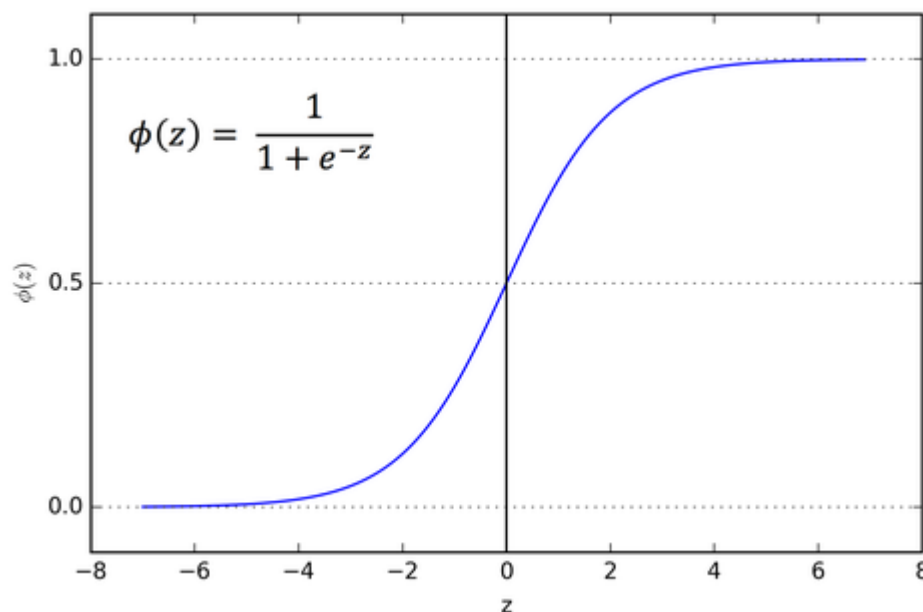


Fig: Sigmoid Function

The main reason why we use sigmoid function is because it exists between **(0 to 1)**. Therefore, it is especially used for models where we have to **predict the probability** as an output. Since probability of anything exists only between the range of **0 and 1**, sigmoid is the right choice.

The function is **differentiable**. That means, we can find the slope of the sigmoid curve at any two points.

The function is **monotonic** but function's derivative is not.

The logistic sigmoid function can cause a neural network to get stuck at the training time.

The **softmax function** is a more generalized logistic activation function which is used for multiclass classification.

2. Tanh or hyperbolic tangent Activation Function

tanh is also like logistic sigmoid but better. The range of the tanh function is from (-1 to 1). tanh is also sigmoidal (s - shaped).

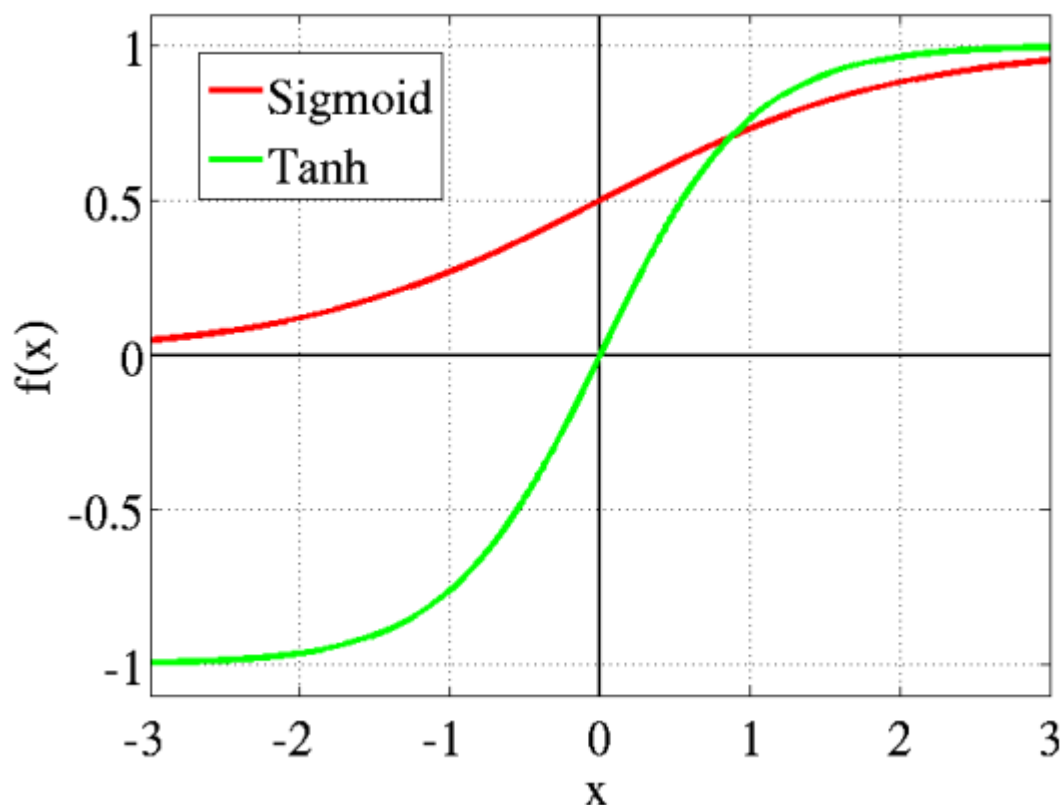


Fig: tanh v/s Logistic Sigmoid

The advantage is that the negative inputs will be mapped strongly negative and the zero inputs will be mapped near zero in the tanh graph.

The function is **differentiable**.

The function is **monotonic** while its **derivative is not monotonic**.

The tanh function is mainly used classification between two classes.

Both tanh and logistic sigmoid activation functions are used in feed-forward nets.

3. ReLU (Rectified Linear Unit) Activation Function

The ReLU is the most used activation function in the world right now. Since, it is used in almost all the convolutional neural networks or deep learning.

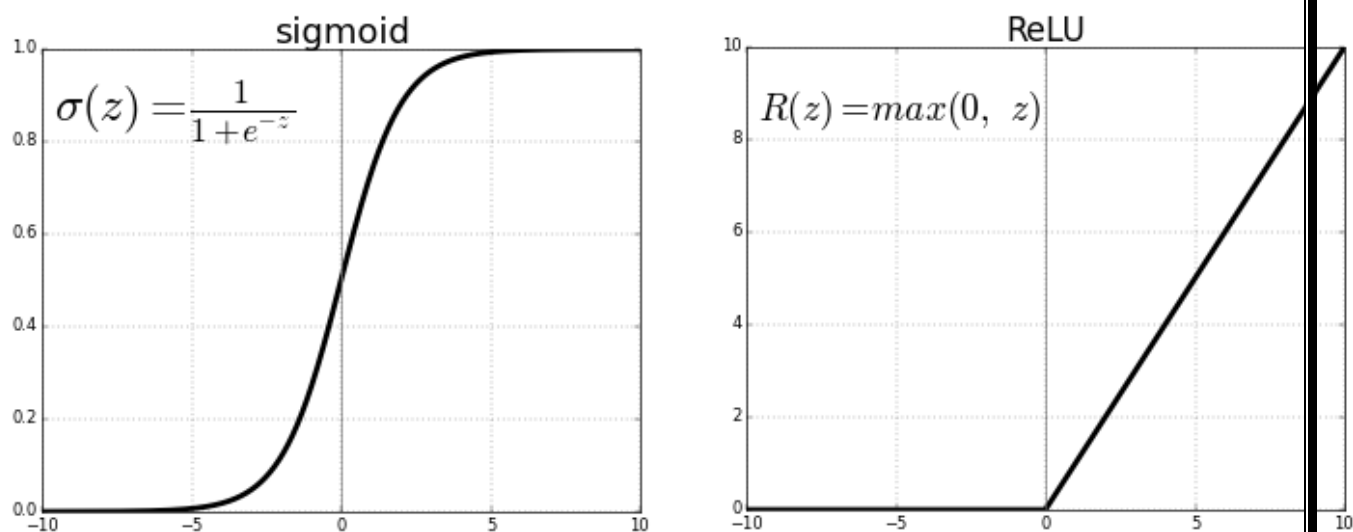


Fig: ReLU v/s Logistic Sigmoid

As you can see, the ReLU is half rectified (from bottom). $f(z)$ is zero when z is less than zero and $f(z)$ is equal to z when z is above or equal to zero.

Range: [0 to infinity)

The function and its derivative **both are monotonic**.

But the issue is that all the negative values become zero immediately which decreases the ability of the model to fit or train from the data properly. That means any negative input given to the ReLU activation function turns the value into zero

immediately in the graph, which in turns affects the resulting graph by not mapping the negative values appropriately.

4. Leaky ReLU

It is an attempt to solve the dying ReLU problem

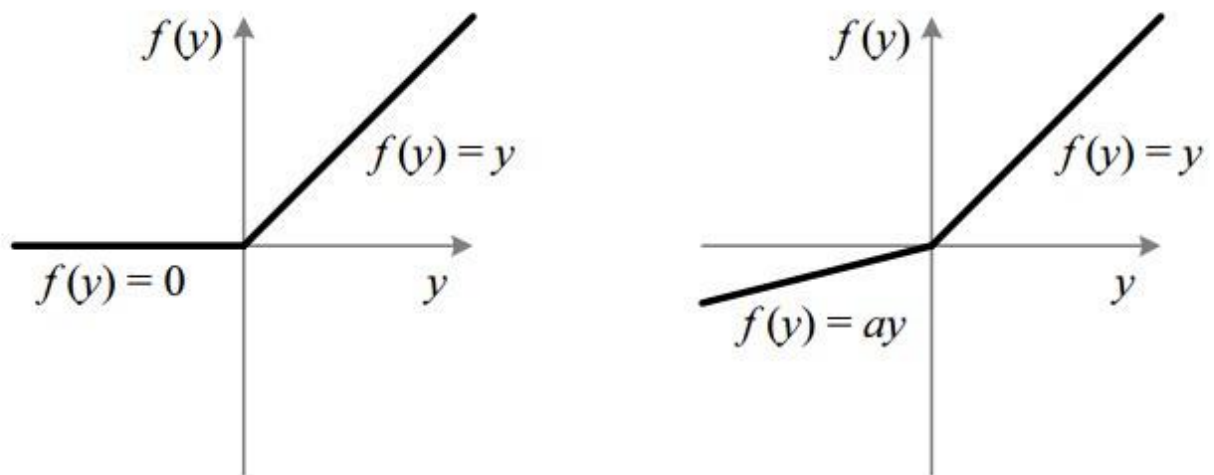


Fig : ReLU v/s Leaky ReLU

Can you see the Leak? 😊

The leak helps to increase the range of the ReLU function. Usually, the value of **a** is 0.01 or so.

When **a** is **not 0.01** then it is called **Randomized ReLU**.

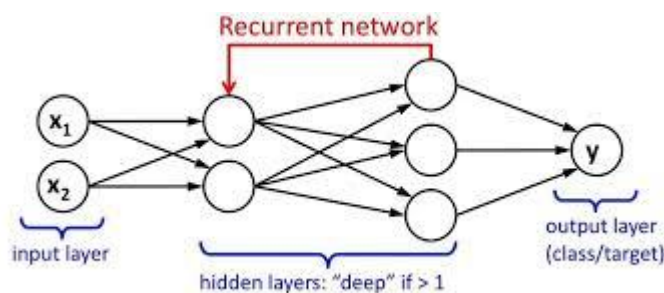
Therefore the **range** of the Leaky ReLU is (-infinity to infinity).

Both Leaky and Randomized ReLU functions are monotonic in nature. Also, their derivatives also monotonic in nature.

Recurrent Neural Network

Recurrent Neural Network(RNN) are a type of Neural Network where the **output from previous step are fed as input to the current step**. In traditional neural networks, all the inputs and outputs are independent of each other, but in cases like when it is required to predict the next word of a sentence, the previous words are required and hence there is a need to remember the previous words. Thus RNN came into existence, which solved this issue with the help of a Hidden Layer. The main and most important feature of RNN is **Hidden state**, which remembers some information about a sequence.

RNN have a “**memory**” which remembers all information about what has been calculated. It uses the same parameters for each input as it performs the same task on all the inputs or hidden layers to produce the output. This reduces the complexity of parameters, unlike other neural networks.



Training through RNN

1. A single time step of the input is provided to the network.
2. Then calculate its current state using set of current input and the previous state.
3. The current h_t becomes h_{t-1} for the next time step.

4. One can go as many time steps according to the problem and join the information from all the previous states.
5. Once all the time steps are completed the final current state is used to calculate the output.
6. The output is then compared to the actual output i.e the target output and the error is generated.
7. The error is then back-propagated to the network to update the weights and hence the network (RNN) is trained.

Advantages of Recurrent Neural Network

1. An RNN remembers each and every information through time. It is useful in time series prediction only because of the feature to remember previous inputs as well. This is called Long Short Term Memory.
2. Recurrent neural network are even used with convolutional layers to extend the effective pixel neighbourhood.

Disadvantages of Recurrent Neural Network

1. Gradient vanishing and exploding problems.
2. Training an RNN is a very difficult task.
3. It cannot process very long sequences if using tanh or relu as an activation function.

Hopfield Neural Networks

The Hopfield Neural Networks, invented by Dr John J. Hopfield consists of one layer of ' n ' fully connected recurrent neurons. It is generally used in performing auto association and optimization tasks. It is calculated using a converging interactive process and it generates a different response than our normal neural nets.

Discrete Hopfield Network: It is a fully interconnected neural network where each unit is connected to every other unit. It behaves in a discrete manner, i.e. it gives finite distinct output, generally of two types:

- **Binary (0/1)**
- **Bipolar (-1/1)**

The weights associated with this network is symmetric in nature and has the following properties.

Structure & Architecture

- Each neuron has an inverting and a non-inverting output.
- Being fully connected, the output of each neuron is an input to all other neurons but not self.

Fig 1 shows a sample representation of a Discrete Hopfield Neural Network architecture having the following elements.

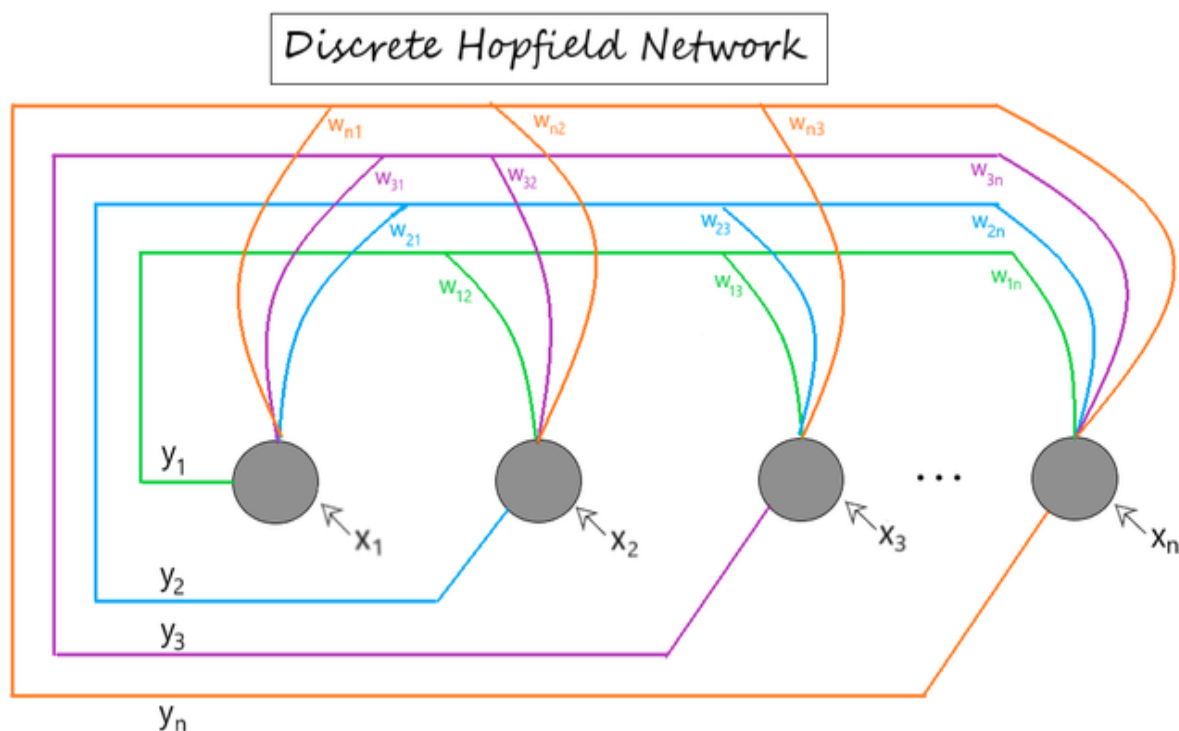


Fig 1: Discrete Hopfield Network Architecture

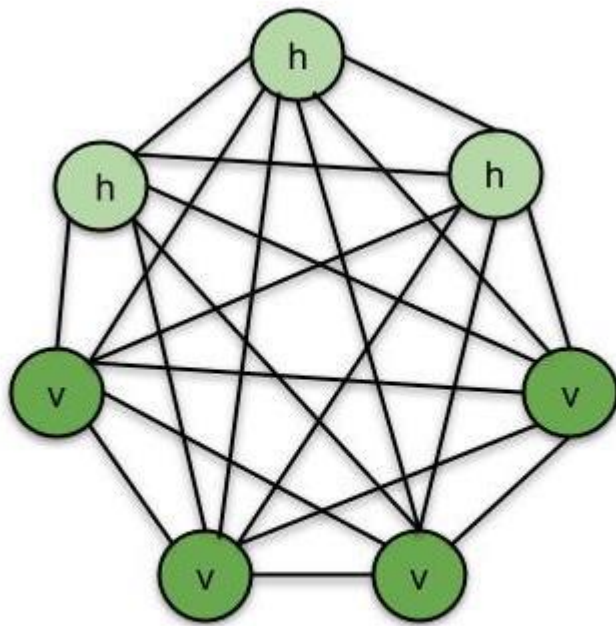
[$x_1, x_2, \dots, x_n$] -> Input to the n given neurons.

[$y_1, y_2, \dots, y_n$] -> Output obtained from the n given neurons

W_{ij} -> weight associated with the connection between the i^{th} and the j^{th} n

Boltzmann Machines

Boltzmann Machines is an unsupervised DL model in which every node is connected to every other node. That is, unlike the ANNs, CNNs, RNNs and SOMs, the Boltzmann Machines are **undirected** (or the connections are bidirectional). Boltzmann Machine is not a deterministic DL model but a **stochastic** or **generative** DL model. It is rather a representation of a certain system. There are two types of nodes in the Boltzmann Machine — **Visible nodes** — those nodes which we can and do measure, and the **Hidden nodes** — those nodes which we cannot or do not measure. Although the node types are different, the Boltzmann machine considers them as the same and everything works as one single system. The training data is fed into the Boltzmann Machine and the weights of the system are adjusted accordingly. Boltzmann machines help us understand abnormalities by learning about the working of the system in normal conditions.



v - visible nodes, h - hidden nodes

Boltzmann Machine

These kinds of neural networks work on the basis of pattern association, which means they can store different patterns and at the time of giving an output they can produce one of the stored patterns by matching them with the given input pattern. These types of memories are also called **Content-Addressable Memory** CAM/CAM. Associative memory makes a parallel search with the stored patterns as data files.

Following are the two types of associative memories we can observe –

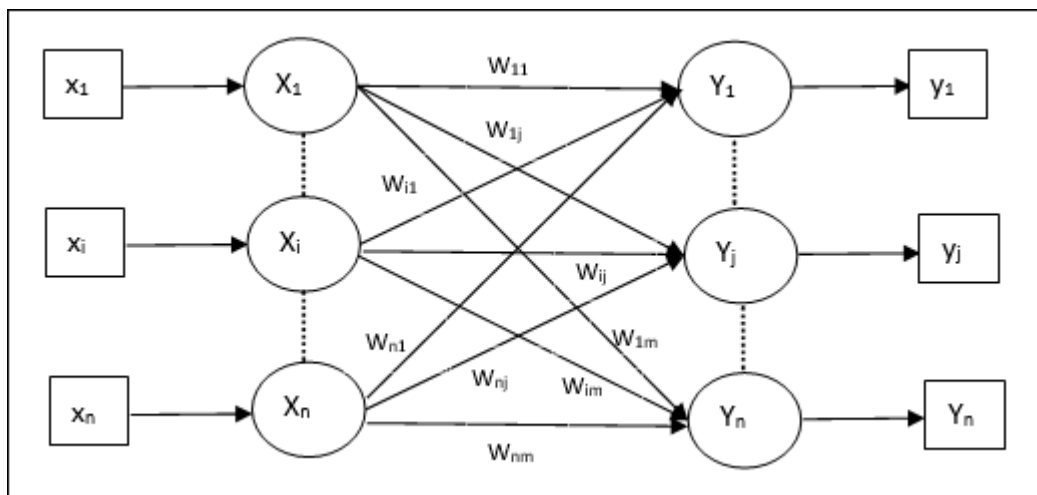
- Auto Associative Memory
- Hetero Associative memory

Auto Associative Memory

This is a single layer neural network in which the input training vector and the output target vectors are the same. The weights are determined so that the network stores a set of patterns.

Architecture

As shown in the following figure, the architecture of Auto Associative memory network has ' n ' number of input training vectors and similar ' n ' number of output target vectors.

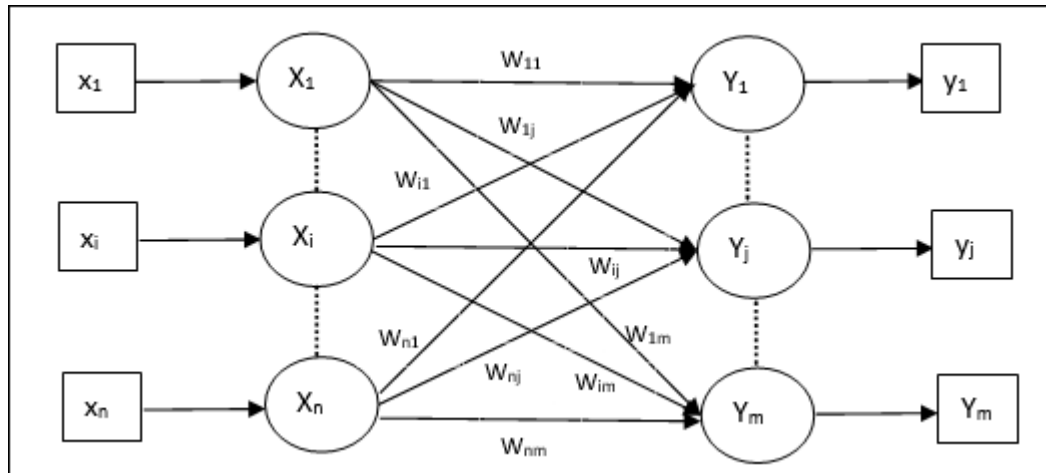


Hetero Associative memory

Similar to Auto Associative Memory network, this is also a single layer neural network. However, in this network the input training vector and the output target vectors are not the same. The weights are determined so that the network stores a set of patterns. Hetero associative network is static in nature, hence, there would be no non-linear and delay operations.

Architecture

As shown in the following figure, the architecture of Hetero Associative Memory network has ‘ n ’ number of input training vectors and ‘ m ’ number of output target vectors.



Hebb's law

Hebbian Learning Rule, also known as Hebb Learning Rule, was proposed by Donald O Hebb. It is one of the first and also easiest learning rules in the neural network. It is used for pattern classification. It is a single layer neural network, i.e. it has one input layer and one output layer. The input layer can have many units, say n . The output layer only has one unit. Hebbian rule works by updating the weights between neurons in the neural network for each training sample.

Hebbian Learning Rule Algorithm :

1. Set all weights to zero, $w_i = 0$ for $i=1$ to n , and bias to zero.
2. For each input vector, $S(\text{input vector}) : t(\text{target output pair})$, repeat steps 3-5.
3. Set activations for input units with the input vector $X_i = S_i$ for $i = 1$ to n .
4. Set the corresponding output value to the output neuron, i.e. $y = t$.
5. Update weight and bias by applying Hebb rule for all $i = 1$ to n :

$$w_i (\text{new}) = w_i (\text{old}) + x_i y$$

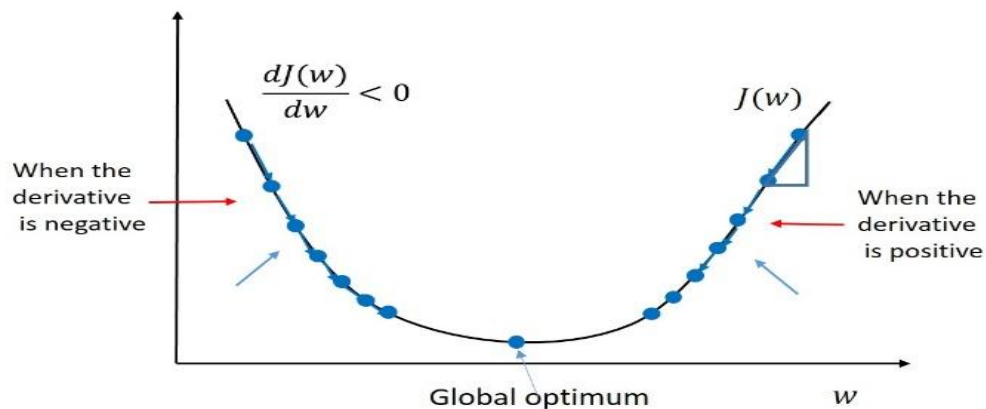
$$b (\text{new}) = b (\text{old}) + y$$

Gradient Descent is an optimization algorithm used for minimizing the cost function in various machine learning algorithms. It is basically used for updating the parameters of the learning model.

Types of gradient Descent:

1. **Batch Gradient Descent:** This is a type of gradient descent which processes all the training examples for each iteration of gradient descent. But if the number of training examples is large, then batch gradient descent is computationally very expensive. Hence if the number of training examples is large, then batch gradient descent is not preferred. Instead, we prefer to use stochastic gradient descent or mini-batch gradient descent.
2. **Stochastic Gradient Descent:** This is a type of gradient descent which processes 1 training example per iteration. Hence, the parameters are being updated even after one iteration in which only a single example has been processed. Hence this is quite faster than batch gradient descent. But again, when the number of training examples is large, even then it processes only one example which can be additional overhead for the system as the number of iterations will be quite large.
3. **Mini Batch gradient descent:** This is a type of gradient descent which works faster than both batch gradient descent and stochastic gradient descent. Here b examples where $b < m$ are processed per iteration. So even if the number of training examples is large, it is processed in batches of b training

examples in one go. Thus, it works for larger training examples and that too with lesser number of iterations.



Perceptron Convergence Theorem:

Perceptron Convergence theorem states that a classifier for two linearly separable classes of patterns is always trainable in a finite number of training steps.

(Or)

Perceptron Convergence Theorem: For any finite set of linearly separable labelled examples, the Perceptron Learning Algorithm will halt after a finite number of iterations.

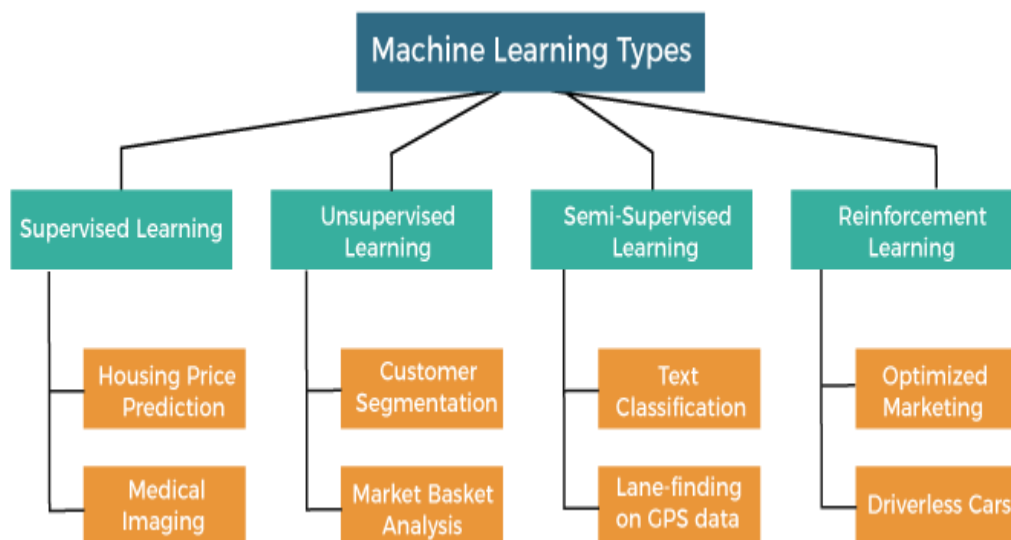
In other words, after a finite number of iterations, the algorithm yields a vector w that classifies perfectly all the examples.

Note: Although the number of iterations is finite, it is usually larger than the size of the training set, because each example needs to be processed more than once.

TYPES OF LEARNINGS IN NEURAL NETWORKS

Based on the methods and way of learning, machine learning is divided into mainly four types, which are:

1. Supervised Machine Learning
2. Unsupervised Machine Learning
3. Semi-Supervised Machine Learning
4. Reinforcement Learning



Supervised Machine Learning

As its name suggests, Supervised machine learning is based on supervision. It means in the supervised learning technique, we train the machines using the "labelled" dataset, and based on the training, the machine predicts the output. Here, the labelled data specifies that some of the inputs are already mapped to the output. More precisely, we can say; first, we train the machine with the input and corresponding output, and then we ask the machine to predict the output using the test dataset.

Let's understand supervised learning with an example. Suppose we have an input dataset of cats and dog images. So, first, we will provide the training to the machine to understand the images, such as the **shape & size of the tail of cat and dog, Shape of eyes, colour, height (dogs are taller, cats are smaller), etc.** After completion of training, we input the picture of a cat and ask the machine to identify the object and predict the output. Now, the machine is well trained, so it will check all the features of the object, such as height, shape, colour, eyes, ears, tail, etc., and find that it's a cat. So, it will put it in the Cat category. This is the process of how the machine identifies the objects in Supervised Learning.

The main goal of the supervised learning technique is to map the input variable(x) with the output variable(y). Some real-world applications of supervised learning are **Risk Assessment, Fraud Detection, Spam filtering**, etc.

Categories of Supervised Machine Learning

Supervised machine learning can be classified into two types of problems, which are given below:

- **Classification**
- **Regression**

a) Classification

Classification algorithms are used to solve the classification problems in which the output variable is categorical, such as **"Yes" or No, Male or Female, Red or Blue, etc.** The classification algorithms predict the categories present in the dataset. Some real-world examples of classification algorithms are **Spam Detection, Email filtering, etc.**

b) Regression

Regression algorithms are used to solve regression problems in which there is a linear relationship between input and output variables. These are used to predict continuous output variables, such as market trends, weather prediction, etc.

Applications of Supervised Learning

Some common applications of Supervised Learning are given below:

- **Image Segmentation**
- **Medical Diagnosis**
- **Fraud Detection**
- **Spam detection**
- **Speech Recognition**

Unsupervised Machine Learning

Unsupervised learning is different from the Supervised learning technique; as its name suggests, there is no need for supervision. It means, in unsupervised machine learning, the machine is trained using the unlabeled dataset, and the machine predicts the output without any supervision.

In unsupervised learning, the models are trained with the data that is neither classified nor labelled, and the model acts on that data without any supervision.

The main aim of the unsupervised learning algorithm is to group or categories the unsorted dataset according to the similarities, patterns, and differences. Machines are instructed to find the hidden patterns from the input dataset.

Let's take an example to understand it more preciously; suppose there is a basket of fruit images, and we input it into the machine learning model. The images are

totally unknown to the model, and the task of the machine is to find the patterns and categories of the objects.

So, now the machine will discover its patterns and differences, such as colour difference, shape difference, and predict the output when it is tested with the test dataset.

Categories of Unsupervised Machine Learning

Unsupervised Learning can be further classified into two types, which are given below:

- **Clustering**
- **Association**

1) Clustering

The clustering technique is used when we want to find the inherent groups from the data. It is a way to group the objects into a cluster such that the objects with the most similarities remain in one group and have fewer or no similarities with the objects of other groups. An example of the clustering algorithm is grouping the customers by their purchasing behaviour.

2) Association

Association rule learning is an unsupervised learning technique, which finds interesting relations among variables within a large dataset. The main aim of this learning algorithm is to find the dependency of one data item on another data item and map those variables accordingly so that it can generate maximum profit. This algorithm is mainly applied in **Market Basket analysis, Web usage mining, continuous production**, etc.

Some popular algorithms of Association rule learning are **Apriori Algorithm, Eclat, FP-growth algorithm.**

- Reinforcement Learning

Reinforcement learning works on a feedback-based process, in which an AI agent (A software component) automatically explore its surrounding by hitting & trail, taking action, learning from experiences, and improving its performance. Agent gets rewarded for each good action and get punished for each bad action; hence the goal of reinforcement learning agent is to maximize the rewards.

REFERENCES:

- <https://www.javatpoint.com/artificial-neural-network>
- <https://www.geeksforgeeks.org/hopfield-neural-network/>
- <https://www.geeksforgeeks.org/hebbian-learning-rule-with-implementation-of-and-gate/>
- Internet
- https://web.mit.edu/course/other/i2course/www/vision_and_learning/perceptron_notes.pdf
- <https://www.javatpoint.com/types-of-machine-learning>

Backpropagation, Chain Rule and Gradient Flow

Introduction

In the previous stages of neural network learning, we learned that a neural network contains weights and biases that determine its predictions.

However, simply having weights is not enough. The network needs a mechanism to learn the appropriate values of these weights.

This is where backpropagation comes in.

Suppose we give an image of a cat to a neural network. Initially, the network may predict:

Cat: 30%

Dog: 70%

The prediction is incorrect. The network needs to understand:

- How wrong was the prediction?
- Which parameters contributed to the error?
- Which parameters should be increased?
- Which parameters should be decreased?
- By how much should they change?

Backpropagation provides the information needed to answer these questions.

What is Backpropagation?

Backpropagation is an algorithm used to calculate the gradients of a neural network's loss with respect to its weights and biases.

In simpler words:

Backpropagation tells the neural network how its parameters contributed to the prediction error.

The process consists of two major directions:

Forward direction

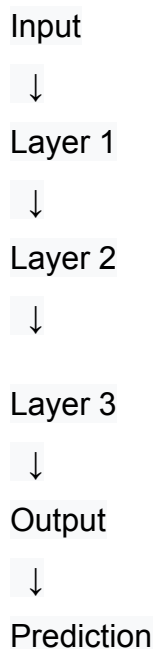
Information moves from: Input→Hidden Layers→Output

Backward direction

Error information moves from: Loss→Output Layer→Hidden Layers

Why Do We Need Backpropagation?

Consider a neural network with three layers.



Suppose the prediction is wrong.

The network needs to adjust the parameters of all three layers.

But the parameters in Layer 1 do not directly produce the final prediction. Their effect passes through Layer 2 then Layer 3 and then the output layer.

For example:

Weight in Layer 1



Neuron output



Layer 2



Layer 3



Final prediction



Loss

Therefore, we need to determine how a change in the Layer 1 weight eventually affects the final loss.

This is the fundamental problem solved by backpropagation.

Forward Propagation

Before discussing the backward pass, we need to understand the forward pass.

During forward propagation, the input travels through the network.

For a simple neuron:

$$z = wx + b$$

where:

- x = input
- w = weight
- b = bias
- z = weighted input

Then an activation function is applied:

$$a = f(z)$$

The output a is passed to the next layer.

For example:

Input



Weighted Sum



Activation Function



Output

This process continues until the network produces its final prediction.

Calculating the Loss

After the network produces a prediction, we need to determine how good or bad that prediction is.

This is the purpose of the loss function.

For example, suppose:

Actual value = 1

Predicted value = 0.7

The prediction is not perfect, so the loss function produces an error value.

The exact loss function depends on the problem.

Common examples include:

- Mean Squared Error (MSE)
- Binary Cross-Entropy
- Categorical Cross-Entropy

For our discussion, we don't need to focus heavily on the mathematics of the loss function.

The important idea is:

The loss tells us how well the network performed.

A smaller loss generally means a better prediction.

What Happens After Calculating the Loss?

Now the network knows:

"My prediction was not correct enough."

But this information alone is not sufficient.

The network must determine:

Which parameters caused the error?

This is where backpropagation begins.

The error information is propagated backward through the network.

Loss



Output Layer



Hidden Layer



Earlier Hidden Layer



Parameters

At every layer, backpropagation determines how much that layer contributed to the final error.

The Chain Rule

The chain rule is the mathematical foundation of backpropagation.

The basic idea is simply:

When one quantity affects another quantity through several intermediate steps, we can determine the overall effect by combining the effects at each step.

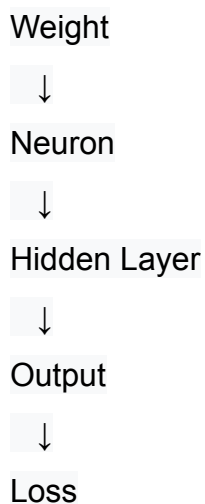
Consider:

$$A \rightarrow B \rightarrow C \rightarrow D$$

If A changes B, B changes C, and C changes D, then a change in A eventually affects D.

The chain rule allows us to trace this effect.

In a neural network:



The chain rule allows us to determine:

How much did that weight ultimately affect the loss?

A Small Mathematical Example

Consider:

$$z=wx+b$$

followed by an activation function:

$$a=f(z)$$

and finally a loss:

$$L=\text{Loss}(a,y)$$

We want to know:

$$\partial w / \partial L$$

Instead of calculating it all at once, we break it into smaller steps:

$$\partial w / \partial L = \partial a / \partial L \times \partial z / \partial a \times \partial w / \partial z$$

This is the chain rule.

The important idea is:

Backpropagation breaks a complicated calculation into smaller local calculations and connects them using the chain rule.

What is a Gradient?

A gradient tells us how much the loss changes when a particular parameter changes.

Suppose we have a weight w .

Its gradient is:

$$\partial w / \partial L$$

We can interpret this as:

"If I change this weight slightly, how will the loss change?"

The gradient has two important pieces of information:

Magnitude

How strongly does the parameter affect the loss?

Direction

Should the parameter move upward or downward to reduce the loss?

Simple Interpretation of a Gradient

Suppose two weights have gradients:

Weight A $\rightarrow$ gradient = large

Weight B $\rightarrow$ gradient = small

This means that Weight A has a stronger immediate influence on the loss than Weight B.

Therefore, the optimizer may make a larger adjustment to Weight A.

This allows different parameters to be updated by different amounts.

Backpropagation and Gradient Descent

Backpropagation and gradient descent are related, but they are not the same thing.

Backpropagation

Calculates the gradients.

Gradient Descent

Uses those gradients to update the parameters.

The process is:

Prediction



Loss



Backpropagation



Gradients



Gradient Descent / Adam



Updated Weights

A simple gradient descent update is:

$$w_{\text{new}} = w_{\text{old}} - \eta(\partial w / \partial L)$$

where:

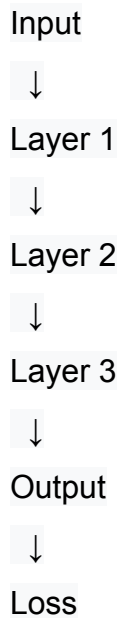
- w = weight
- η = learning rate
- $\partial w / \partial L$ = gradient

Backpropagation calculates; the optimizer updates.

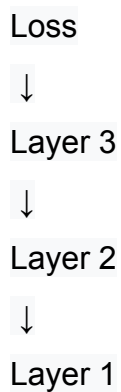
Gradient Flow

Gradient flow refers to the movement of gradient information backward through the network during backpropagation.

Consider:



The gradient flows in the opposite direction:



Every layer uses the incoming gradient to determine how its own parameters should be adjusted.

Why Gradient Flow Matters

Imagine a network with 100 layers.

The gradient has to travel backward through many layers.

Loss



Layer 100



Layer 99



Layer 98



...



Layer 2



Layer 1

If the gradient is preserved properly, the early layers can learn.

But if the gradient becomes too small or too large while traveling backward, training becomes difficult.

This gives us two important problems:

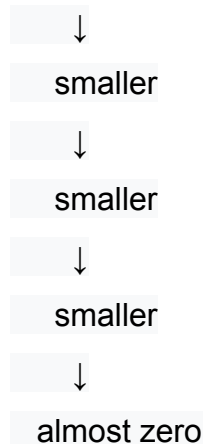
- Vanishing gradient
- Exploding gradient

Vanishing Gradient Problem

The vanishing gradient problem occurs when gradients become extremely small as they move backward through a deep network.

Conceptually:

Gradient at output



Eventually, the early layers receive almost no useful learning signal.

As a result:

- early layers learn very slowly,
- training becomes inefficient,
- the network may fail to learn useful representations.

Why Does the Gradient Vanish?

During backpropagation, gradients are repeatedly transformed as they pass through different layers.

If each transformation reduces the gradient slightly, the effect accumulates over many layers.

For example, imagine a gradient is repeatedly reduced by half:

```
graph TD; A[1] --> B[0.5]; B --> C[0.25]; C --> D[0.125]; D --> E[0.0625]; E --> F[...];
```

1
↓
0.5
↓
0.25
↓
0.125
↓
0.0625
↓
...

After many layers, it can become extremely small.

This is the basic intuition behind the vanishing gradient problem.

Exploding Gradient Problem

The opposite problem is the exploding gradient problem.

Here, the gradient becomes increasingly large as it moves backward.

Conceptually:

Gradient



Larger



Much larger



Extremely large

This can result in:

- very large weight updates,
- unstable training,
- numerical problems,
- failure to converge.

Therefore, a good neural network needs a healthy gradient flow.

Role of Activation Functions

Activation functions have a significant impact on gradient flow.

Common activation functions include:

- Sigmoid
- Tanh
- ReLU
- Leaky ReLU
- GELU

Older networks frequently used sigmoid and tanh.

However, sigmoid can produce very small gradients in certain regions.

This can make gradient propagation difficult in deep networks.

ReLU

One of the most widely used activation functions is ReLU.

Its basic behavior is:

Input $< 0 \rightarrow$ Output = 0

Input $> 0 \rightarrow$ Output = Input

ReLU became popular partly because it generally allows better gradient propagation for positive activations than sigmoid.

However, it can also suffer from the dying ReLU problem, where some neurons can become inactive and stop contributing useful gradients.

How Modern Networks Improve Gradient Flow

Modern deep learning architectures use several techniques to make training deep networks easier.

Some important techniques include:

1. Better activation functions

Examples:

- ReLU
- Leaky ReLU
- GELU

2. Proper weight initialization

Weights need to be initialized appropriately so that activations and gradients do not become excessively large or small.

3. Normalization

Techniques such as Batch Normalization and Layer Normalization can help stabilize training.

4. Residual connections

Used famously in ResNet, residual connections provide shortcut paths through the network.

5. Appropriate optimization algorithms

Methods such as Adam can make parameter updates more effective.

Backpropagation in CNNs

Backpropagation is especially important for Convolutional Neural Networks.

Consider an image classification network:

Image



Convolution



Activation



Pooling



Convolution



Fully Connected Layer



Prediction



Loss

During backpropagation:

Loss



Fully Connected Layer



Convolution



Pooling



Earlier Convolution

The gradients are used to update the convolution filters and other parameters.
Over many training iterations, these filters learn useful visual features.

Example: Image Classification

Suppose we are training a CNN to classify brain CT images into:

- Normal
- Ischemic stroke
- Hemorrhagic stroke

Initially, the model may make many incorrect predictions.

The training process is:

Step 1: Input image

The CT image enters the network.

Step 2: Forward propagation

The CNN extracts features and produces a prediction.

Step 3: Calculate loss

The prediction is compared with the correct class.

Step 4: Backpropagation

The error information is propagated backward.

Step 5: Calculate gradients

The network determines how its filters and other parameters contributed to the error.

Step 6: Update parameters

The optimizer adjusts the parameters.

Step 7: Repeat

This process occurs for many images and many training epochs.

Eventually, the CNN learns increasingly useful features for distinguishing the classes.

Backpropagation and Feature Learning

One of the most important advantages of neural networks is that they can learn features automatically.

For a CNN, the hierarchy may look conceptually like:

Early Layers



Edges and simple patterns



Middle Layers



Textures and shapes



Deep Layers



Complex structures



Classification

Backpropagation provides the learning signal that allows these features to improve during training.

A Simple Analogy

Consider a student solving a mathematics problem.

The student gives an answer.

The teacher says:

"Your answer is incorrect."

But simply saying "wrong" isn't enough.

The student needs feedback:

"This step was correct, but this step caused the mistake."

The student then goes back through the solution and corrects the problematic steps.

Backpropagation works in a similar conceptual way.

Network makes prediction



Prediction is wrong



Find the source



Propagate feedback backward



Adjust parameters



Try again

Backpropagation: Complete Conceptual Picture

The complete learning cycle can be summarized as follows:

Forward Phase

Input → Network → Prediction

The network uses its current parameters to make a prediction.

Loss Calculation

Prediction → Loss

The loss function determines how far the prediction is from the expected result.

Backward Phase

Loss → Network

Backpropagation determines how the parameters contributed to the loss.

Optimization Phase

Gradients → Parameter Updates

The optimizer modifies the parameters.

Repeat

The process continues until the model learns a useful representation.

Important Terminology

Term	Meaning
Forward Propagation	Passing input through the network to produce an output
Loss	Measure of how incorrect the prediction is
Backpropagation	Calculating gradients by propagating information backward
Chain Rule	Mathematical rule used to calculate the effect through multiple operations
Gradient	Indicates how the loss changes with respect to a parameter
Gradient Flow	Movement of gradients backward through the network
Optimizer	Uses gradients to update model parameters
Learning Rate	Controls the size of parameter updates
Vanishing Gradient	Gradient becomes extremely small
Exploding Gradient	Gradient becomes extremely large
Residual Connection	Shortcut connection that helps information and gradients flow

Optimizers, Training Dynamics, Regularization and Generalization

Neural Network Training: The Big Picture

- Training a neural network involves repeatedly performing:
Input Data → Forward Pass → Prediction → Calculate Loss → Backpropagation → Calculate Gradients → Optimizer → Update Weights → Repeat
- Optimization asks: How should the weights be updated? The answer is provided by optimizers.
- Generalization asks: How can the model perform well on unseen data rather than simply memorizing training data? Regularization and appropriate training strategies help address this.

Optimizers

- An optimizer is an algorithm that updates model parameters using gradients calculated during backpropagation.
- The basic update is:

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta(\partial L / \partial \theta)$$

where θ is a model parameter,
 L is the loss,
 $\partial L / \partial \theta$ is the gradient, and
 η is the learning rate

- Gradient tells the model which direction to move; learning rate tells the model how large a step to take.

Stochastic Gradient Descent (SGD)

- SGD normally calculates gradients using a mini-batch rather than the entire training dataset.
- Advantages: efficient for large datasets, lower memory requirements than full-batch gradient descent, and frequent parameter updates.
- Limitation: different mini-batches produce slightly different gradients, so the optimization path contains some noise.

Momentum

- Momentum extends SGD by considering previous updates in addition to the current gradient.
- It can reduce oscillation, accelerate convergence, and help the optimizer move consistently toward useful directions.
- A common momentum value is 0.9.
- PyTorch example: `optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)`

Adam

- Adam (Adaptive Moment Estimation) combines momentum-like gradient tracking with adaptive scaling of parameter updates.
- Adam can effectively use different step sizes for different parameters.
- A common starting learning rate is 0.001.
- PyTorch example: `optimizer = torch.optim.Adam(model.parameters(), lr=0.001)`
- Adam often converges quickly and is a strong baseline, but SGD with Momentum can sometimes provide better generalization after suitable tuning.

Optimizer Comparison

- SGD: uses mini-batch gradients; common starting LR 0.01–0.1.
- SGD + Momentum: uses current and previous gradients; common starting LR 0.01–0.1.
- Adam: uses momentum-like behavior and adaptive updates; common starting LR 0.001.
- AdamW: Adam with decoupled weight decay; common starting LR 0.001.

These are only starting points, not universal rules.

Learning Rate

- The learning rate controls how much model parameters change during each update.
- If it is too small, training becomes slow and may take a very long time to converge.
- If it is too large, updates may overshoot, causing oscillation or unstable training.
- Choosing an appropriate learning rate is one of the most important parts of neural-network training.

Learning Rate Scheduling

- A learning-rate scheduler changes the learning rate during training. Usually, a relatively larger learning rate is useful early in training, while a smaller rate is useful later for fine adjustment.
- Step Decay reduces the learning rate at fixed intervals.
- Cosine Annealing gradually decreases the learning rate according to a cosine-shaped schedule.
- ReduceLROnPlateau reduces the learning rate when a monitored metric, such as validation loss, stops improving.

Training Dynamics

- Training dynamics describe how the model behaves during training.
- Important quantities include training loss, validation loss, training accuracy, validation accuracy, and learning rate.
- A healthy training process generally shows decreasing loss and improving accuracy, followed by slower progress, learning-rate reduction, and eventual convergence.

Generalization

- Generalization is the ability of a trained model to perform well on unseen data.
- A good model learns general patterns rather than memorizing individual training examples.

Overfitting and Underfitting

- Overfitting occurs when a model performs very well on training data but poorly on unseen data.
Example: training accuracy 99%, validation accuracy 75%.
- Underfitting occurs when the model performs poorly on both training and validation data.
Example: training accuracy 70%, validation accuracy 68%.
- Good generalization is indicated when training and validation performance are both good and reasonably close.

Regularization

- Regularization refers to techniques used to reduce overfitting and improve generalization.
- Important techniques include Dropout, Weight Decay, Data Augmentation, and Batch Normalization, which also has a regularizing effect although its primary purpose is normalization and training stabilization.

Dropout

- Dropout randomly deactivates some neurons during training.
- It prevents the model from becoming too dependent on particular neurons and encourages more robust representations.
- Example: `nn.Dropout(0.5)`, where 0.5 is the dropout probability during training.
- Dropout is active during training and disabled during inference/testing.

Batch Normalization

- Batch Normalization (BatchNorm) normalizes intermediate activations using mini-batch statistics during training.
- Benefits include more stable training, often faster convergence, and some regularization.
- Example: `nn.Conv2d(32, 64, 3)`, `nn.BatchNorm2d(64)`, `nn.ReLU()`.
- Important distinction: Dropout is primarily a regularization technique, while BatchNorm is primarily a normalization and optimization technique.

Weight Decay

- Weight decay discourages the model from developing excessively large weights.
- A common L2-style formulation is: $L_{\text{total}} = L_{\text{data}} + \lambda \sum w_i^2$, where λ controls regularization strength.
- Example: `optimizer = torch.optim.AdamW(model.parameters(), lr=0.001, weight_decay=0.01)`.
- AdamW applies weight decay in a way that is decoupled from Adam's adaptive gradient update.

Data Augmentation

- Data augmentation creates varied versions of existing training samples.
- Common image transformations include horizontal flipping, small rotations, cropping, scaling, translation, and brightness/contrast changes.
- It improves robustness by preventing the model from relying too heavily on specific appearances in the training dataset.

Important Rule for Data Augmentation

- An augmentation should not change the correct class label.
- Only use transformations that are realistic for the particular problem. This is especially important in specialized domains such as medical image classification.

Training vs Validation Data

- Training data can use random augmentation.
- Validation and test data should generally use only consistent preprocessing such as resizing and normalization, not random augmentation.
- This ensures that evaluation represents consistent unseen data.

Combining Optimization and Regularization

- A practical deep-learning model may use several techniques together.
- Example: Data Augmentation → Neural Network with BatchNorm/Dropout → Loss → Backpropagation → AdamW or SGD → Weight Decay → Learning-Rate Scheduler → Updated Parameters.
- These techniques solve different problems: optimizers update parameters; learning rate controls update size; schedulers change the learning rate; Dropout reduces neuron co-adaptation; BatchNorm stabilizes activations; Weight Decay controls large weights; Data Augmentation increases training-data diversity.

Practical Training Example

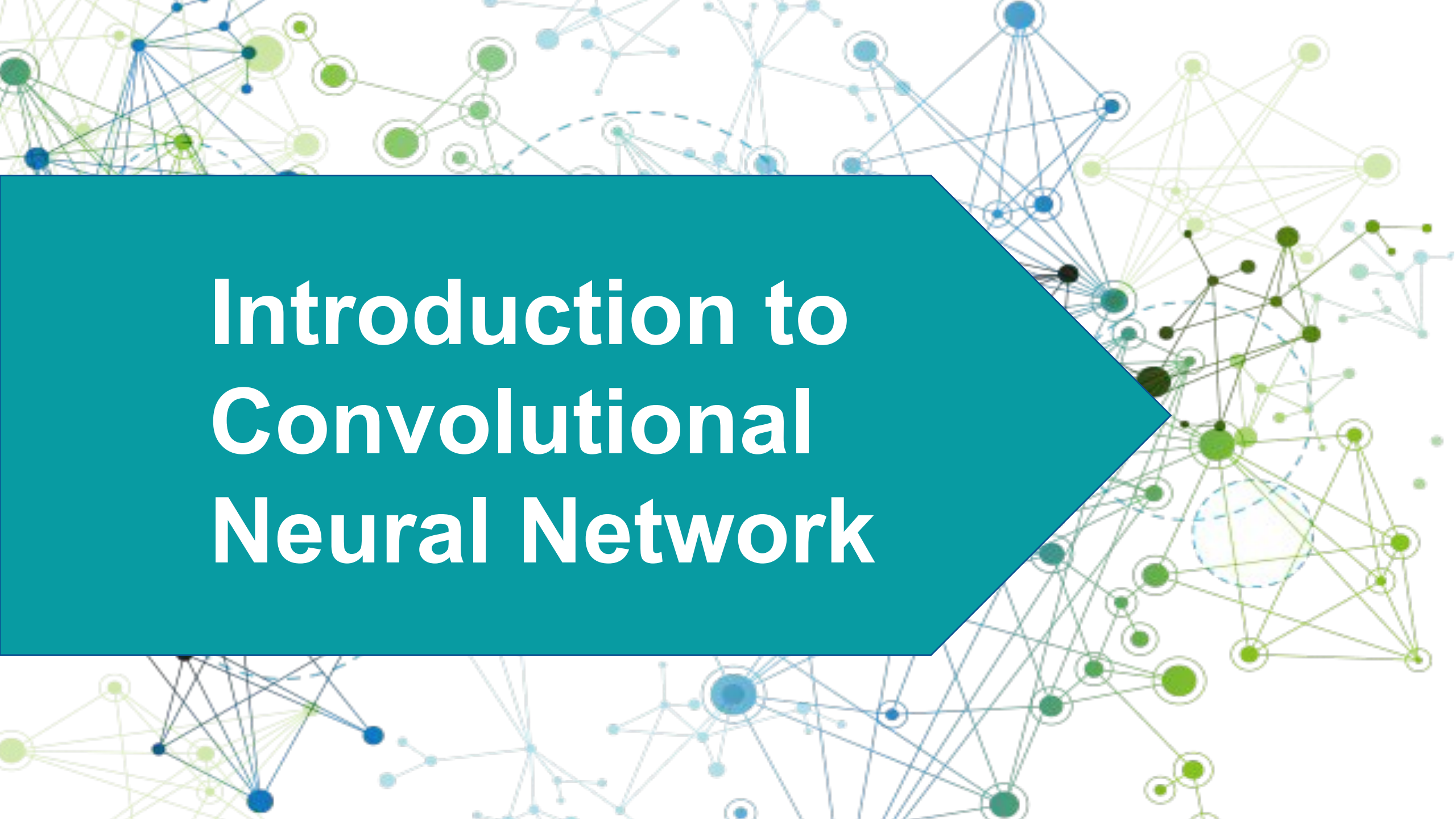
- If training accuracy is 98% and validation accuracy is 80%, the large gap suggests overfitting.
- Possible actions include realistic data augmentation, appropriate weight decay, dropout where appropriate, monitoring validation performance, and using a learning-rate schedule.
- The objective is to improve validation/generalization performance, not simply increase training accuracy.

Common Problems and Solutions

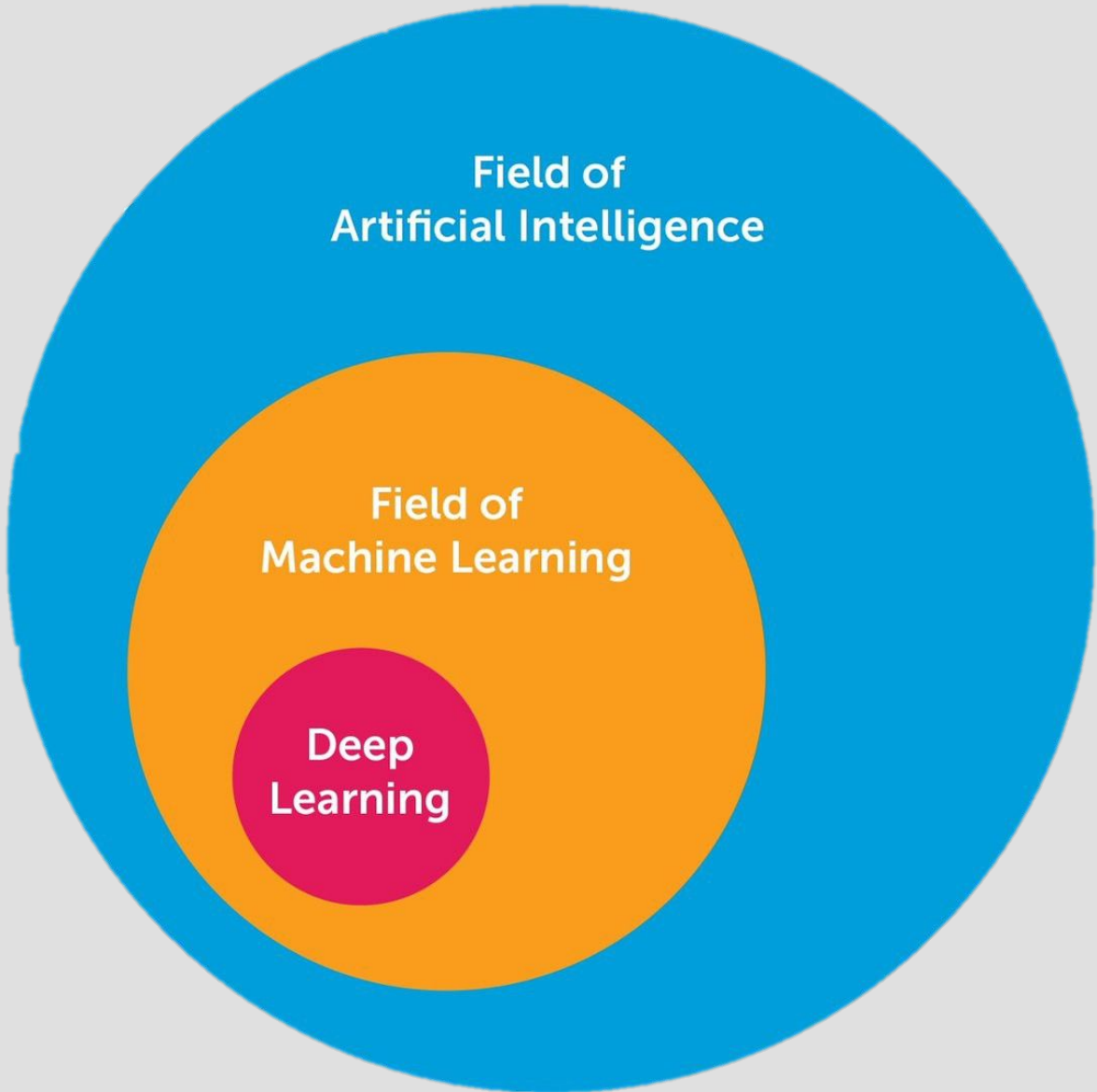
- Loss decreases very slowly → learning rate may be too small → consider increasing it.
- Loss oscillates → learning rate may be too large → reduce learning rate or use a suitable scheduler.
- Training accuracy is low → possible underfitting → increase model capacity or reduce excessive regularization.
- Training accuracy is very high but validation accuracy is low → possible overfitting → use augmentation, dropout, or weight decay.
- Training is unstable → review learning rate, optimizer, preprocessing, and BatchNorm configuration.
- Validation performance stops improving → consider learning-rate scheduling, regularization, or early stopping.

Key Takeaways

- SGD updates parameters using mini-batch gradients.
- Momentum uses previous update information to make SGD smoother and faster.
- Adam combines momentum-like behavior with adaptive parameter updates.
- Learning rate controls the size of parameter updates.
- Learning-rate schedules adjust the learning rate during training.
- Generalization means performing well on unseen data.
- Overfitting means excellent training performance but poor unseen-data performance.
- Dropout randomly disables neurons during training.
- BatchNorm normalizes activations and stabilizes training.
- Weight decay discourages excessively large weights.
- Data augmentation increases the diversity of training examples.
- Too much regularization can cause underfitting.



Introduction to Convolutional Neural Network

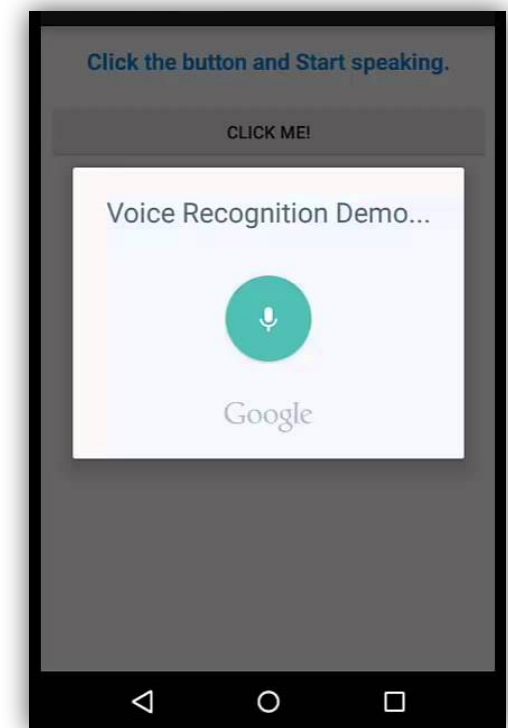
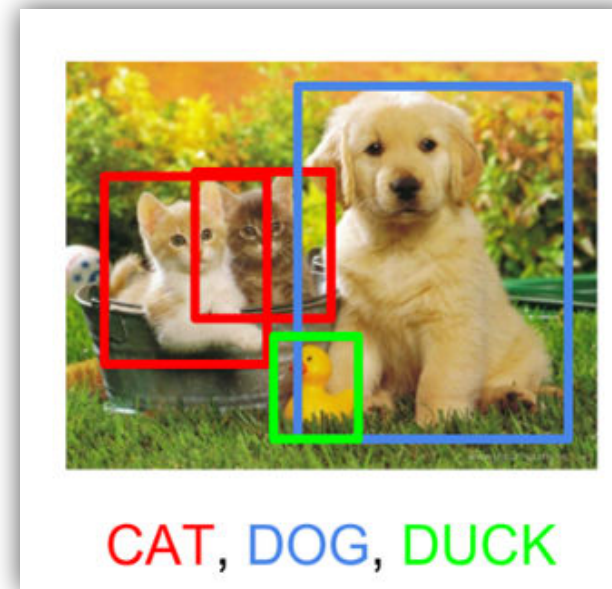


A Venn diagram illustrating the relationship between three fields of study. It consists of three concentric circles. The outermost circle is blue and labeled 'Field of Artificial Intelligence'. Inside it is an orange circle labeled 'Field of Machine Learning'. Inside the orange circle is a smaller pink circle labeled 'Deep Learning'. To the left of the circles is a vertical black line.

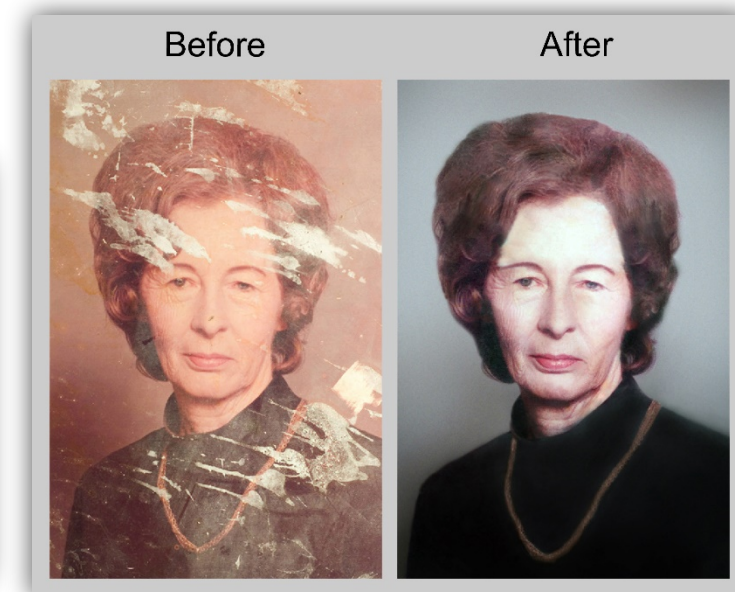
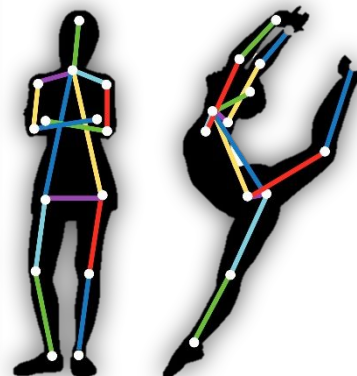
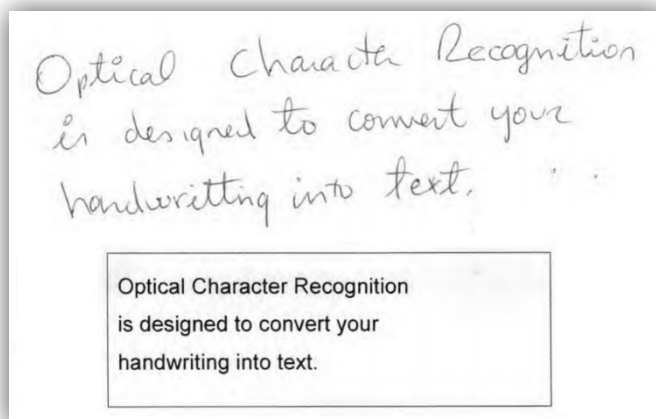
Field of
Artificial Intelligence

Field of
Machine Learning

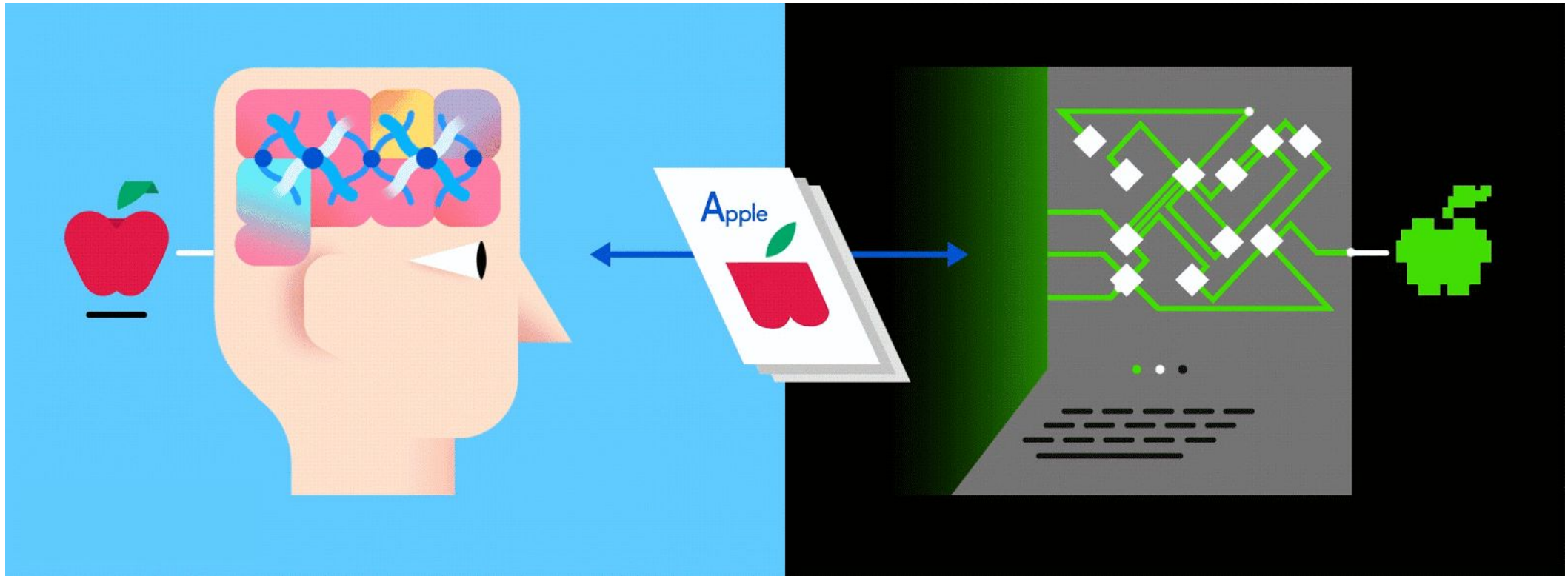
Deep
Learning



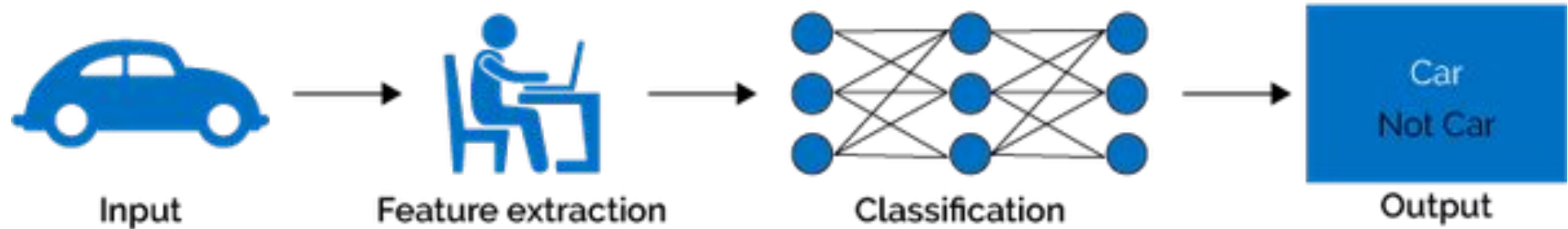
Applications



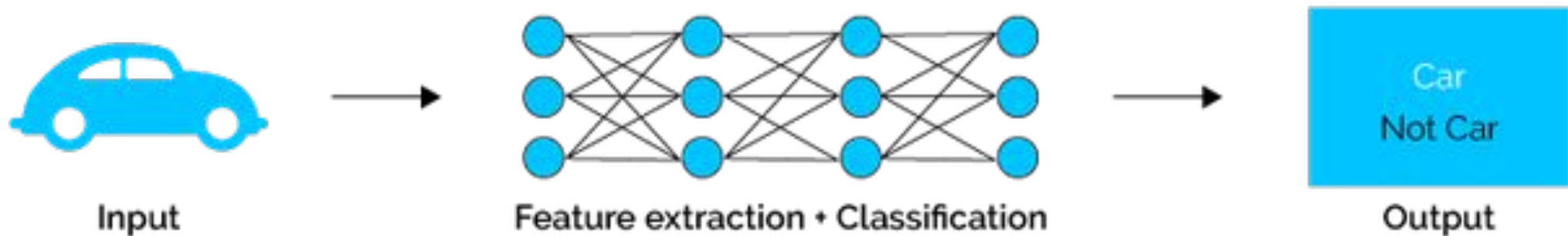
Neural Network



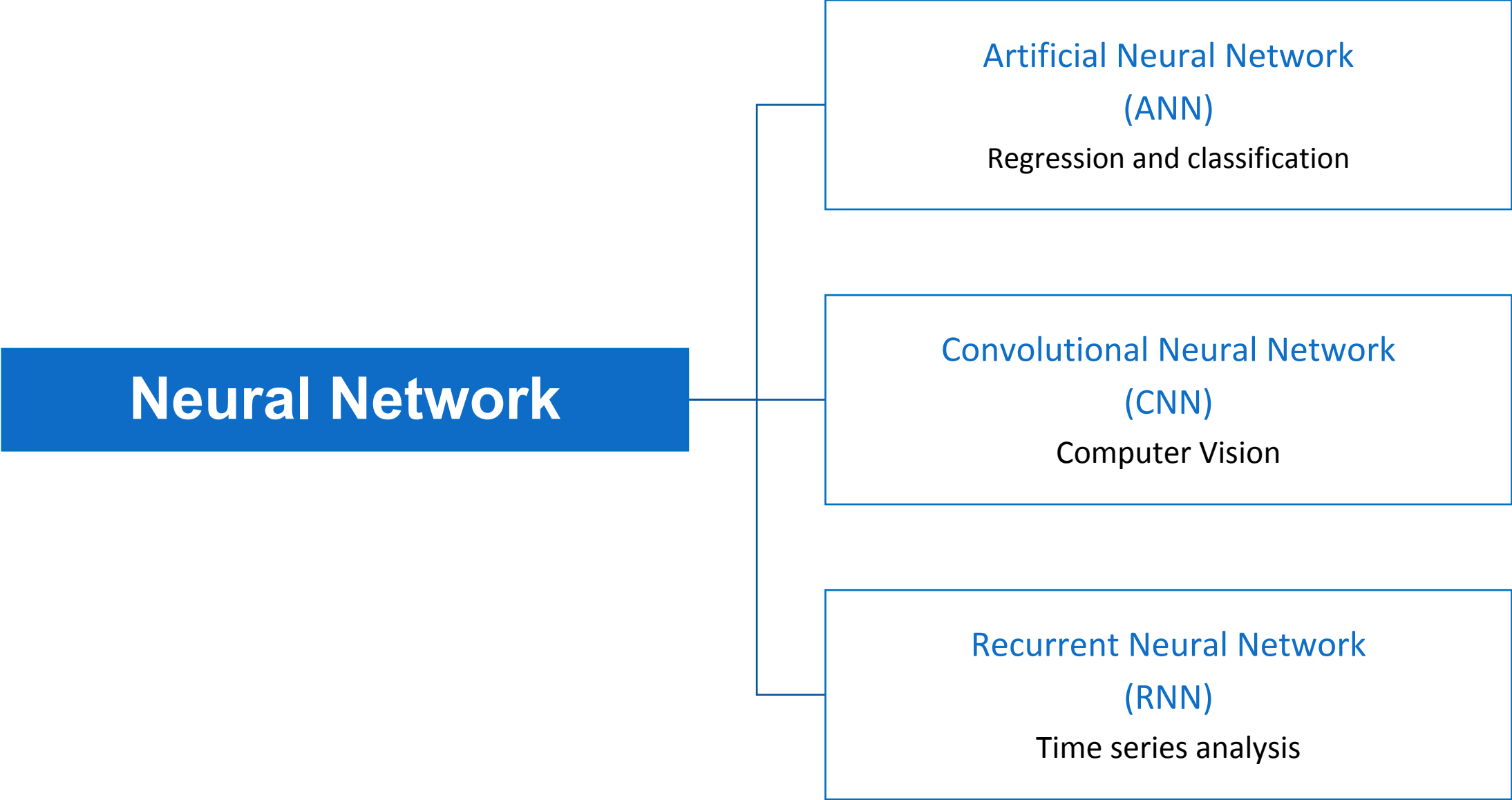
Shallow learning



Deep learning



Neural Network



```
graph LR; NN[Neural Network] --- ANN[Artificial Neural Network (ANN)]; NN --- CNN[Convolutional Neural Network (CNN)]; NN --- RNN[Recurrent Neural Network (RNN)];
```

Artificial Neural Network
(ANN)

Regression and classification

Convolutional Neural Network
(CNN)

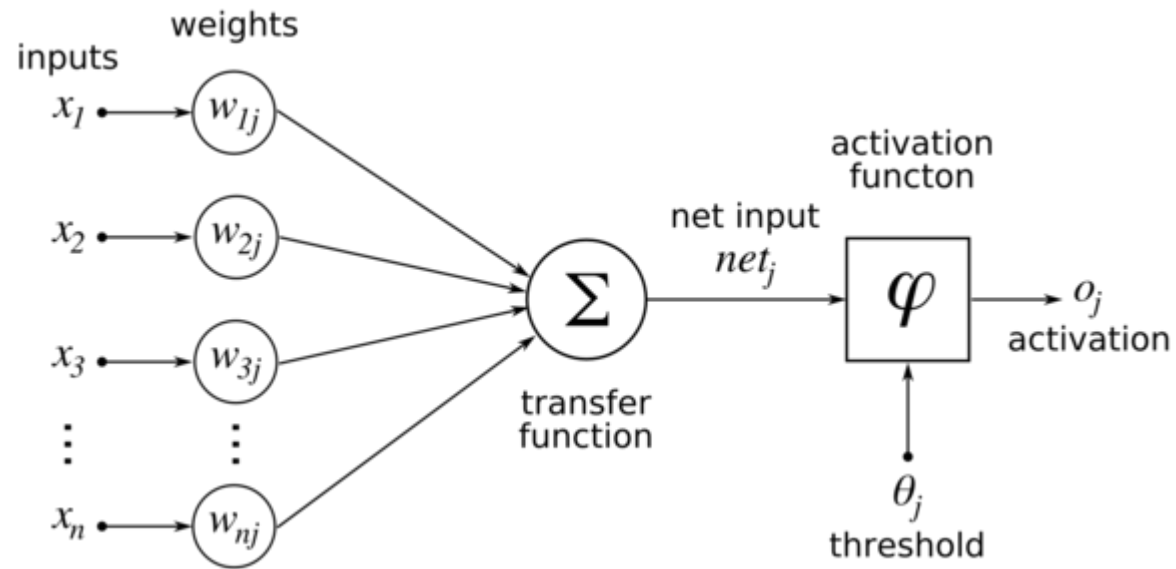
Computer Vision

Recurrent Neural Network
(RNN)

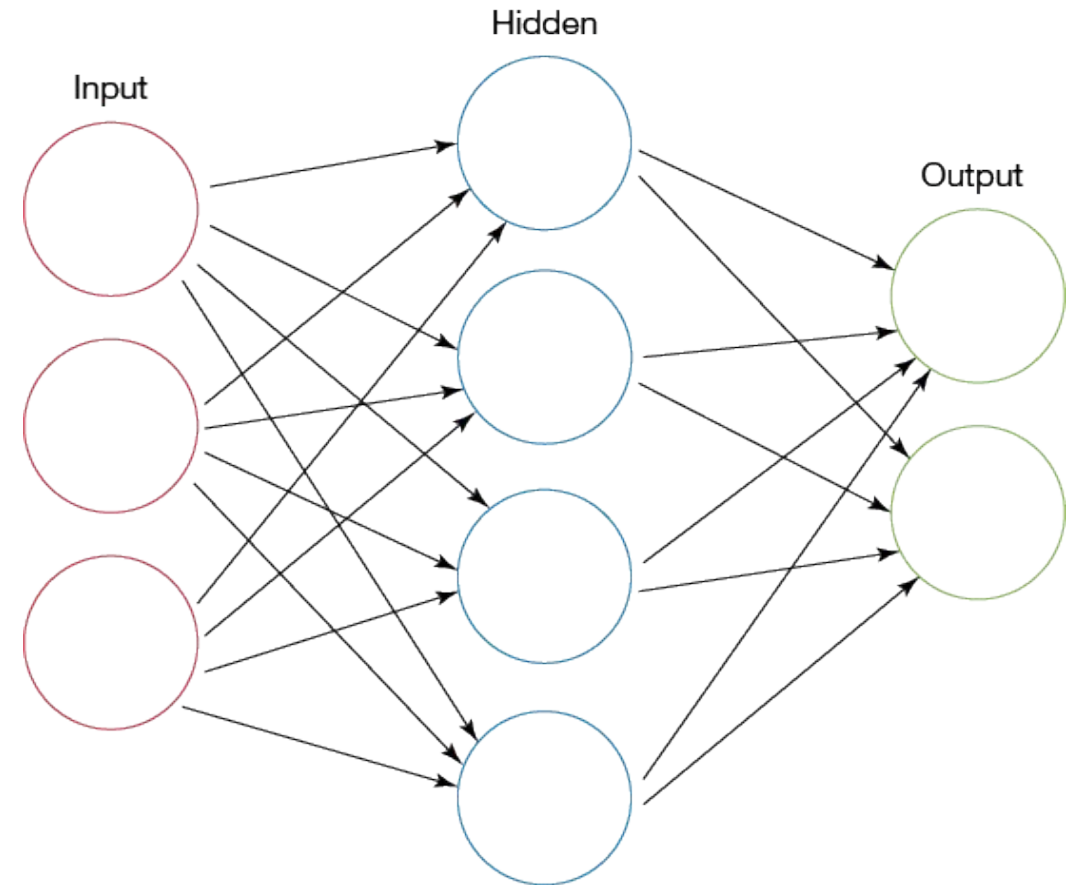
Time series analysis

Artificial Neural Network

Single layer perceptron



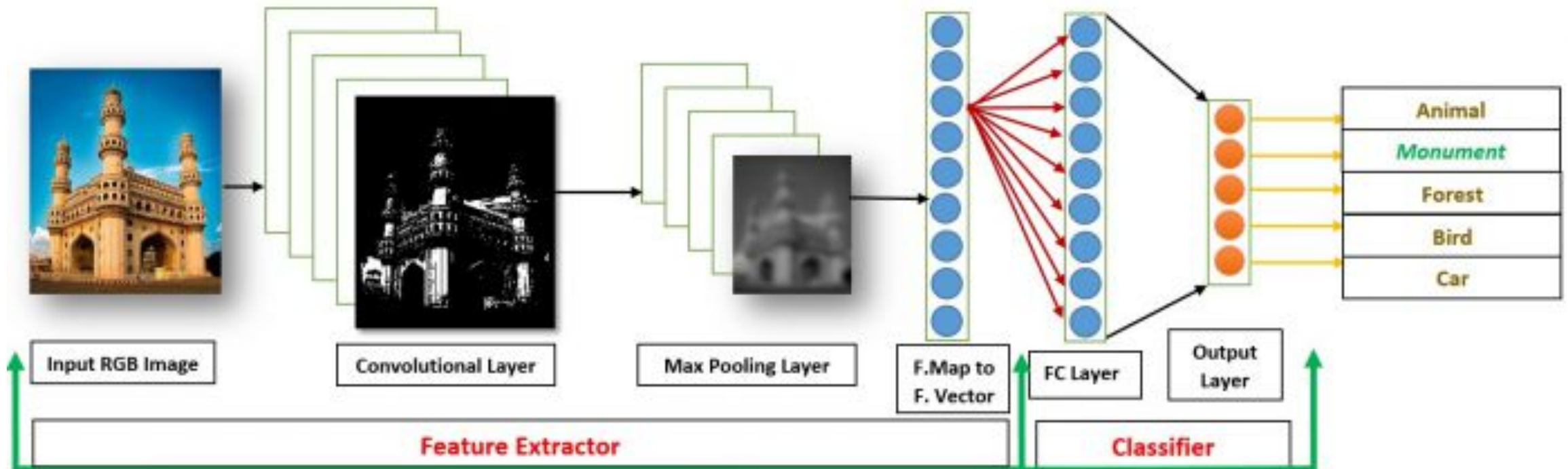
Multi-layer perceptron



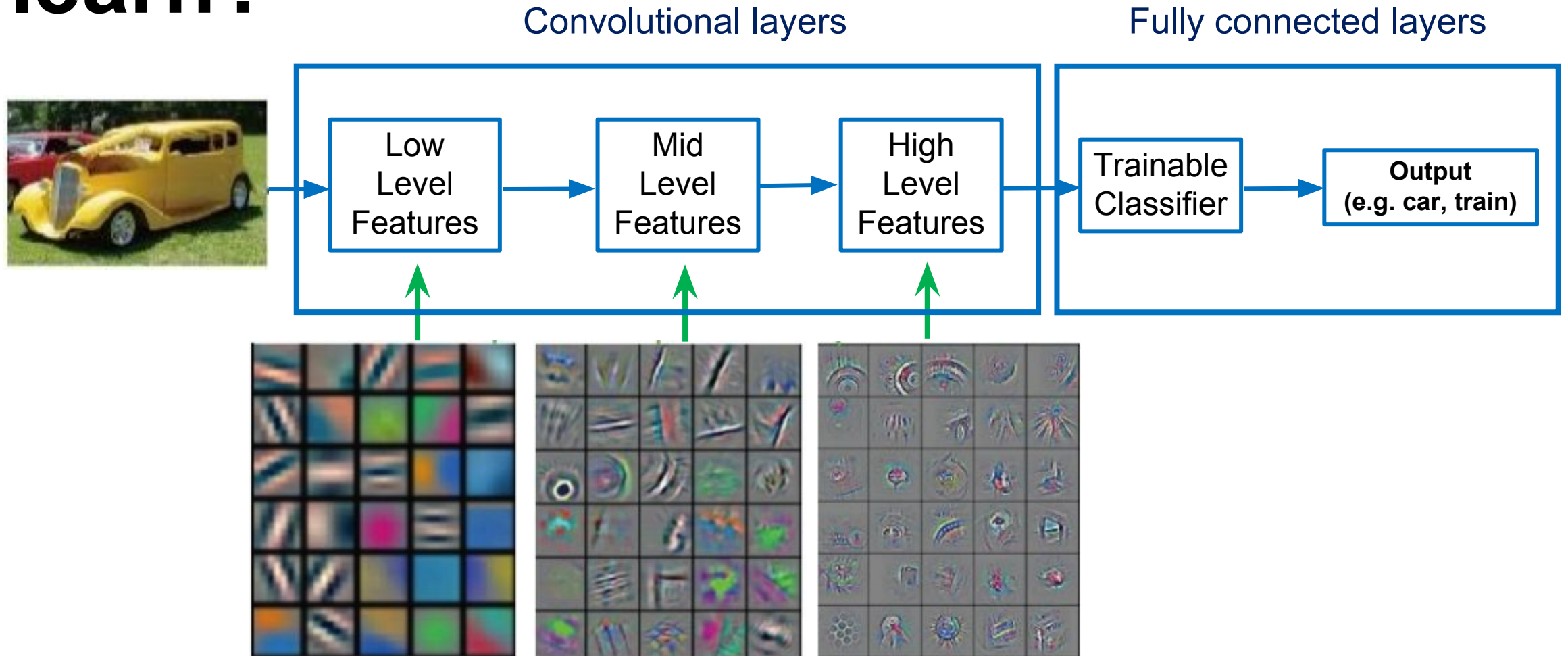
Convolutional Neural Network

Introduction

Convolutional Neural Networks (**CNNs**) learns **multi-level features** and **classifier** in a **joint fashion** and performs much **better than traditional approaches** for various image classification and segmentation problems.



CNN – What do they learn?

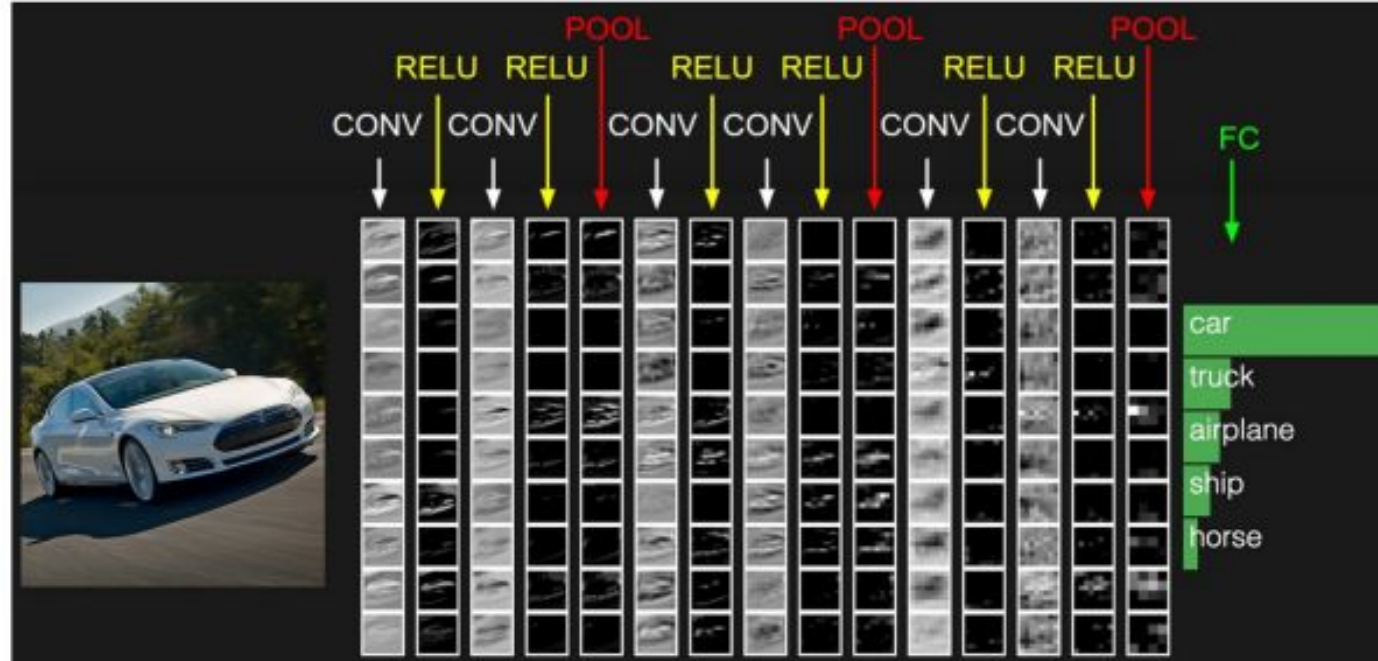


CNN -

Components

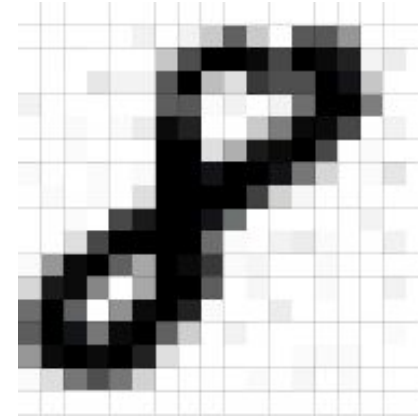
There are four main components in the CNN:

1. Convolution
2. Non-Linearity
3. Pooling or Sub Sampling
4. Classification (Fully Connected Layer)



Input

- An Image is a matrix of pixel values.
- If we consider a gray scale image, the value of each pixel in the matrix will range from 0 to 255.
- If we consider an RGB image, each pixel will have the combined values of R, G and B.



What We See

```
08 02 22 97 38 15 00 40 00 75 04 05 07 78 52 12 50 77 91 08
49 49 99 40 17 81 18 57 60 87 17 40 98 43 69 48 04 56 62 00
81 49 31 73 55 79 14 29 93 71 40 67 53 88 30 03 49 13 36 65
52 70 95 23 04 60 11 42 69 24 68 56 01 32 56 71 37 02 36 91
22 31 16 71 51 67 63 89 41 92 36 54 22 40 40 28 66 33 13 80
24 47 32 60 99 03 45 02 44 75 33 53 78 36 84 20 35 17 12 50
32 98 81 28 64 23 67 10 26 38 40 67 59 54 70 66 18 38 64 70
67 24 20 68 02 62 12 20 95 63 94 39 63 08 40 91 66 49 94 21
24 55 58 05 66 73 99 24 97 17 78 78 94 83 14 88 34 89 63 72
21 36 23 09 75 00 74 44 20 45 35 14 00 61 33 97 34 31 33 95
78 17 53 28 22 75 31 47 15 94 03 80 04 62 16 14 09 53 54 92
16 39 05 42 96 35 31 47 55 58 88 24 00 17 54 24 36 29 85 57
86 56 00 48 35 71 89 07 05 44 44 37 44 60 21 58 51 54 17 58
19 80 81 68 05 94 47 69 28 73 92 13 86 52 17 77 04 89 55 40
04 52 08 83 97 35 99 16 07 97 57 32 16 26 24 79 33 27 98 66
88 36 68 87 57 62 20 72 03 46 33 67 46 55 12 32 63 93 53 69
04 42 16 73 35 25 39 11 24 94 72 18 08 46 29 32 40 62 76 36
20 69 36 41 72 30 23 88 34 62 99 69 82 67 59 85 74 04 36 16
20 73 35 29 78 31 90 01 74 31 49 71 48 86 81 16 23 57 05 54
01 70 54 71 83 51 54 69 16 92 33 48 61 43 52 01 89 19 47 48
```

What Computers See

Convolution

The primary purpose of Convolution in case of a CNN is to extract features from the input image.

1	0	1
0	1	0
1	0	1

Filter / Kernel / Feature detector

Image

1x1	1x0	1x1	0	0
0x0	1x1	1x0	1	0
0x1	0x0	1x1	1	1
0	0	1	1	0
0	1	1	0	0

4		

Convolved Feature /
Activation Map /
Feature Map

Convolution...



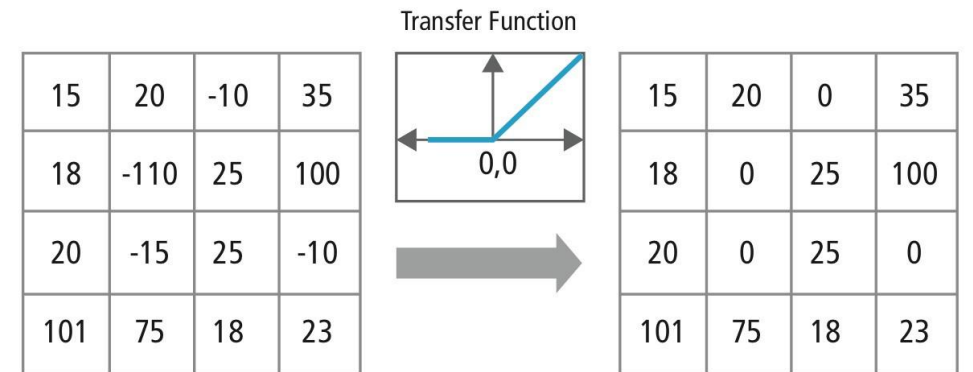
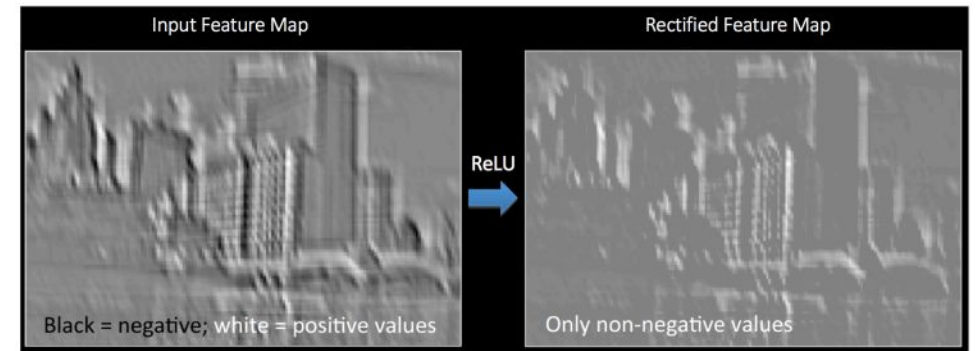
Input

Convolution...

- The size of the output volume is controlled by three parameters that we need to decide before the convolution step is performed:
 - ✓ **Depth:** Depth corresponds to the number of filters we use for the convolution operation.
 - ✓ **Stride:** Stride is the number of pixels by which we slide our filter matrix over the input matrix.
 - ✓ **Zero-padding:** Sometimes, it is convenient to pad the input matrix with zeros around the border, so that we can apply the filter to bordering elements of our input image matrix.
 - With zero-padding *wide convolution*
 - Without zero-padding *narrow convolution*

Non-Linearity (ReLU)

- Replaces all negative pixel values in the feature map by zero.
- The purpose of ReLU is to introduce non-linearity in CNN, since most of the real-world data would be non-linear.
- Other non-linear functions such as tanh $(-1,1)$ or sigmoid $(0,1)$ can also be used instead of ReLU $(0,\text{input})$.



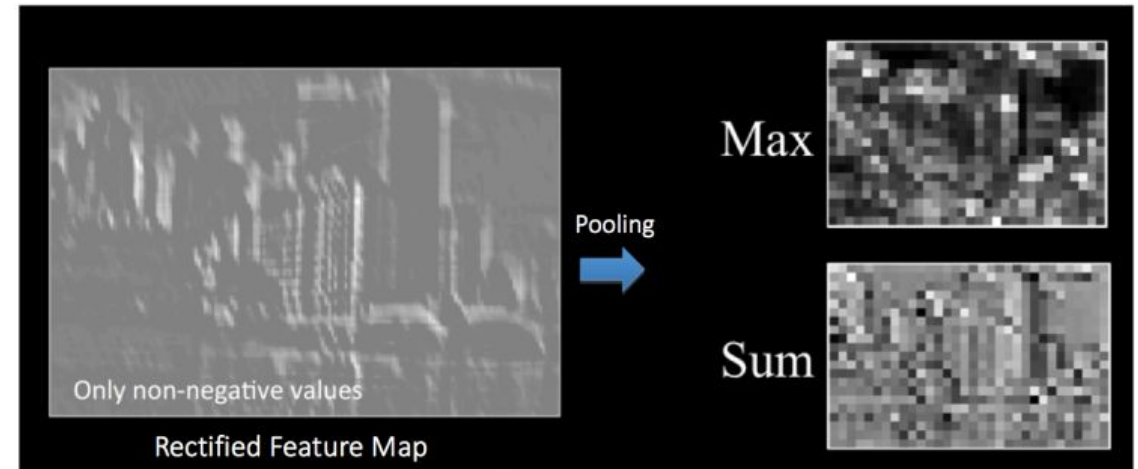
Pooling

Reduces the dimensionality of each feature map but retains the most important information. Pooling can be of different types: Max, Average, Sum etc.

1	3	2	9
7	4	1	5
8	5	2	3
4	2	1	4

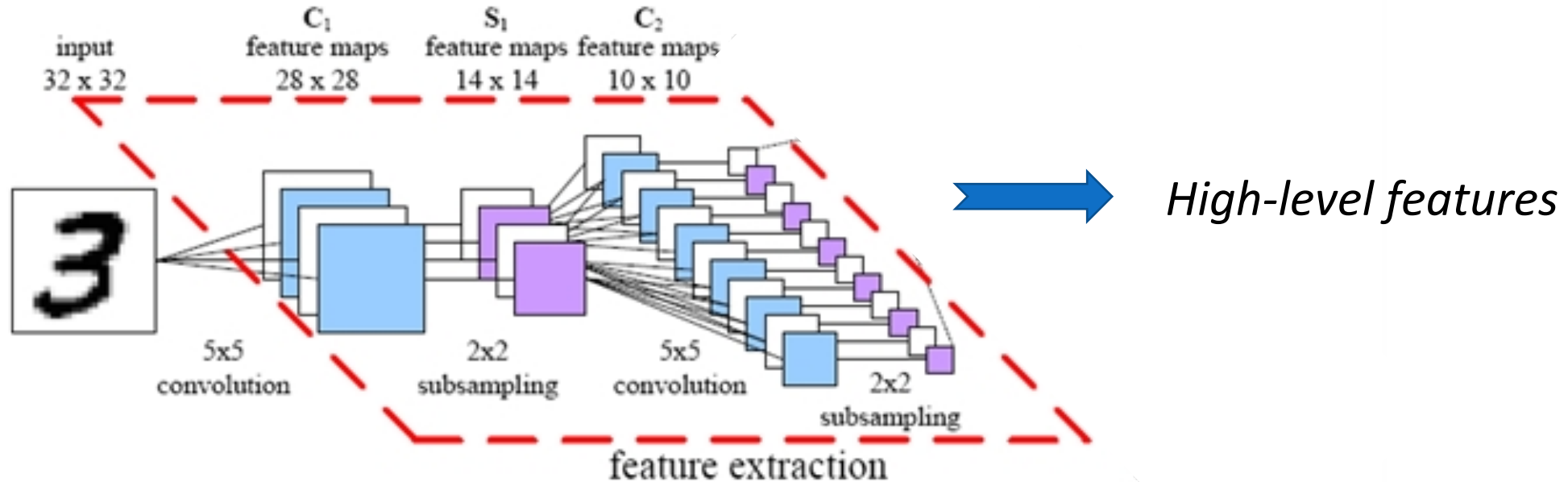
2x2 region

7	9
8	



Story so far

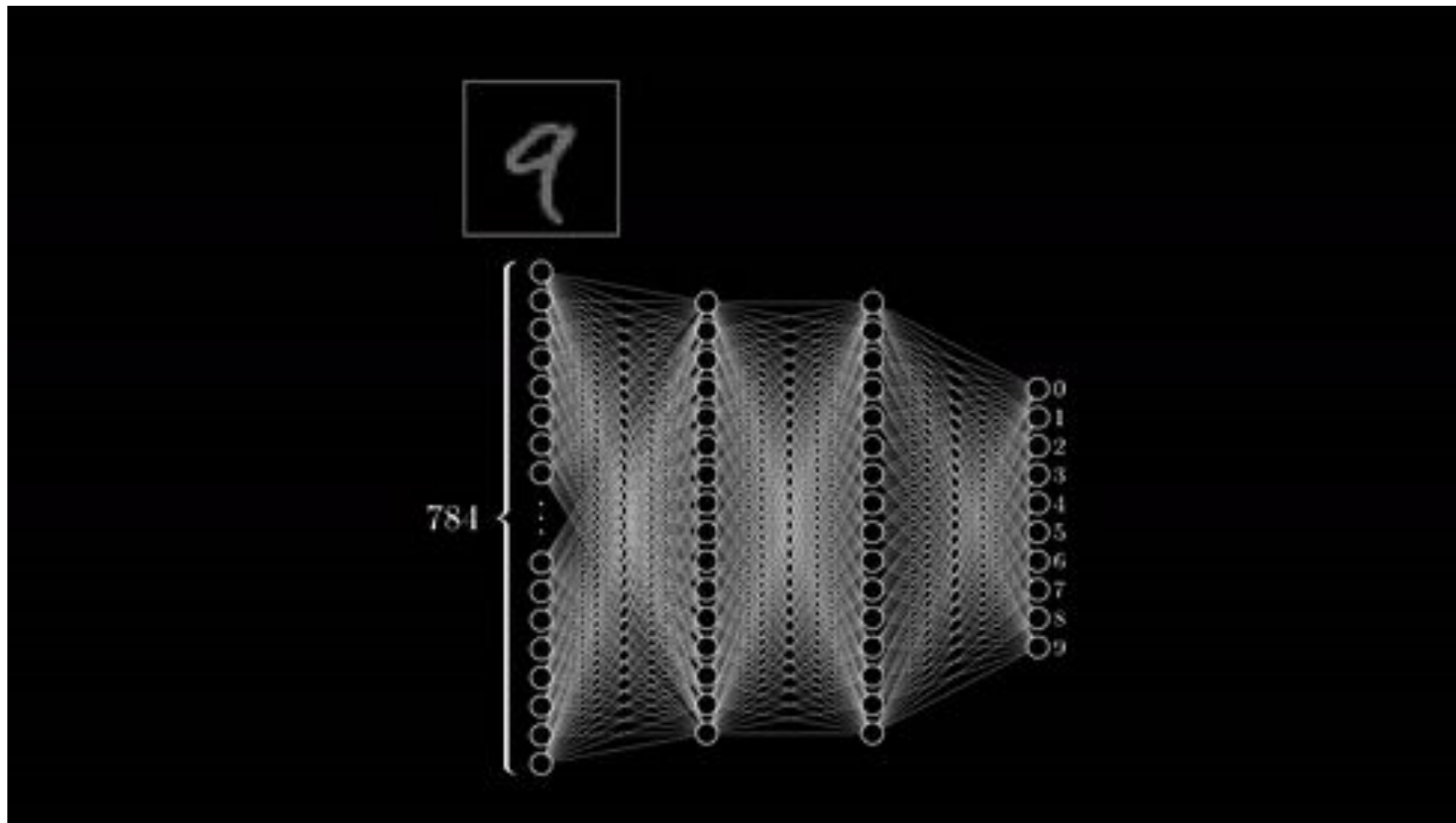
- Together these layers extract the useful features from the images.
- The output from the convolutional and pooling layers represent high-level features of the input image.



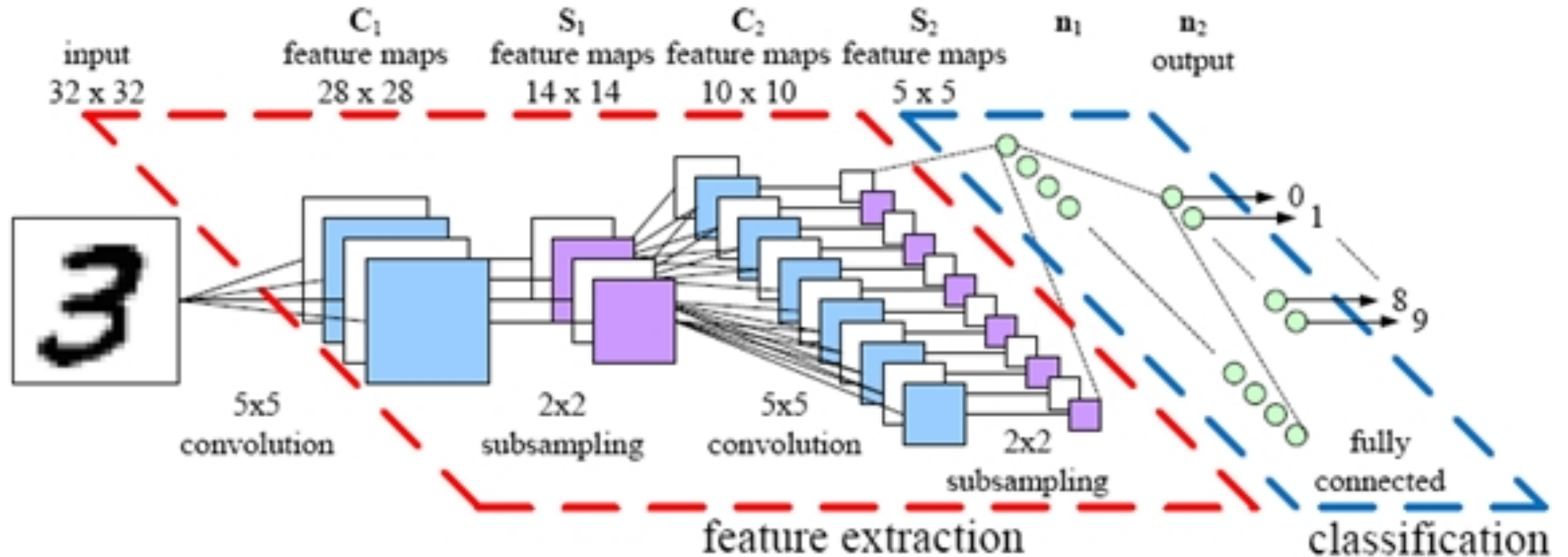
Fully Connected Layer

- A traditional Multi-Layer Perceptron.
- The term “Fully Connected” implies that every neuron in the previous layer is connected to every neuron on the next layer.
- Their activations can hence be computed with a matrix multiplication followed by a bias offset.
- The purpose of the Fully Connected layer is to use the high-level features for classifying the input image into various classes based on the training dataset.

Fully Connected Layer...



Overall CNN Architecture



Putting it all together – Training using Backpropagation

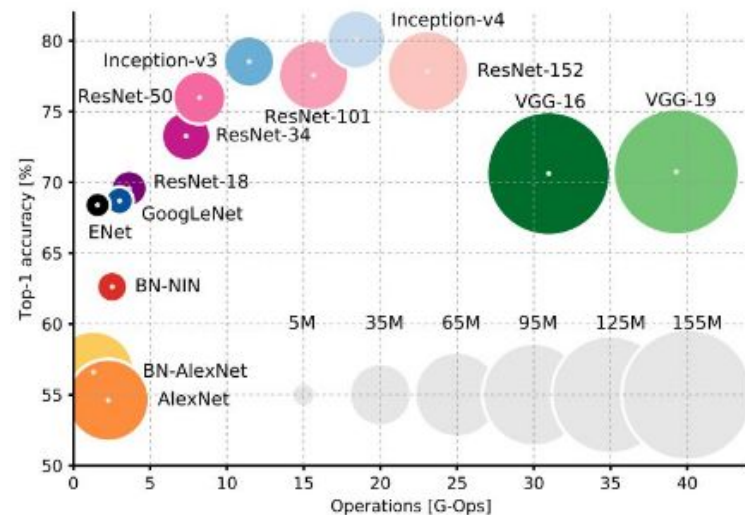
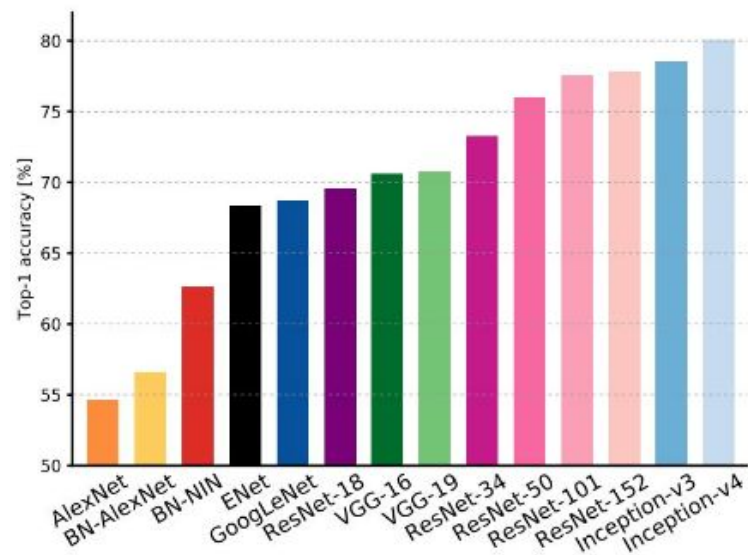
- **Step 1:** We initialize all filters and parameters / weights with random values.
- **Step 2:** The network takes a training image as input, goes through the forward propagation step (convolution, ReLU and pooling operations along with forward propagation in the Fully Connected layer) and finds the output probabilities for each class.
 - Let's say the output probabilities for the boat image above are [0.2, 0.4, 0.1, 0.3].
 - Since weights are randomly assigned for the first training example, output probabilities are also random.
- **Step 3:** Calculate the total error at the output layer (summation over all 4 classes).

- **Step 4:** Use Backpropagation to calculate the gradients of the error with respect to all weights in the network and use gradient descent to update all filter values/ weights and parameter values to minimize the output error.
 - The weights are adjusted in proportion to their contribution to the total error.
 - When the same image is input again, output probabilities might now be $[0.1, 0.1, 0.7, 0.1]$, which is closer to the target vector $[0, 0, 1, 0]$.
 - This means that the network has learnt to classify this particular image correctly by adjusting its weights / filters such that the output error is reduced.
 - Parameters like number of filters, filter sizes, architecture of the network etc. have all been fixed before Step 1 and do not change during training process – only the values of the filter matrix and connection weights get updated.
- **Step 5:** Repeat steps 2-4 with all images in the training set.

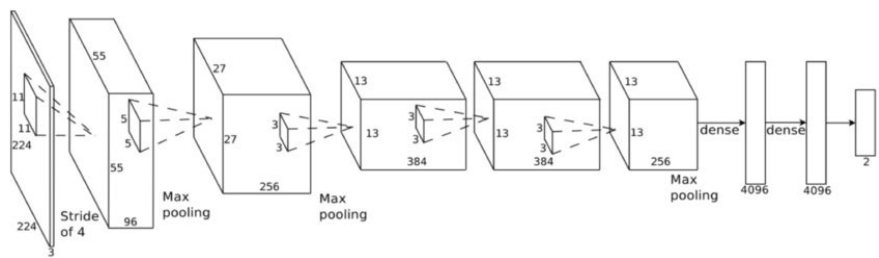
A thin vertical black line is positioned to the left of the title text.

CNN Architectures

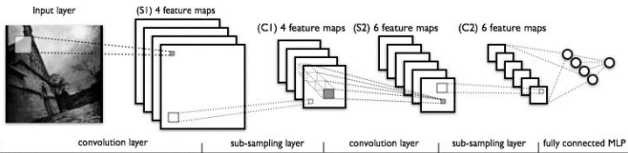
Year	CNN Architecture	Developed By
1998	LeNet	Yann LeCun <i>et al.</i>
2012	AlexNet	Alex Krizhevsky, Geoffrey Hinton, and Ilya Sutskever
2013	ZFNet	Matthew Zeiler and Rob Fergus
2014	GoogleNet	Google
2014	VGGNet	Simonyan and Zisserman
2015	ResNet	Kaiming He
2017	DenseNet	Gao Huang, Zhuang Liu, Laurens van der Maaten, and Kilian Q. Weinberger



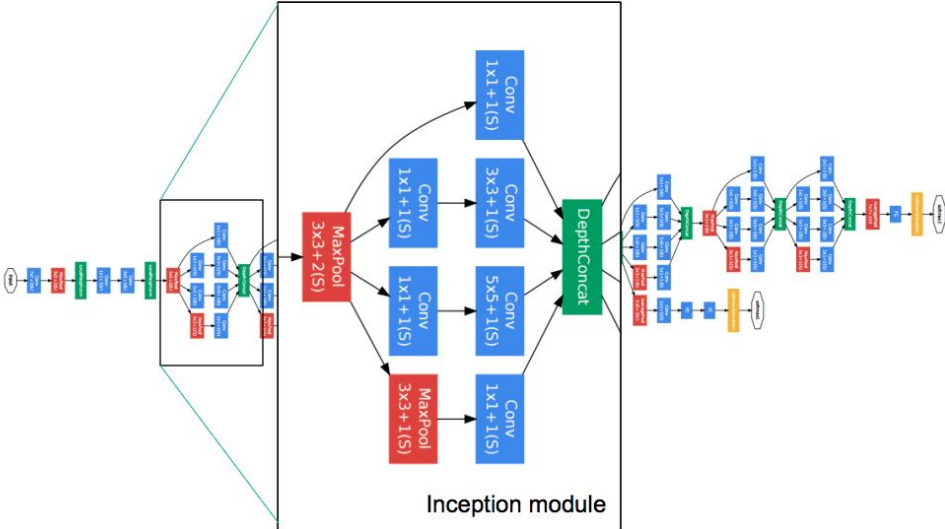
AlexNet



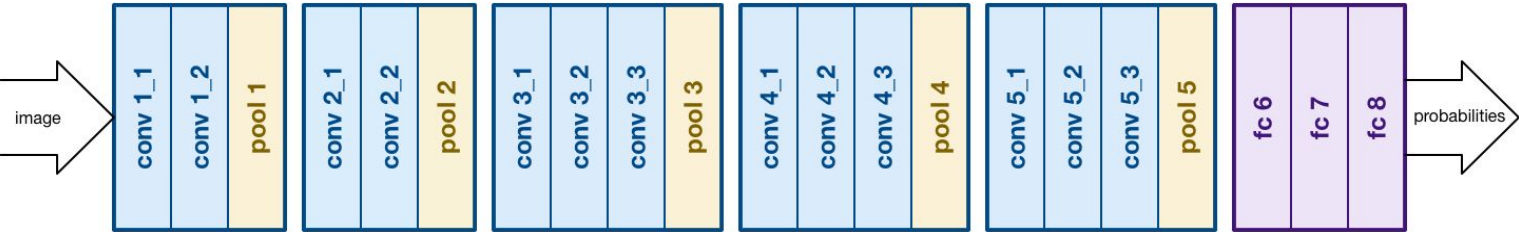
LeNet



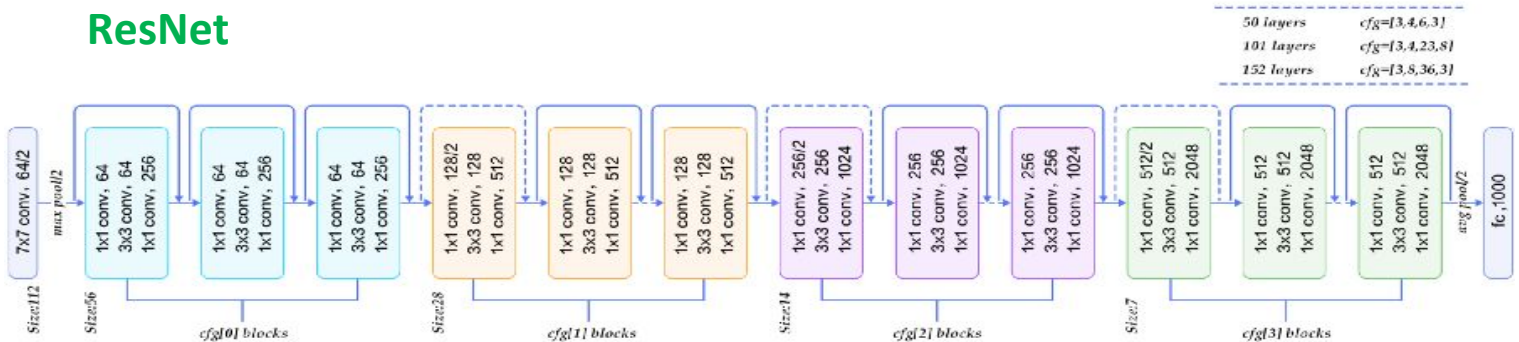
GoogleNet



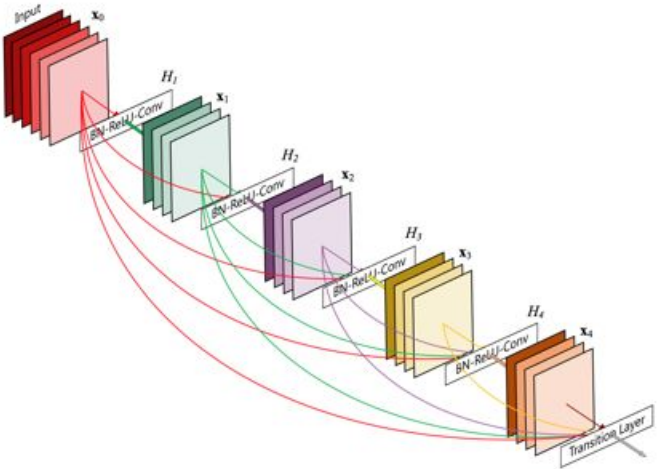
VGG-16



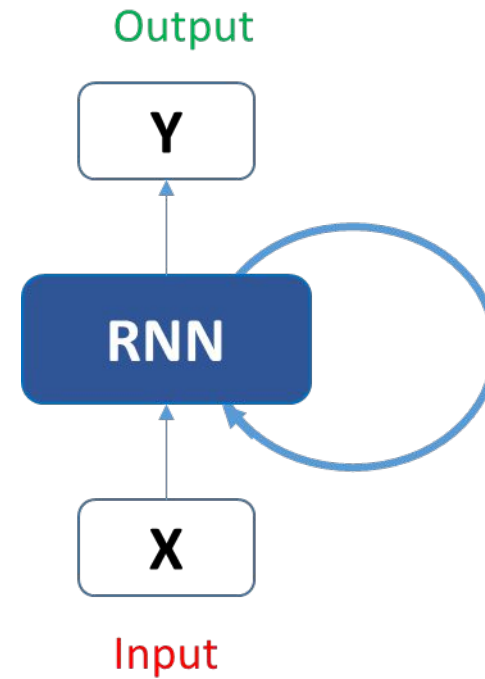
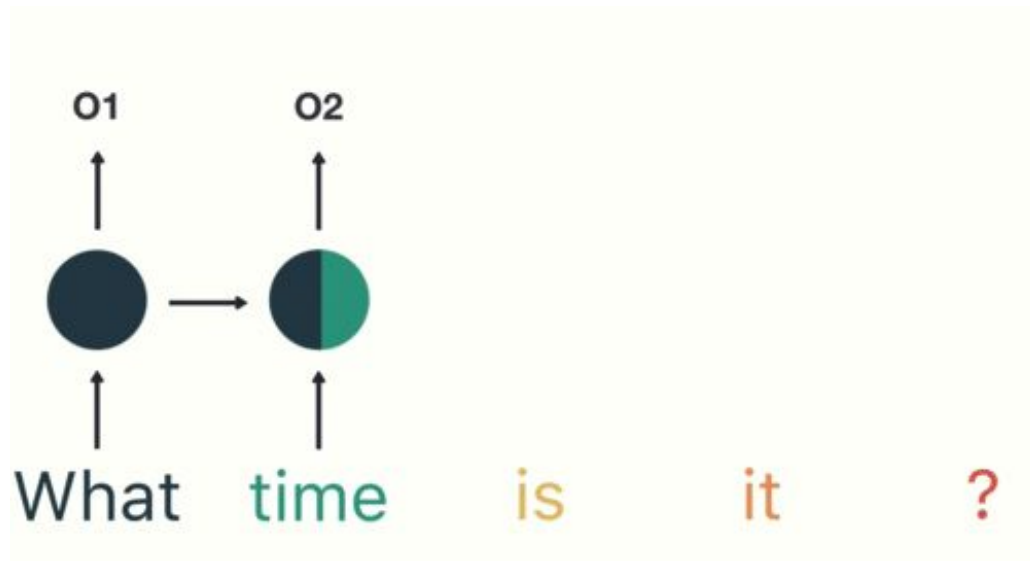
ResNet



DenseNet



Recurrent Neural Network - RNN

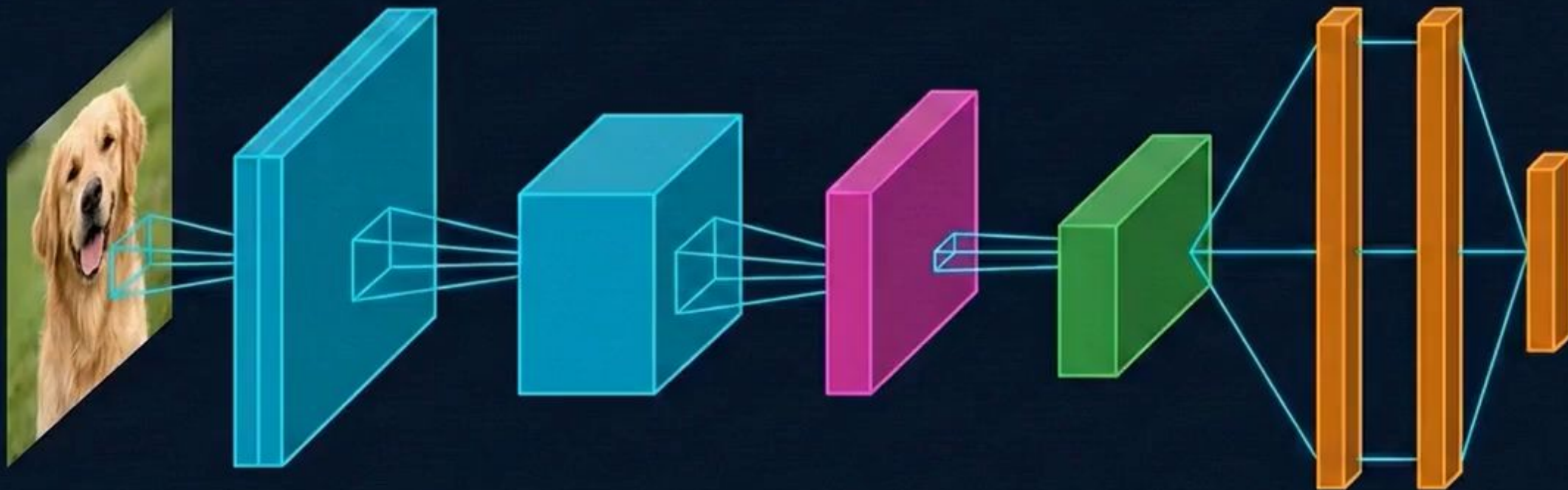


Visualizing a CNN

- **Adam Harley** created amazing visualizations of a Convolutional Neural Network trained on the MNIST Database of handwritten digits
 - 2D Visualisation of a CNN - <http://scs.ryerson.ca/~aharley/vis/conv/flat.html>
 - 3D Visualisation of a CNN - <http://scs.ryerson.ca/~aharley/vis/conv/>

Thank You!







Computer Vision and Image Processing

Lecture 21

Vision Transformers

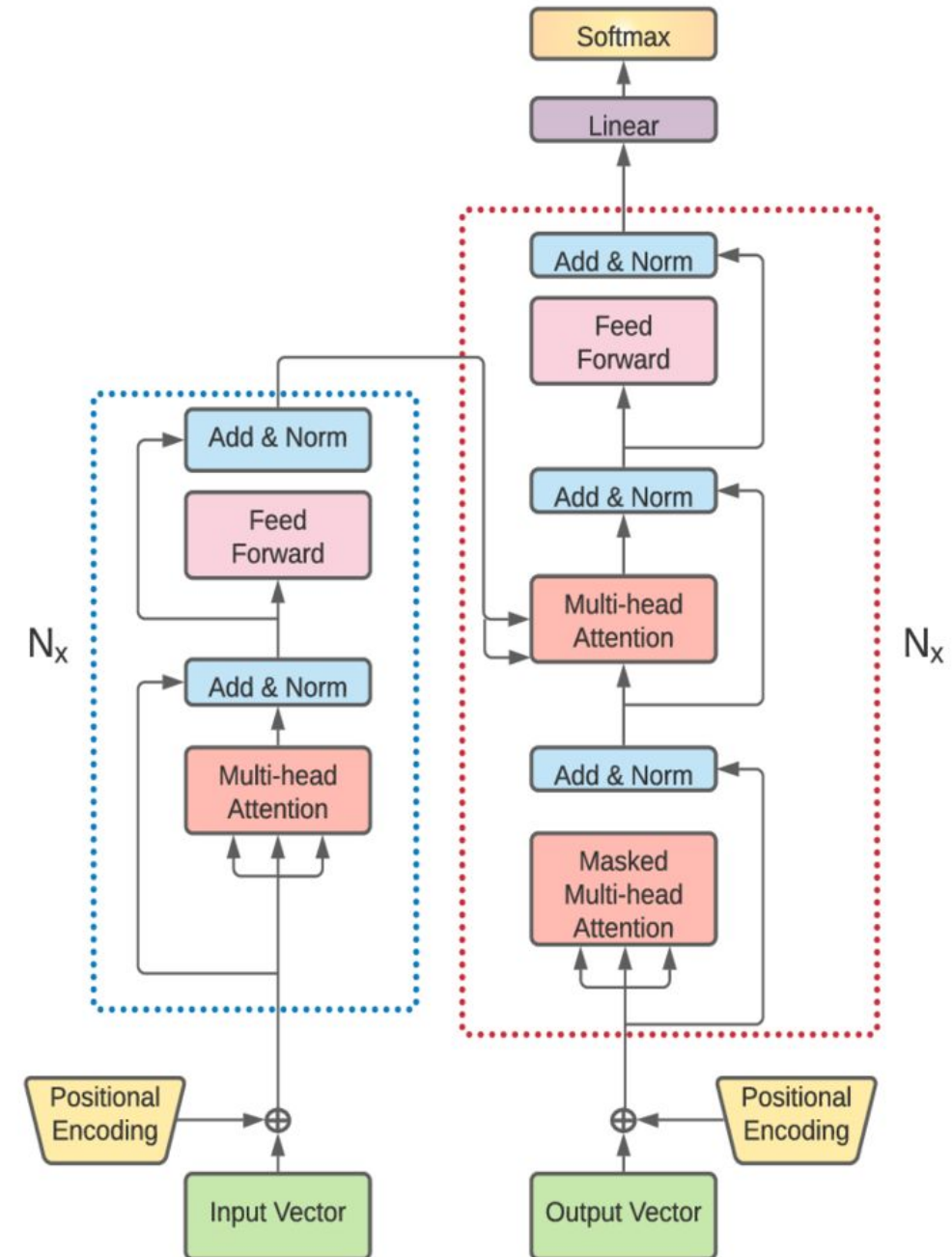
What are Transformers?

- A deep learning architecture originally developed for Natural Language Processing (NLP).
 - Used in:
 - ❖ large-scale natural language processing
 - ❖ computer vision (vision transformers)
 - ❖ Reinforcement learning
 - ❖ Audio
 - ❖ multimodal learning
 - ❖ Robotics
 - Led to the development of pre-trained systems, such as generative pre-trained transformers (GPTs) and BERT (bidirectional encoder representations from transformers).
- Primarily used in the AI subfields of natural language processing (NLP) and computer vision (CV).

Transformer Architecture

Key Components

- Attention Mechanism: Enables the model to focus on relevant parts of the input.
- Self-Attention and Multi-Head Attention: Captures different aspects of the input simultaneously.
- Positional Encoding: Retains the order of input sequences.



Why Go Beyond NLP?

Unified Architecture: Transformers offer a generic framework adaptable across domains — from language to vision and beyond.

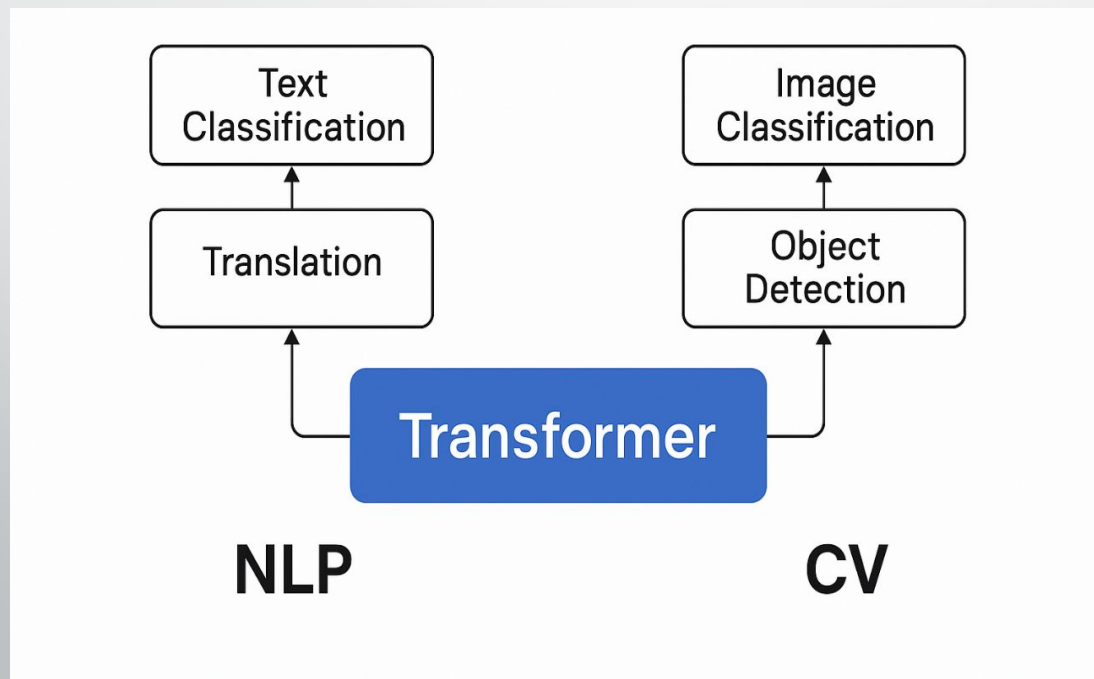
Success in NLP: Transformers like BERT and GPT have significantly outperformed previous models, inspiring cross-domain applications.

Limitations of CNNs: Convolutional Neural Networks struggle with modeling long-range dependencies and global context in images.

Transformer Strengths in Vision: Self-attention mechanisms allow Vision Transformers to capture global relationships in images effectively.

Scalability & Parallelism: Transformers process data in parallel, making them more efficient on modern hardware compared to sequential models.

Data-Driven Performance: With sufficient training data, ViTs can match or surpass CNNs in key computer vision benchmarks.



From Text to Vision – The Evolution

- Vision Transformers(ViTs) has been introduced that extends the transformers to Computer vision tasks.
- Supervise-trained to predict the image label from the patches of the image.
- Alternative to Convolutional Neural Networks (CNNs) that are currently state-of-the-art (SOTA) in different image recognition computer vision tasks.
- ViT models outperform the current SOTA CNNs by almost x4 in terms of computational efficiency and accuracy.

Vision Transformers

ViT Architecture



How it works?

The overall structure of the vision transformer architecture consists of the following steps:

1. Split an image into patches (fixed sizes)
 2. Flatten the image patches
 3. Create lower-dimensional linear embeddings from these flattened image patches
 4. Include positional embeddings
 5. Feed the sequence as an input to a SOTA transformer encoder
 6. Pre-train the ViT model with image labels, then fully supervised on a big dataset
- Fine-tune the downstream dataset for image classification

Real-World ViT Use Cases and Applications

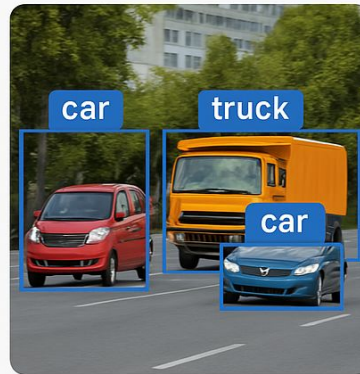
- **Image Classification:** ViT has shown excellent performance on large-scale image classification tasks like ImageNet.
- **Object Detection:** Transformer-based models like **DETR** (Detection Transformer) are being used for object detection tasks, improving accuracy by capturing long-range dependencies between objects in images.
- **Image Segmentation:** Transformers are also being used in medical imaging for tasks like tumor segmentation.

Real-World ViT Use Cases and Applications

Applications of Vision Transformers



Image
Classification



Object
Detection



Segmentation

Real-World ViT Use Cases and Applications



Real-World ViT Use Cases and Applications



ViT vs. CNN

Feature	Vision Transformer (ViT)	Convolutional Neural Network (CNN)
Feature Extraction	Global, using non-overlapping image patches	Local, through convolutional filters
Architecture	Based on transformers	Built around convolutional layers
Data Requirement	Requires more data to perform optimally	Less data-intensive, can perform well on smaller datasets
Computational Cost	Higher, due to the need for large-scale pre-training	Relatively lower
Use Case Suitability	Excels in tasks requiring an understanding of global patterns	Ideal for tasks where local features are significant

Strengths of Vision Transformer Architecture

- **Global Context:** ViTs are excellent at capturing long-range dependencies between image patches, giving them a better understanding of the global context of images.
- **Scalability:** ViTs scale well with larger datasets and deeper architectures, making them highly effective for large-scale vision tasks.
- **Flexibility:** Since ViTs don't rely on convolutions, they can be easily adapted to different tasks and domains beyond image classification, such as video analysis and object detection.

Challenges and Limitations

- **Data Requirements:** ViTs are data-hungry and typically require large-scale datasets for effective training. Fine-tuning ViTs pre-trained on large datasets (like ImageNet) is a common practice when data is limited.
- **Computational Resources:** Vision Transformers demand significant computational power, especially during training, due to the quadratic complexity of the self-attention mechanism.

Conclusion

- **Transformers have redefined deep learning**, starting from NLP and now reshaping the landscape of computer vision through Vision Transformers (ViTs).
- **ViTs demonstrate that attention mechanisms**, when applied to visual data, can match or exceed the performance of traditional Convolutional Neural Networks (CNNs), particularly when trained on large-scale datasets.
- Despite their **computational demands and data requirements**, ViTs offer a flexible, scalable, and architecture-agnostic framework that opens new frontiers across modalities.
- Ongoing research into **data-efficient training, hybrid models, and lightweight ViTs** is rapidly addressing their current limitations, making them increasingly viable for real-world applications.
- As deep learning continues to evolve, **transformers are poised to become a unifying model across domains**, enabling more robust, generalizable, and intelligent AI systems.



The End